

Towards Understanding Asynchronous Advantage Actor-critic: Convergence and Linear Speedup

Han Shen

Rensselaer Polytechnic Institute

SHENH5@RPI.EDU

Kaiqing Zhang

Massachusetts Institute of Technology

KAIQING@MIT.EDU

Mingyi Hong

University of Minnesota, Minneapolis

MHONG@UMN.EDU

Tianyi Chen

Rensselaer Polytechnic Institute

CHENT18@RPI.EDU

Abstract

Asynchronous and parallel implementation of standard reinforcement learning (RL) algorithms is a key enabler of the tremendous success of modern RL. Among many asynchronous RL algorithms, arguably the most popular and effective one is the asynchronous advantage actor-critic (A3C) algorithm. Although A3C is becoming the workhorse of RL, its theoretical properties are still not well-understood, including its non-asymptotic analysis and the performance gain of parallelism (a.k.a. linear speedup). This paper revisits the A3C algorithm and establishes its non-asymptotic convergence guarantees. Under both i.i.d. and Markovian sampling, we establish the local convergence guarantee for A3C in the general policy approximation case and the global convergence guarantee in softmax policy parameterization. Under i.i.d. sampling, A3C obtains sample complexity of $\mathcal{O}(\epsilon^{-2.5}/N)$ per worker to achieve ϵ accuracy, where N is the number of workers. Compared to the best-known sample complexity of $\mathcal{O}(\epsilon^{-2.5})$ for two-timescale AC, A3C achieves *linear speedup*, which justifies the advantage of parallelism and asynchrony in AC algorithms theoretically for the first time. Numerical tests on synthetic environment, OpenAI Gym environments and Atari games have been provided to verify our theoretical analysis.

1. Introduction

Reinforcement learning (RL) has achieved impressive performance in many domains such as robotics (Lillicrap et al., 2016; Mnih et al., 2015) and video games (Mnih et al., 2016). However, these empirical successes are often at the expense of significant computation. To unlock high computation capabilities, the state-of-the-art RL approaches rely on sampling data from massive parallel simulators on multiple machines (Nair et al., 2015; Mnih et al., 2016; Assran et al., 2019; Chen et al., 2018). Empirically, these approaches can significantly *reduce training time* when implemented in an *asynchronous* manner. One popular method that achieves the state-of-art performance is the asynchronous variant of the actor-critic (AC) algorithm, referred to as A3C (Mnih et al., 2016).

A3C builds on the original AC algorithm (Konda, 2002). At a high level, AC simultaneously performs policy optimization (a.k.a. the actor step) using the policy gradient (PG) method (Sutton et al., 2000) and policy evaluation (a.k.a. the critic step) using the temporal difference learning (TD) algorithm (Sutton, 1988). To ensure scalability to large state-action spaces, both actor and critic steps can combine with various function approximation techniques. To ensure stability, AC is often implemented in a two time-scale fashion, where the actor step runs in the slow timescale and the

critic step runs in the fast timescale. Similar to other on-policy RL algorithms, AC uses samples generated from the target policy. Thus, data sampling is entangled with the learning procedure, which generates significant *overhead*. To speed up the sampling process of AC, A3C introduces multiple workers with a shared policy, and each worker has its own simulator to perform data sampling. The shared policy can be then updated using samples collected from multiple workers.

Despite the empirical success achieved by A3C, to the best of our knowledge, its theoretical property is not well-understood. The following *theoretical* questions remain unclear: **Q1)** Under what assumption does A3C converge? If so, does it converge to the global optimal solution? **Q2)** What is its convergence rate? **Q3)** Can A3C obtain benefit (or linear speedup) using parallelism and asynchrony?

For **Q3)**, we are interested in the *training time linear speedup* with N workers, which is the ratio between the training time using a single worker and that using N workers. Since asynchronous parallelism mitigates the effect of stragglers and keeps workers busy, the training time speedup can be measured roughly by the sample complexity (i.e., computational) linear speedup (Lian et al., 2016):

$$\text{Speedup}(N) = \frac{\text{sample complexity with one worker}}{\text{average sample complexity per worker with } N \text{ workers}}. \quad (1)$$

If $\text{Speedup}(N) = \Theta(N)$, the speedup is linear, and the training time roughly reduces linearly as the number of workers increases. This paper aims to answer this question, towards the goal of providing theoretical justification for the empirical successes of parallel and asynchronous RL.

1.1. Related works

The PG method and its global convergence. The global optimality of the stationary points of policy optimization problems has been shown in (Bhandari and Russo, 2019). Then the finite-time convergence rate for exact PG method with softmax policy was established in (Agarwal et al., 2020) by utilizing a gradient-dominance type result under relative entropy regularized objective function. Later, (Mei et al., 2020) extended this result to the entropy regularized setting and established linear convergence rate for exact PG method under softmax parameterization. Later, (Bhandari and Russo, 2021) has proved linear convergence rate for general class PG methods. In the stochastic setting, (Zhang et al., 2019) has established local optimal convergence for stochastic PG with unbiased rollout and increasing step sizes, and (Wang et al., 2019) has established the global convergence of stochastic neural PG with increasing batch of i.i.d. samples. Later, (Zhang et al., 2021) proved that the minibatch version of PG achieves global convergence with the help of relative entropy regularization. But none of them consider the global convergence of the AC method. On the application side, the PG method has been broadly applied in various settings; see e.g. (Chai and Lau, 2019, 2020; El-Laham and Bugallo, 2021; Cervino et al., 2021). In (Chai and Lau, 2019), the actor critic method was used to jointly optimize the trajectory, transmission and caching content delivery of the unmanned aerial vehicles. In (Chai and Lau, 2020), the policy gradient method was used to jointly optimize the streaming rate and transmission power. In (El-Laham and Bugallo, 2021), the PG method was used to help the learning of a distribution adaptation strategy. In (Cervino et al., 2021), the PG method is used in a multitask learning algorithm which seeks to improve generalization to new tasks.

Analysis of AC algorithm. AC method was first proposed by (Borkar and Konda, 1997; Konda, 2002), with asymptotic convergence guarantees provided in (Borkar and Konda, 1997; Konda, 2002; Bhatnagar et al., 2009). It was not until recently that the *non-asymptotic* analyses of AC have

been established. The finite-sample guarantee for the batch AC algorithm has been established in (Yang et al., 2018; Kumar et al., 2019; Fu et al., 2020) with i.i.d. sampling. Later, in (Qiu et al., 2019), the finite-sample analysis was established for the double-loop nested AC algorithm under the Markovian setting. An improved analysis for the Markovian setting with minibatch updates has been presented in (Xu et al., 2020a) for the nested AC method. More recently, (Xu et al., 2020b; Wu et al., 2020) have provided the first finite-time analyses for the two-timescale AC algorithms under Markov sampling, with both $\tilde{O}(\epsilon^{-2.5})$ sample complexity, which is the best-known sample complexity for two-timescale AC. Through the lens of bi-level optimization, (Hong et al., 2020) has provided finite-sample guarantees for two-timescale AC, when a *natural* policy gradient step is used in the actor. Recently, (Fu et al., 2020) also analyzed the single-timescale AC algorithm under an exact critic oracle. On a less relevant line of research, AC-based multi-agent RL has been studied in (Zhang et al., 2018; Christianos et al., 2020; Qu et al., 2020). However, none of the existing works has analyzed the effect of the asynchronous and parallel updates in AC.

Parallel and distributed RL methods. In (Mnih et al., 2016), the original A3C method was proposed and became the workhorse in empirical RL. Later, (Babaeizadeh et al., 2017) has provided a GPU-version of A3C which significantly decreases training time. Recently, the A3C algorithm is further optimized in modern computers by (Stooke and Abbeel, 2019), where a large batch variant of A3C with improved efficiency is also proposed. In (Espeholt et al., 2018), an importance weighted distributed AC algorithm IMPALA has been developed to solve a collection of problems with one single set of parameters. A gossip-based distributed AC algorithm has been proposed in (Assran et al., 2019) which achieves performance competitive to A3C. Additionally, distributed RL is closely related to the multi-agent RL, both of which have a broad range of applications (Kar et al., 2013; Sadeghi et al., 2018; Wu et al., 2021). In (Kar et al., 2013), a distributed algorithm based on Q-learning was proposed and was shown to achieve convergence under a sparse communication network. In (Sadeghi et al., 2018), an asynchronous caching approach which utilized PG to find an optimal caching policy was developed. A robust decentralized TD learning method was proposed in (Wu et al., 2021) to defend against malicious agents in a multi-agent network.

Asynchronous stochastic optimization. For solving general optimization problems, asynchronous stochastic methods have received much attention recently. Due to the possible speedup that can be achieved by asynchronous optimization, it has also been extensively applied to various machine learning areas including RL (Mnih et al., 2016; Sadeghi et al., 2018) and distributed learning (Wu et al., 2018). The study of asynchronous stochastic methods can be traced back to 1980s (Bertsekas and Tsitsiklis, 1989). With the batch size M , (Agarwal and Duchi, 2011) analyzed asynchronous SGD (async-SGD) for convex functions, and derived a convergence rate of $\mathcal{O}(K^{-\frac{1}{2}} M^{-\frac{1}{2}})$ if delay K_0 is bounded by $\mathcal{O}(K^{\frac{1}{4}} M^{-\frac{3}{4}})$. This result implies linear speedup. (Feyzmahdavian et al., 2015) extended the analysis of (Agarwal and Duchi, 2011) to smooth convex with nonsmooth regularization and derived a similar rate. Recent studies by (Lian et al., 2016) improved upper bound of K_0 to $\mathcal{O}(K^{\frac{1}{2}} M^{-\frac{1}{2}})$. However, all these works have focused on the single-timescale SGD with a single variable, which cannot capture the stochastic recursion of the AC and A3C algorithms. To best of our knowledge, non-asymptotic analysis of asynchronous two-timescale SGD has remained unaddressed, and its speedup analysis is an uncharted territory.

1.2. This work

In this context, we revisit A3C with TD(0) for the critic update. The goal is to provide *non-asymptotic* guarantee and *linear speedup* justification for this popular algorithm.

Our contributions. Compared to the existing literature on both the AC algorithms and the async-SGD, our contributions can be summarized as follows.

c1) We revisit two-timescale A3C and establish its convergence rates with both i.i.d. and Markovian sampling. We first proves the local convergence rate for A3C in the general function approximation case, and then proves that A3C achieves global convergence for the softmax policy parameterization. To the best of our knowledge, this is the first non-asymptotic convergence result for *asynchronous parallel* AC algorithms.

c2) We characterize the sample complexity of A3C. In the i.i.d. setting, A3C achieves a sample complexity of $\mathcal{O}(\epsilon^{-2.5}/N)$ per worker, where N is the number of workers. Compared to the best-known complexity of $\mathcal{O}(\epsilon^{-2.5})$ for i.i.d. two-timescale AC (Hong et al., 2020), A3C achieves *linear speedup*, thanks to the parallelism and asynchrony. In the Markovian setting, if delay is bounded, the sample complexity of A3C matches the order of the non-parallel AC algorithm (Wu et al., 2020).

c3) We test A3C on a synthetic environment to verify our theoretical guarantees with both i.i.d. and Markovian sampling. We also test A3C on the classic control tasks and Atari Games.

Technical challenges. Compared to the recent analysis of nonparallel two-timescale AC in (Wu et al., 2020; Xu et al., 2020b; Hong et al., 2020), several new challenges arise due to the parallelism and asynchrony.

Markovian noise coupled with asynchrony and delay. The analysis of two-timescale AC algorithm is non-trivial because of the Markovian noise coupled with both the actor and critic steps. Different from the nonparallel AC that only involves a single Markov chain, A3C introduces multiple Markov chains (one per worker) that mix at different speeds. This is because at a given iteration, workers collect different number of samples and thus their chains mix to different degrees. As we will show later, the worker with the slowest mixing chain will determine the convergence.

Linear speedup for SGD with two coupled sequences. Parallel async-SGD has been shown to achieve linear speedup recently (Lian et al., 2016; Sun et al., 2017). Different from async-SGD, asynchronous AC is a two-timescale stochastic *semi-gradient* algorithm for solving the more challenging *bilevel* optimization problem (see (Hong et al., 2020)). The errors induced by asynchrony and delay are intertwined with both the actor and critic updates via a nested structure, which makes the sharp analysis more challenging. Our linear speedup analysis should be also distinguished from that of mini-batch async-SGD (Lian et al., 2017), where the speedup is a result of *variance reduction* thanks to the larger batch size generated by parallel workers.

2. Preliminaries

2.1. Markov decision process and policy gradient

A Markov decision process (MDP) can be described by $\mathcal{M} = \{\mathcal{S}, \mathcal{A}, \mathcal{P}, R, \gamma\}$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $\mathcal{P}(s'|s, a)$ is the probability of transitioning to $s' \in \mathcal{S}$ from state $s \in \mathcal{S}$ and action $a \in \mathcal{A}$, $r(s, a, s')$ is the reward associated with the transition (s, a, s') , and $\gamma \in [0, 1)$ is a discount factor. Throughout the paper, we assume the reward r is upper-bounded by a constant $r_{\max}$. A policy $\pi : \mathcal{S} \rightarrow \Delta(\mathcal{A})$ is defined as a mapping from the state space $\mathcal{S}$ to the probability distribution over the action space $\mathcal{A}$.

Considering discrete time t in an infinite horizon, a policy π can generate a trajectory $(s_0, a_0, \dots)$ with $a_t \sim \pi(\cdot|s_t)$ and $s_{t+1} \sim \mathcal{P}(\cdot|s_t, a_t)$. Given a policy π , we define the state and state action value

functions as

$$\begin{aligned} V_\pi(s) &:= \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t, s_{t+1}) \mid s_0 = s \right], \\ Q_\pi(s, a) &:= \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t, s_{t+1}) \mid s_0 = s, a_0 = a \right] \end{aligned} \quad (2)$$

where $\mathbb{E}$ is taken over the trajectory $(s_0, a_0, s_1, a_1, \dots)$ generated under policy π . With the above definitions, the advantage function is $A_\pi(s, a) := Q_\pi(s, a) - V_\pi(s)$. With η denoting the initial state distribution, the discounted state visitation measure induced by policy π is defined as $d_\pi(s) := (1 - \gamma) \sum_{t=0}^{\infty} \gamma^t \mathbb{P}(s_t = s \mid s_0 \sim \eta, \pi)$. We also overload the notation and define the state-action visitation distribution $d_\pi(s, a) = (1 - \gamma) \sum_{t=0}^{\infty} \gamma^t \mathbb{P}(s_t = s \mid s_0 \sim \eta, \pi) \pi(a|s)$. In the case where π is parameterized by θ , we use d_θ as shorthand notations for d_{π_θ} .

The goal of RL is to find an optimal policy π^* defined as $\pi^* \in \arg \max_\pi J(\pi) := (1 - \gamma) \mathbb{E}_{s \sim \eta} [V_\pi(s)]$, with the optimal return defined as $J^* := \max_\pi J(\pi)$. When the state and action spaces are large, finding the optimal policy π becomes computationally intractable. To overcome the inherent difficulty of learning a function, the policy gradient methods search the best performing policy over a class of parameterized policies. We parameterize the policy with parameter $\theta \in \mathbb{R}^d$, and solve the optimization problem as

$$\max_{\theta \in \mathbb{R}^d} J(\theta) \quad \text{with} \quad J(\theta) := (1 - \gamma) \mathbb{E}_{s \sim \eta} [V_{\pi_\theta}(s)]. \quad (3)$$

To maximize $J(\theta)$ with respect to θ , one can update θ using the policy gradient (Sutton et al., 2000)

$$\nabla J(\theta) = \mathbb{E}_{s, a \sim d_\theta} [A_{\pi_\theta}(s, a) \psi_\theta(s, a)], \quad (4)$$

where $\psi_\theta(s, a) := \nabla \log \pi_\theta(a|s)$. Since computing $\mathbb{E}$ in (4) is expensive if not impossible, popular policy gradient-based algorithms iteratively update θ using stochastic estimate of (4) such as REINFORCE (Williams, 1992) and G(PO)MDP (Baxter and Bartlett, 2001).

It is also a common practice to adopt regularization and augment the objective function to

$$J_\lambda(\theta) := J(\theta) - \lambda \mathbb{E}_{s \sim \eta_p} [D_{KL}(\pi_p(\cdot|s) \parallel \pi_\theta(\cdot|s))] \quad (5)$$

with a regularization constant $\lambda \geq 0$. Here η_p is a prior distribution of states, π_p is a prior policy. The regularization term encourages π_θ to imitate π_p , incorporating prior knowledge into training process. When π_p and η_p are set as uniform distributions, the regularization term is reduced to the relative-entropy regularization widely analyzed in the literature (Agarwal et al., 2020; Bhandari and Russo, 2019; Zhang et al., 2021). Moreover, the regularization prevents degenerate solutions that can lead to the pitfall of certain policy parametrization (Bhandari and Russo, 2019). Given π_p and η_p , we use $R(\theta)$ as a shorthand notation of $-\mathbb{E}_{s \sim \eta_p} [D_{KL}(\pi_p(\cdot|s) \parallel \pi_\theta(\cdot|s))]$.

2.2. Actor-critic with value function approximation

Both REINFORCE and G(PO)MDP-based policy gradient algorithms rely on a Monte-Carlo estimate of the value function $V_{\pi_\theta}(s)$ and thus $\nabla J(\theta)$ by generating a trajectory per iteration. However, policy gradient methods based on Monte-Carlo estimate typically suffer from high variance and large

sampling cost. An alternative way is to recursively refine the estimate of $V_{\pi_\theta}(s)$. For a policy π_θ , it is known that $V_{\pi_\theta}(s)$ satisfies the Bellman equation (Sutton and Barto, 2018), that is

$$V_{\pi_\theta}(s) = \mathbb{E}_{a \sim \pi_\theta(\cdot|s), s' \sim \mathcal{P}(\cdot|s,a)} [r(s, a, s') + \gamma V_{\pi_\theta}(s')]. \quad (6)$$

In practice, when the state space $\mathcal{S}$ is prohibitively large, one cannot afford the computational and memory complexity of computing $V_{\pi_\theta}(s)$ and $A_{\pi_\theta}(s, a)$. To overcome this curse-of-dimensionality, a popular method is to approximate the value function using function approximation techniques. Given the state feature mapping $\phi(\cdot) : \mathcal{S} \rightarrow \mathbb{R}^{d'}$ for some $d' > 0$, we approximate the value function linearly as $V_{\pi_\theta}(s) \approx \hat{V}_\omega(s) := \phi(s)^\top \omega$, where $\omega \in \mathbb{R}^{d'}$ is the critic parameter.

Given π_θ , the task of finding the best ω such that $V_{\pi_\theta}(s) \approx \hat{V}_\omega(s)$ is usually addressed by TD learning (Sutton, 1988). Given π_θ , the task of finding the best ω such that $V_{\pi_\theta}(s) \approx \hat{V}_\omega(s)$ is usually addressed by TD learning (Sutton, 1988). Formally, we first define

$$A_{\theta,\phi} := \mathbb{E}_{s \sim \mu_{\pi_\theta}, s' \sim \mathcal{P}_{\pi_\theta}} [\phi(s)(\gamma\phi(s') - \phi(s))^\top], \quad (7a)$$

$$b_{\theta,\phi} := \mathbb{E}_{s \sim \mu_{\pi_\theta}, a \sim \pi_\theta} [r(s, a, s')\phi(s)]. \quad (7b)$$

where $\mathcal{P}_{\pi_\theta}(s'|s) := \sum_a \mathcal{P}(s'|s, a)\pi_\theta(a|s)$, and μ_{π_θ} is the stationary distribution of the Markov chain with transition distribution $\mathcal{P}$ and policy π_θ . Then given a policy π_θ , the *exact* TD update takes the following form:

$$\omega_{k+1} = \omega_k + \beta(A_{\theta,\phi}\omega_k + b_{\theta,\phi}). \quad (8)$$

When analyzing TD, the following standard assumption is often made:

Assumption 1 *For all $s \in \mathcal{S}$, the feature vector $\phi(s)$ is normalized so that $\|\phi(s)\|_2 \leq 1$. For all eligible θ , the symmetric part of $A_{\theta,\phi}$, denoted as $(A_{\theta,\phi} + A_{\theta,\phi}^\top)/2$, is negative definite and has a largest eigenvalue upper bounded by $-\lambda$.*

Assumption 1 is common in analyzing TD with linear function approximation; see e.g., (Bhandari et al., 2018; Wu et al., 2020, 2021). In fact, as shown in (Bhandari et al., 2018), when redundant features are removed such that the feature covariance matrix is full-rank, this assumption is satisfied. With this assumption, $A_{\theta,\phi}$ is full-rank, thus the update in (8) admits a unique stationary point $\omega^*(\theta) = -A_{\theta,\phi}^{-1}b_{\theta,\phi}$. Moreover, there exists a constant $R_\omega := \frac{r_{\max}}{\sqrt{\lambda}(1-\gamma)^{\frac{3}{2}}}$ such that $\|\omega^*(\theta)\|_2 \leq R_\omega$.

We often use the stochastic approximation of the TD update in (8). With k th transition defined as $x_k := (s_k, a_k, s_{k+1})$, the corresponding TD target is

$$\hat{\delta}(x_k, \omega_k) := r(s_k, a_k, s_{k+1}) + \gamma\phi(s_{k+1})^\top \omega_k - \phi(s_k)^\top \omega_k \quad (9)$$

and the critic gradient $g(x_k, \omega_k) := \hat{\delta}(x_k, \omega_k)\nabla\hat{V}_{\omega_k}(s_k)$. We update the parameter ω via

$$\omega_{k+1} = \Pi_{R_\omega}(\omega_k + \beta g(x_k, \omega_k)), \quad (10)$$

where β is the critic step size, and Π_{R_ω} is a projection operator that projects a vector to a l_2 norm ball with radius R_ω . The projection step is often used to control the norm of gradient. In AC, it prevents the actor and critic updates from going too far in the ‘wrong’ direction; see e.g., (Konda, 2002; Wu et al., 2020; Xu et al., 2020b; Zou et al., 2019).

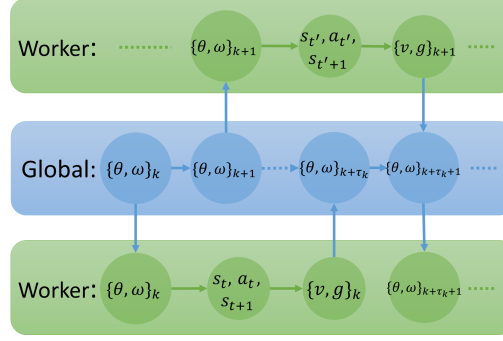


Figure 1: Implementation of A3C with two workers.

Using the definition that $A_{\pi_\theta}(s, a) = \mathbb{E}_{s' \sim \mathcal{P}}[r(s, a, s') + \gamma V_{\pi_\theta}(s')] - V_{\pi_\theta}(s)$, we can also rewrite (4) as $\nabla J(\theta) = \mathbb{E}_{s, a \sim d_\theta, s' \sim \mathcal{P}}[(r(s, a, s') + \gamma V_{\pi_\theta}(s') - V_{\pi_\theta}(s)) \psi_\theta(s, a)]$. Leveraging the value function approximation, we can then approximate the regularized policy gradient as

$$\widehat{\nabla} J_\lambda(\theta) = \widehat{\nabla} J(\theta) + \lambda \widehat{\nabla} R(\theta) = \underbrace{\widehat{\delta}(x, \omega) \psi_\theta(s, a)}_{v(x, \theta, \omega)} + \lambda \psi_\theta(x^p). \quad (11)$$

where $v(x, \theta, \omega)$ is an estimator of $\nabla J(\theta)$, and $x^p := (s^p \sim \eta_p, a^p \sim \pi_p(\cdot | s_p))$. Then it is easy to check that $\psi_\theta(x^p)$ is an unbiased estimator of $\nabla R(\theta)$. This gives rise to the policy update

$$\theta_{k+1} = \theta_k + \alpha (v(x_k, \theta_k, \omega_k) + \lambda \psi_\theta(x^p)), \quad (12)$$

where α is the stepsize for the actor update. To ensure convergence when simultaneously performing critic and actor updates, the stepsizes α and β often decay at two different rates, which is referred to the two-timescale AC (Konda, 2002; Wu et al., 2020).

3. A3C Implementation

To speed up the training process, AC can be implemented over N workers in a shared memory setting *without* coordinating among workers (Mnih et al., 2016). Each worker has its own simulator to perform sampling, and then collaboratively updates the shared policy π_θ using AC updates. As there is no synchronization after each update, the policy used by workers to generate samples may be outdated, which introduces staleness.

Notations on samples. Subscription t in x_t and x_t^p indicates the sample is generated in t th local iteration of a worker. When Markovian sampling is used, subscription t in $x_t = (s_t, a_t, s_{t+1})$ also indicates that it is the t th transition of the local Markov chain. We use k to denote the global counter (or iteration), which increases by one whenever a worker finishes the actor and critic updates in the shared memory. We use subscription (k) in $(s_{(k)}, a_{(k)}, s'_{(k)})$ and $(s_{(k)}^p, a_{(k)}^p)$ to indicate the samples used in the k th update.

Algorithm flow. Specifically, we initialize θ_0, ω_0 in the shared memory. Each worker will initialize the simulator with initial state s_0 . Without coordination, workers will load θ, ω in the shared memory. The worker then generates samples with either i.i.d. or Markovian sampling method. In Markovian sampling case, we maintain separate Markov chains for actor and critic. For critic, we

Algorithm 1 A3C: each worker’s view.

```

1: Global initialize: Global counter  $k=0$ , initial  $\theta_0, \omega_0$  in the shared memory.
2: Worker initialize: Counter  $t=0$ . Sample  $s_0 \sim \eta, \hat{s}_0 \sim \eta$ .
3: for  $t = 0, 1, 2, \dots$  do
4:   Read  $\theta, \omega$  in the shared memory.
5:   option 1 (i.i.d. sampling):
6:      $x_t = (s_t \sim \mu_{\pi_{\theta_t}}, a_t \sim \pi_{\theta_t}(\cdot|s_t), s'_t \sim \mathcal{P}(\cdot|s_t, a_t))$ .
7:      $\hat{x}_t = (\hat{s}_t \sim d_{\pi_{\theta_t}}, \hat{a}_t \sim \pi_{\theta_t}(\cdot|\hat{s}_t), \hat{s}'_t \sim \mathcal{P}(\cdot|\hat{s}_t, \hat{a}_t))$ .
8:   option 2 (Markovian sampling):
9:      $x_t = (s_t, a_t \sim \pi_{\theta}(\cdot|s_t), s_{t+1} \sim \mathcal{P}(\cdot|s_t, a_t))$ .
10:     $\hat{x}_t = (\hat{s}_t, \hat{a}_t \sim \pi_{\theta}(\cdot|\hat{s}_t), \hat{s}'_{t+1} \sim \mathcal{P}(\cdot|\hat{s}_t, \hat{a}_t))$ .
11:    With probability  $\gamma$ :  $\hat{s}_{t+1} = \hat{s}'_{t+1}$ ; Otherwise:  $\hat{s}_{t+1} \sim \eta$ .
12:    Compute  $g(x_t, \omega) = \hat{\delta}(x_t, \omega) \nabla_{\omega} \hat{V}_{\omega}(s_t)$ .
13:    Compute  $v(\hat{x}_t, \theta, \omega) = \hat{\delta}(\hat{x}_t, \omega) \psi_{\theta}(\hat{s}_t, \hat{a}_t)$ .
14:    Compute  $\psi_{\theta}(x_t^p)$  with  $x_t^p = (s_t^p \sim \eta_p, a_t^p \sim \pi_p(\cdot|s_t^p))$ .
15:    In the shared memory, perform update (13).
16: end for
    
```

generate samples following the original transition kernel $\mathcal{P}$. While the actor’s chain can be viewed as evolving under a transition kernel $\hat{\mathcal{P}} = \gamma\mathcal{P} + (1 - \gamma)\eta$. At each iteration, we have a probability of $1 - \gamma$ to reset the chain, thus taking the initial state distribution into account. If the actor’s chain evolves under $\mathcal{P}$ like critic, asymptotically the initial distribution η is forgotten, which will introduce an asymptotic error. Once samples are obtained, each worker locally computes the gradients, and then updates the parameters in shared memory asynchronously by

$$\omega_{k+1} = \Pi_{R_{\omega}} (\omega_k + \beta g(x_{(k)}, \omega_{k-\tau_k})) \quad (13a)$$

$$\theta_{k+1} = \theta_k + \alpha (v(\hat{x}_{(k)}, \theta_{k-\tau_k}, \omega_{k-\tau_k}) + \lambda \psi_{\theta_{k-\tau_k}}(x_{(k)}^p)) \quad (13b)$$

where τ_k is the delay in the k th actor and critic updates. See A3C in Algorithm 1 and Figure 1.

Parallel sampling. The AC update (10) and (12) uses samples generated “on-the-fly” from the target policy π_{θ} , which brings overhead. Compared with (10) and (12), the A3C update (13) allows parallel sampling from N workers, which is the key to linear speedup. We consider the case where only one worker can update parameters in the shared memory at the same time and the update cannot be interrupted. In practice, (13) can also be performed in a mini-batch fashion.

Separate sampling protocols. In Algorithm 1, we maintain separate sampling protocols for actor and critic. This is due to the mismatch between the actor and critic sampling distribution. As indicated by (7), the desired sampling distribution of critic is $\mu_{\pi_{\theta}}$. The policy gradient (4) requires sampling from $d_{\pi_{\theta}}$. However, $d_{\pi_{\theta}}$ and $\mu_{\pi_{\theta}}$ are in general different, and the difference is non-diminishing. Therefore, if one uses the same samples for actor and critic, either the actor or the critic update will have a non-diminishing bias.

To mitigate the asymptotic bias, it is just natural to choose different sampling protocols for actor and critic. Our theoretical analysis justifies this choice by proving that such sampling method gives unbiased stochastic gradients asymptotically. We also provide experiments to demonstrate the superiority of the separated sampling methods.

4. Convergence Analysis of A3C

In this section, we analyze the convergence of A3C in both i.i.d. and Markovian settings. Throughout this section, $\mathcal{O}(\cdot)$ contains constants that are independent of N and K_0 .

To analyze the performance of A3C, we make the following assumptions.

Assumption 2 *There exists K_0 such that the delay at each iteration is bounded by $\tau_k \leq K_0, \forall k$.*

Assumption 2 ensures the viability of analyzing the asynchronous update; see the same assumption in e.g., (Lian et al., 2016; Assran et al., 2019; Wu et al., 2018). In practice, the delay usually scales as the number of workers, that is $K_0 = \Theta(N)$.

Assumption 3 *For any $\theta, \theta' \in \mathbb{R}^d$, $s \in \mathcal{S}$ and $a \in \mathcal{A}$, there exist constants C_ψ, L_ψ, L_π such that: i) $\|\psi_\theta(s, a)\|_2 \leq C_\psi$; ii) $\|\psi_\theta(s, a) - \psi_{\theta'}(s, a)\|_2 \leq L_\psi \|\theta - \theta'\|_2$; iii) $|\pi_\theta(a|s) - \pi_{\theta'}(a|s)| \leq L_\pi \|\theta - \theta'\|_2$.*

Assumption 3 is common in analyzing policy gradient-type algorithms which has also been made by e.g., (Agarwal et al., 2020; Zhang et al., 2019). This assumption holds for many policy parameterization methods such as tabular softmax policy (Agarwal et al., 2020), Gaussian policy (Doya, 2000) and Boltzmann policy (Konda and Borkar, 1999).

Assumption 4 *For any θ , assume the Markov chains with transition kernels $\mathcal{P}$ and $\hat{\mathcal{P}}$ are irreducible and aperiodic under policy π_θ . Then there exist constants $\kappa > 0$ and $\rho \in (0, 1)$ such that*

$$\sup_{s \in \mathcal{S}} d_{TV}(\mathbb{P}(s_t \in \cdot | s_0 = s, \pi_\theta), \mu_{\pi_\theta}) \leq \kappa \rho^t, \quad (14a)$$

and

$$\sup_{s \in \mathcal{S}} d_{TV}(\mathbb{P}(\hat{s}_t \in \cdot | \hat{s}_0 = s, \pi_\theta), d_{\pi_\theta}) \leq \kappa \rho^t. \quad (14b)$$

where s_t is the t th state of the Markov chain with transition kernel $\mathcal{P}$, and $\hat{s}_t$ is the t th state of the Markov chain with transition kernel $\hat{\mathcal{P}}$.

Assumption 4 assumes the Markov chain mixes at a geometric rate. This assumption has also been made by other analysis on Markovian sampling; see e.g. (Bhandari et al., 2018; Wu et al., 2020). It is worth noting that the second part of our assumption, that is (14b), holds as long as $\gamma < 1$.

We define the critic approximation error as

$$\epsilon_{\text{app}} := \max_{\theta \in \mathbb{R}^d} \sqrt{\mathbb{E}_{s \sim \mu_\theta} |V_{\pi_\theta}(s) - \hat{V}_{\omega_\theta^*}(s)|^2} \quad (15)$$

where μ_θ is the stationary distribution under π_θ and $\mathcal{P}$. This error captures the quality of the critic function approximation; see also (Qiu et al., 2019; Wu et al., 2020, 2021). When the MDP is tabular and the feature matrix is full-rank, the value function V_{π_θ} is in the span of the features. In this case, we have $\epsilon_{\text{app}} = 0$.

We first give the convergence result of critic update.

Theorem 1 (Critic convergence) *Suppose Assumptions 1–4 hold. Consider Algorithm 1 with i.i.d. sampling and $\hat{V}_\omega(s) = \phi(s)^\top \omega$. Select step size $\alpha = K^{-\frac{3}{5}}$ and $\beta = K^{-\frac{2}{5}}$. Then it holds that*

$$\frac{1}{K} \sum_{k=1}^K \mathbb{E} \|\omega_k - \omega_{\theta_k}^*\|_2^2 = \mathcal{O}\left(\frac{K_0^2}{K^{\frac{4}{5}}}\right) + \mathcal{O}\left(\frac{K_0}{K^{\frac{3}{5}}}\right) + \mathcal{O}\left(\frac{1}{K^{\frac{2}{5}}}\right). \quad (16)$$

Given the critic convergence, we can present the convergence result of actor update.

Theorem 2 (Actor convergence) *Under the same assumptions of Theorem 1, select step size $\alpha = K^{-\frac{3}{5}}$ and $\beta = K^{-\frac{2}{5}}$. Then it holds that*

$$\frac{1}{K} \sum_{k=1}^K \mathbb{E} \|\nabla J_\lambda(\theta_k)\|_2^2 = \mathcal{O}\left(\frac{1}{K^{\frac{2}{5}}}\right) + \mathcal{O}\left(\frac{K_0^2}{K^{\frac{4}{5}}}\right) + \mathcal{O}\left(\frac{K_0}{K^{\frac{3}{5}}}\right) + \mathcal{O}(\epsilon_{\text{app}}). \quad (17)$$

If $K_0 = \Theta(N) = \mathcal{O}(K^{\frac{1}{5}})$, then it holds that

$$\frac{1}{K} \sum_{k=1}^K \mathbb{E} \|\nabla J_\lambda(\theta_k)\|_2^2 = \mathcal{O}(K^{-\frac{2}{5}}) + \mathcal{O}(\epsilon_{\text{app}}) \quad (18)$$

where $\mathcal{O}(\cdot)$ contains constants independent of N and K_0 .

Corollary 1 (Linear speedup) *To reach ϵ -accuracy in (18), the required number of iterations is $\mathcal{O}(\epsilon^{-2.5})$. Since each iteration of A3C only uses one sample (one transition), the sample complexity is $\mathcal{O}(\epsilon^{-2.5})$, which matches the state-of-the-art sample complexity of two-timescale AC running on one worker. Then under A3C, the average sample complexity per worker is $\mathcal{O}(\epsilon^{-2.5}/N)$ which indicates linear speedup in (1). The negative effect of parameter staleness introduced by parallel asynchrony vanishes asymptotically with the step size. Vanished staleness allows for parallel computing from workers to speedup the training process.*

Remark 1 (Comparison to async-SGD analysis) *Different from async-SGD (e.g., (Lian et al., 2016)), the optimal critic parameter ω_θ^* is constantly drifting as θ changes at each iteration. This necessitates setting the actor update to be at a faster time scale than the critic. In this sense, the policy is static relative to the critic asymptotically. In actor update, the gradient $v(x, \theta, \omega)$ is biased because of inexact value function. The bias introduced by the critic optimality gap and the function approximation error correspond to the last two terms in (17).*

4.1. Convergence result with Markovian sampling

Theorem 3 (Critic convergence) *Suppose Assumptions 1–4 hold. Consider Algorithm 1 with Markovian sampling and $\hat{V}_\omega(s) = \phi(s)^\top \omega$. Select step size $\alpha = K^{-\frac{3}{5}}$ and $\beta = K^{-\frac{2}{5}}$. Then it holds that*

$$\frac{1}{K} \sum_{k=1}^K \mathbb{E} \|\omega_k - \omega_k^*\|_2^2 = \mathcal{O}\left(\frac{1}{K^{\frac{2}{5}}}\right) + \mathcal{O}\left(\frac{K_0^2 \log^2 K}{K^{\frac{3}{5}}}\right) + \mathcal{O}\left(\frac{K_0 \log K}{K^{\frac{2}{5}}}\right). \quad (19)$$

The following theorem gives the convergence rate of actor update in Algorithm 1.

Theorem 4 (Actor convergence) *Under the same assumptions of Theorem 3, select step size $\alpha = K^{-\frac{3}{5}}$ and $\beta = K^{-\frac{2}{5}}$. Then it holds that*

$$\frac{1}{K} \sum_{k=1}^K \mathbb{E} \|\nabla J_\lambda(\theta_k)\|_2^2 = \mathcal{O}\left(\frac{K_0^2 \log^2 K}{K^{\frac{3}{5}}}\right) + \mathcal{O}\left(\frac{K_0 \log K}{K^{\frac{2}{5}}}\right) + \mathcal{O}(\epsilon_{\text{app}}). \quad (20)$$

If we further assume $K_0 = \Theta(N) = \mathcal{O}(K^{\frac{1}{5}})$. It holds that

$$\frac{1}{K} \sum_{k=1}^K \mathbb{E} \|\nabla J_\lambda(\theta_k)\|_2^2 = \tilde{\mathcal{O}}\left(K_0 K^{-\frac{2}{5}}\right) + \mathcal{O}(\epsilon_{\text{app}}) \quad (21)$$

where $\tilde{\mathcal{O}}(\cdot)$ hides constants and the logarithmic order of K .

Different from i.i.d. sampling, the stochastic gradients $g(x, \omega)$ and $v(x, \theta, \omega)$ are biased for Markovian sampling, and the bias decreases as the chain mixes. The mixing time corresponds to the logarithmic terms $\log K$ in (19) and (20). Because of asynchrony, at a given iteration, workers collect different number of samples and their chains mix to different degrees. The worker with the slowest mixing chain will determine the rate of convergence. The product of K_0 and $\log K$ in (19) and (20) appears due to the slowest mixing chain. As the last term in (19) dominates other terms asymptotically, the convergence rate degrades as the number of workers increases. While the theoretical linear speedup is difficult to establish in the Markovian setting, we will empirically test it in Section 5.

Remark 2 (Challenges compared to AC analysis) *Unlike synchronous AC, A3C introduces asynchrony and delay in both the actor and critic updates. At each iteration k , the delayed parameters will introduce extra error in $g(x, \omega_{k-\tau_k}) - g(x, \omega_k)$ and $v(x, \theta_{k-\tau_k}, \omega_{k-\tau_k}) - v(x, \theta_k, \omega_k)$. Furthermore, it also causes delays in sampling since samples are drawn from the delayed policy $\pi_{\theta_{k-\tau_k}}$ instead of π_{θ_k} . This delay will get amplified as every state on the Markov chain is generated by policies with different delays. At local counter t (t th transition on local Markov chain), we compare the chain transition in synchronous and asynchronous settings:*

$$\begin{aligned} \text{sync} : & s_t \xrightarrow{\theta_t} a_t \xrightarrow{\mathcal{P}} s_{t+1} \xrightarrow{\theta_{t+1}} a_{t+1} \cdots ; \\ \text{async} : & s_t \xrightarrow{\theta_{k-\tau_k}} a_t \xrightarrow{\mathcal{P}} s_{t+1} \xrightarrow{\theta_{k+d_t-\tau_k+d_t}} a_{t+1} \cdots \end{aligned}$$

where k is the global counter at which the local Markov chain takes t th transition, τ_k is the delay of policy used to generate t th local transition, d_t is the number of global updates between two local transitions. Clearly, the parameter delay makes the Markov chain more difficult to analyze.

4.2. Global convergence under structured problem

A3C is a gradient ascent type algorithm, thus can only achieve local convergence under a generally non-concave objective function $J_\lambda(\theta)$ w.r.t. θ . However, under some special structured problem, A3C can be shown to achieve global convergence. In this section, we consider the class of MDP which has finite state space and action space. Suppose the policy is parameterized by the softmax function:

$$\pi_\theta(a|s) = \frac{\exp(\theta_{s,a})}{\sum_{s,a} \exp(\theta_{s,a})} \quad (22)$$

where $\theta \in \mathbb{R}^{|S||A|}$ and $\theta_{s,a}$ is the policy parameter corresponds to pair (s, a) . The softmax policy class cannot represent deterministic policies with finite θ . To avoid driving θ to infinity, it is crucial to penalize the deterministic policies with the regularization term introduced in (5). To do so, we set

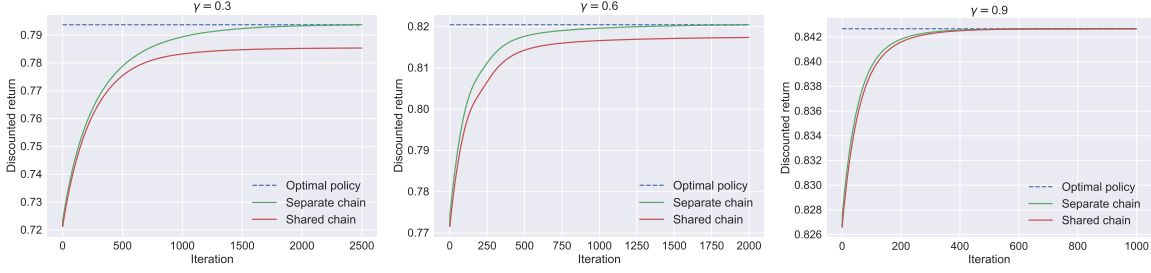


Figure 2: Algorithm 1 with separate chain sampling (option 2) vs shared chain sampling (setting $\hat{x}_t = x_t$ in the algorithm). The asymptotic error roughly scales proportionally to $1 - \gamma$. With a smaller γ , the objective function J becomes more shortsighted, and thus initial state distribution (restarting the chain) plays a more important role. If the actor shares the sample with critic, then a lack of chain restarting will introduce an unavoidable asymptotic error that grows larger as γ becomes smaller. Separate chain sampling works thanks to the random restarting with a probability scaling with γ .

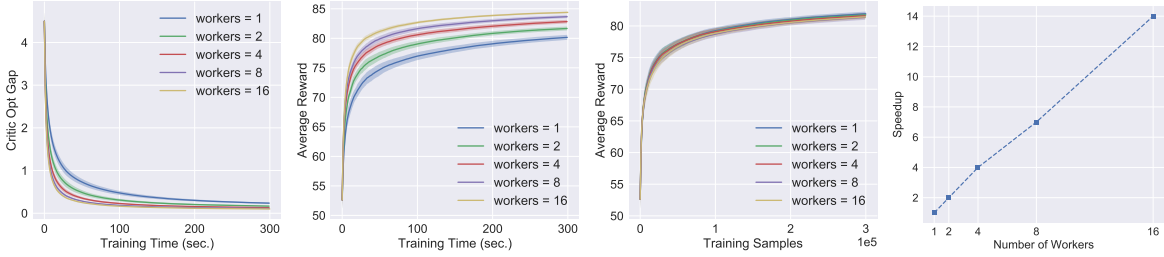


Figure 3: Convergence results of A3C with i.i.d. sampling in synthetic environment.

the priors η_p and π_p as uniform distribution on state and action space, then the objective function can be rewritten as

$$J_\lambda(\theta) = J(\theta) + \frac{\lambda}{|S||\mathcal{A}|} \sum_{s,a} \log \pi_\theta(a|s) + \lambda \log |\mathcal{A}|. \quad (23)$$

Define the state feature matrix $\Phi' := [\phi(s^1), \phi(s^2), \dots, \phi(s^{|S|})]^\top \in \mathbb{R}^{|S| \times d'}$ of which rows are features. We make the following assumption on Φ' .

Assumption 5 For any eligible θ , there exists $\omega_\theta \in \mathbb{R}^{d'}$ such that $\Phi' \omega_\theta = V_{\pi_\theta}$.

This assumption assumes that the value function V_{π_θ} can be accurately approximated by linear functions. For the assumption to hold, it suffices to select a squared full-rank feature matrix Φ' . It is worth noting that when this assumption does not hold, our result in Theorem 5 holds with an extra error term, which is the function approximation error ϵ_{app} .

To establish global convergence, a gradient-dominance type condition was proven in (Agarwal et al., 2020):

Lemma 1 With softmax policy parameterization and uniform priors, if $\|\nabla J_\lambda(\theta)\|_2 \leq \frac{\lambda}{2|S||\mathcal{A}|}$, then $J^* - J(\theta) \leq \epsilon_\lambda := \frac{2\lambda}{1-\gamma} \left\| \frac{d_{\pi^*}}{\eta} \right\|_\infty$.

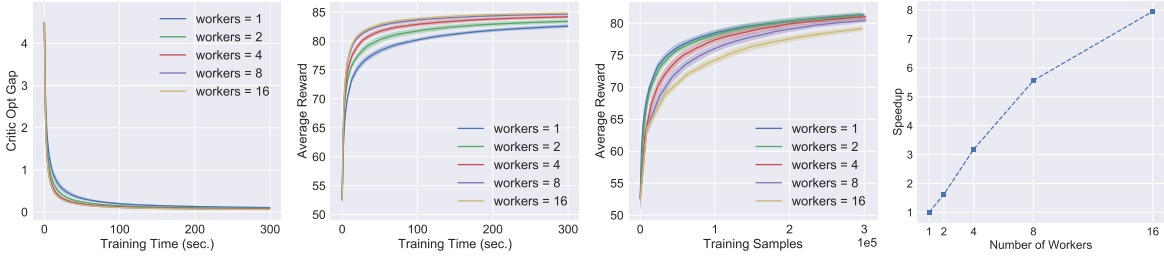


Figure 4: Convergence results of A3C with Markovian sampling in synthetic environment.

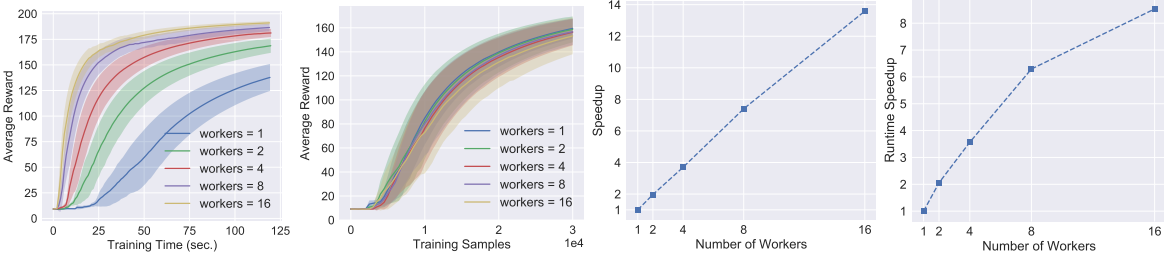


Figure 5: Speedup of A3C in OpenAI gym classic control task (Carpole).

For an arbitrary accuracy ϵ , if we set $\lambda = \frac{(1-\gamma)\epsilon}{2\|\frac{d\pi^*}{d\eta}\|_\infty}$, then we have $\epsilon_\lambda = \epsilon$. Note that in order for $\|\frac{d\pi^*}{d\eta}\|_\infty$ to be finite, we need $\eta(s) > 0$ for any $s \in \mathcal{S}$, which can be assumed without loss of generality. In the case where $\eta > 0$ does not hold, one can start with an exploratory initial state distribution $\eta' > 0$ like in (Agarwal et al., 2020), and our result still holds. This lemma allows us to establish connection between the gradient norm and optimality gap, giving rise to the following theorem.

Theorem 5 Suppose Assumptions 1, 2 and 4–5 hold. Consider Algorithm 1 with softmax policy and linear critic function $\hat{V}_\omega(s) = \phi(s)^\top \omega$. Select step size $\alpha = K^{-\frac{3}{5}}$, $\beta = K^{-\frac{2}{5}}$ and let $K_0 = \Theta(N) = \mathcal{O}(K^{\frac{1}{5}})$, then it holds for i.i.d. sampling

$$J^* - \frac{1}{K} \sum_{k=1}^K \mathbb{E}[J(\theta_k)] = \mathcal{O}\left(\lambda^{-2} K^{-\frac{2}{5}}\right) + \epsilon_\lambda, \quad (24a)$$

and for Markovian sampling

$$J^* - \frac{1}{K} \sum_{k=1}^K \mathbb{E}[J(\theta_k)] = \tilde{\mathcal{O}}\left(\lambda^{-2} K_0 K^{-\frac{2}{5}}\right) + \epsilon_\lambda. \quad (24b)$$

5. Numerical Experiments

We test the impact of separate sampling and the speedup property of A3C in both synthetically generated and Gym environments. The tests on synthetic environment were performed in a 16-core CPU computer, and those on Atari games were run in a 4 GPU computer.

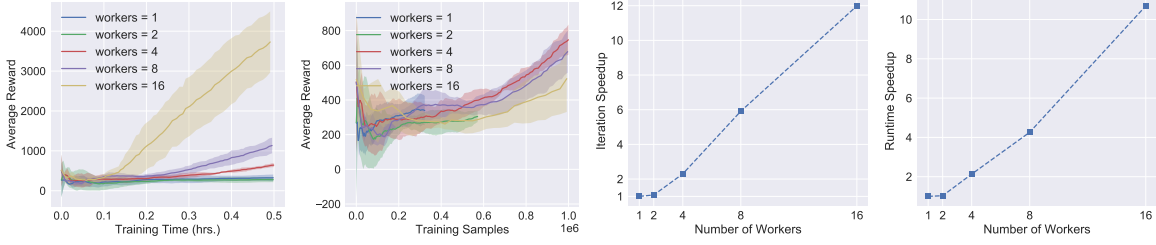


Figure 6: Speedup of A3C in OpenAI Gym Atari game (Beamrider).

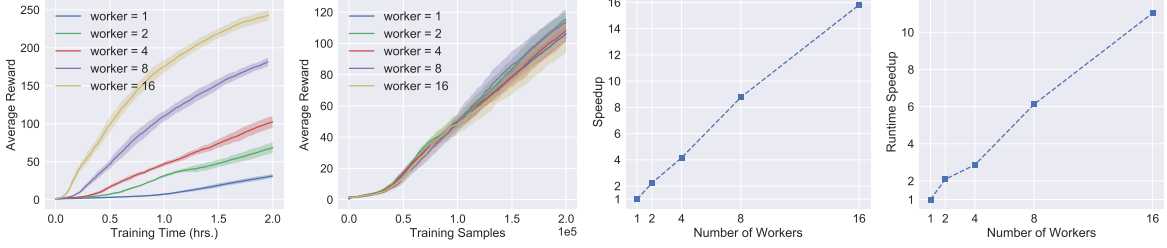


Figure 7: Speedup of A3C in OpenAI Gym Atari game (Breakout).

5.1. Separate sampling protocol

We compare the separate chain sampling method in Algorithm 1 with the shared chain sampling method. The shared chain method is simply using the same sample for both actor and critic, i.e., setting $\hat{x}_t = x_t$ in Algorithm 1.

To clearly demonstrate the impact of sampling, we mitigate the impact from other sources such as delay and MDP non-ergodicity by considering a synthetic environment with 1 worker. In this test, we use the tabular softmax policy parameterization. The synthetic MDP has a state space $|\mathcal{S}| = 10$, an discrete action space of $|\mathcal{A}| = 4$. State features each has a dimension of 10. Elements of the transition matrix, the reward and the state features are randomly sampled from a uniform distribution over $(0, 1)$.

It can be clearly observed from Figure 2 that using the same sample for actor and critic leads to an asymptotic error scaled with choice of γ . The intuitive explanation is in the caption of Figure 2. Although when $\gamma \rightarrow 1$, the error is small (but still exists), we want our algorithm design to not restrict the choice of γ , and thus adopt the separate chain sampling method.

5.2. Linear speedup

Experiment settings. For the synthetic environment, we used linear value function approximation and tabular softmax policy (Agarwal et al., 2020). For CartPole, we used a 3-layer MLP with 128 neurons and sigmoid activation function in each layer. The first two layers are shared for both actor and critic network. For the Atari games, we used a convolution-LSTM network. For network details, see (Dgriff, 2018).

For the separate sampling protocol test, we have $\alpha = 0.6$ and critic step size $\beta = 0.7$, along with $\lambda = 0.3$. For the speedup tests in synthetic environment, we set actor step size $\alpha_k = \frac{0.05}{(1+k)^{0.6}}$ and critic step size $\beta_k = \frac{0.05}{(1+k)^{0.4}}$. In tests of CartPole, we run Algorithm 1 with a minibatch of 20 samples. We update the actor network with a step size of $\alpha_k = \frac{0.01}{(1+k)^{0.6}}$ and critic network with a step size of $\beta_k = \frac{0.01}{(1+k)^{0.4}}$. See Table 1 for hyper-parameters in Atari game tests.

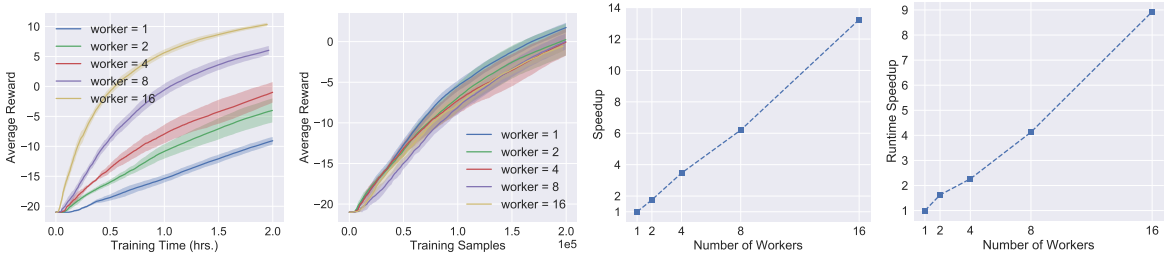


Figure 8: Speedup of A3C in OpenAI Gym Atari game (Pong).

Hyper-parameters	Value
Number of workers	1,2,4,8,16
Optimizer	Adam
Step size	0.00015
Batch size	20
Discount factor	0.99
Entropy coefficient	0.01
Frame size	80×80
Frame skip rate	4
Grayscale	Yes
Training reward clipping	$[-1,1]$

Table 1: Hyper-parameters of A3C in the Atari games.

Synthetic environment. We first test the speedup property of A3C in a synthetic environment with $|\mathcal{S}| = 100$, $|\mathcal{A}| = 5$ and state feature with dimension 10. The reward and transition matrix of the MDP are randomly generated in the same way as that in section 5.1. We evaluate the convergence of actor in terms of the average reward and the critic in terms of the gap $\|\omega_k - \omega_{\theta_k}^*\|_2$.

Figures 3 and 4 show the training time and sample complexity of running A3C with i.i.d. sampling and Markovian sampling respectively. The speedup plot is measured by the number of samples needed to achieve a target running average reward under different number of workers. All the results are average over 10 Monte-Carlo runs. Figure 3 shows that the sample complexity of A3C stays the same with different number of workers under i.i.d. sampling. Also, it can be observed from the speedup plot of Figure 3 that the A3C achieves roughly linear speedup, which is consistent with Corollary 1. The speedup of A3C with Markovian sampling shown in Figure 4 is roughly linear when number of workers is small.

OpenAI Gym environments. We also test the speedup property of A3C with neural network parametrization in the classic control (Carpole) and the Atari (Breakout and Pong) environments. In Figures 5-8, each curve was averaged over 5 Monte-Carlo runs with 95% confidence interval. Figures 5-8 show the speedup of A3C under different number of workers, where the average reward is computed by taking the running average of test rewards. The speedup is respectively measured by the number of samples and training time needed per worker to achieve a target average reward. Although not justified theoretically, Figures 5-8 suggest that the sample complexity speedup is roughly linear, and the runtime speedup slightly degrades when the number of workers increases.

This is partially due to hardware limit. Similar observation was obtained in async-SGD (Lian et al., 2016, Fig. 4).

6. Conclusions

This paper revisits the A3C algorithm. With linear value function approximation, the convergence of the A3C algorithm has been established under both i.i.d. and Markovian sampling settings. Under i.i.d. sampling, A3C achieves linear speedup compared to the best-known sample complexity of AC, theoretically justifying the benefit of parallelism and asynchrony for the first time. Under Markov sampling, such a linear speedup can be observed in most benchmark tasks. One limitation of this paper is that theoretical linear speedup cannot be established in the Markovian setting. This motivates two interesting directions: i) developing new tools of analyzing two-timescale SGD with Markov sampling; and, ii) designing better algorithms than A3C to achieve better speedup.

References

- A. Agarwal and J. C. Duchi. Distributed delayed stochastic optimization. In *Proceedings of Advances in Neural Information Processing Systems*, 2011.
- A. Agarwal, S. M. Kakade, J. D. Lee, and G. Mahajan. Optimality and approximation with policy gradient methods in markov decision processes. In *Proceedings of Thirty Third Conference on Learning Theory*, 2020.
- M. Assran, J. Romoff, N. Ballas, J. Pineau, and M. Rabbat. Gossip-based actor-learner architectures for deep reinforcement learning. In *Proceedings of Advances in Neural Information Processing Systems*, 2019.
- M. Babaeizadeh, I. Frosio, S. Tyree, J. Clemons, and J. Kautz. Reinforcement learning through asynchronous advantage actor-critic on a gpu. In *Proceedings of International Conference on Learning Representations*, 2017.
- J. Baxter and P. L. Bartlett. Infinite-horizon policy-gradient estimation. *J. Artificial Intelligence Res.*, 15:319–350, 2001.
- D. P. Bertsekas and J. N. Tsitsiklis. *Parallel and distributed computation: numerical methods*. Prentice-Hall, 1989.
- J. Bhandari and D. Russo. Global optimality guarantees for policy gradient methods. *arXiv preprint:1906.01786*, 2019.
- J. Bhandari and D. Russo. On the linear convergence of policy gradient methods for finite mdps. In *Proceedings of The 24th International Conference on Artificial Intelligence and Statistics*, 2021.
- J. Bhandari, D. Russo, and R. Singal. A finite time analysis of temporal difference learning with linear function approximation. In *Proceedings of Conference on Learning Theory*, 2018.
- S. Bhatnagar, R. S. Sutton, M. Ghavamzadeh, and M. Lee. Natural actor critic algorithms. *Automatica*, 45:2471–2482, 2009.

- V. Borkar and V. Konda. The actor-critic algorithm as multi-time-scale stochastic approximation. *Sadhana*, 22(4):525–543, 1997.
- J. Cervino, J. A. Bazerque, M. Calvo-Fullana, and A. Ribeiro. Multi-task reinforcement learning in reproducing kernel hilbert spaces via cross-learning. *IEEE Transactions on Signal Processing*, 69: 5947–5962, 2021.
- S. Chai and V. K. N. Lau. Joint rate and power optimization for multimedia streaming in wireless fading channels via parametric policy gradient. *IEEE Transactions on Signal Processing*, 67(17): 4570–4581, 2019.
- S. Chai and V. K. N. Lau. Online trajectory and radio resource optimization of cache-enabled uav wireless networks with content and energy recharging. *IEEE Transactions on Signal Processing*, 68:1286–1299, 2020.
- T. Chen, K. Zhang, G. B. Giannakis, and T. Başar. Communication-efficient distributed reinforcement learning. *arXiv preprint: 1812.03239*, 2018.
- F. Christianos, L. Schäfer, and S. Albrecht. Shared experience actor-critic for multi-agent reinforcement learning. In *Proceedings of Advances in Neural Information Processing Systems*, 2020.
- Dgriff. Pytorch implementation of a3c. https://github.com/dgriff777/rl_a3c_pytorch, 2018.
- K. Doya. Reinforcement learning in continuous time and space. *Neural Computation*, 12(1):219–245, 2000.
- Y. El-Laham and M. F. Bugallo. Policy gradient importance sampling for bayesian inference. *IEEE Transactions on Signal Processing*, 69:4245–4256, 2021.
- L. Espeholt, H. Soyer, R. Munos, K. Simonyan, V. Mnih, T. Ward, Y. Doron, V. Firoiu, T. Harley, I. Dunning, S. Legg, and K. Kavukcuoglu. Impala: Scalable distributed deep-rl with importance weighted actor-learner architectures. *arXiv preprint:1802.01561*, 2018.
- H. R. Feyzmahdavian, A. Aytakin, and M. Johansson. An asynchronous mini-batch algorithm for regularized stochastic optimization. *arXiv preprint:1505.04824*, 2015.
- Z. Fu, Z. Yang, and Z. Wang. Single-timescale actor-critic provably finds globally optimal policy. *arXiv preprint:2008.00483*, 2020.
- M. Hong, H.-T. Wai, Z. Wang, and Z. Yang. A two-timescale framework for bilevel optimization: Complexity analysis and application to actor-critic. *arXiv preprint:2007.05170*, 2020.
- S. Kar, J. Moura, and V. Poor. QD-learning: A collaborative distributed strategy for multi-agent reinforcement learning through Consensus+Innovations. *IEEE Transactions on Signal Processing*, 61(7):1848–1862, 2013.
- V. Konda. *Actor-critic algorithms*. PhD thesis, Department of Electrical Engineering and Computer Science, Massachusetts Institute of Technology, 2002.

- V. Konda and V. Borkar. Actor-critic-type learning algorithms for markov decision processes. *SIAM Journal on Control and Optimization*, 38(1):94–123, 1999.
- H. Kumar, A. Koppel, and A. Ribeiro. On the sample complexity of actor-critic method for reinforcement learning with function approximation. *arXiv preprint:1910.08412*, 2019.
- X. Lian, H. Zhang, C. Hsieh, Y. Yijun Huang, and J. Liu. A comprehensive linear speedup analysis for asynchronous stochastic parallel optimization from zeroth-order to first-order. In *Advances in Neural Information Processing Systems*, 2016.
- X. Lian, C. Zhang, H. Zhang, C.-J. Hsieh, W. Zhang, and J. Liu. Can decentralized algorithms outperform centralized algorithms? a case study for decentralized parallel stochastic gradient descent. In *Proceedings of Advances in Neural Information Processing Systems*, 2017.
- T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra. Continuous control with deep reinforcement learning. In *Proceedings of International Conference on Learning Representations*, 2016.
- J. Mei, C. Xiao, C. Szepesvari, and D. Schuurmans. On the global convergence rates of softmax policy gradient methods. In *Proceedings of International Conference on Machine Learning*, 2020.
- V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529, 2015.
- V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. P. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In *Proceedings of International Conference on Machine Learning*, 2016.
- A. Nair, P. Srinivasan, S. Blackwell, C. Alcicek, R. Fearon, A. De Maria, V. Panneershelvam, M. Suleyman, C. Beattie, S. Petersen, et al. Massively parallel methods for deep reinforcement learning. *arXiv preprint:1507.04296*, 2015.
- S. Qiu, Z. Yang, J. Ye, and Z. Wang. On the finite-time convergence of actor-critic algorithm. In *Optimization Foundations for Reinforcement Learning Workshop at Advances in Neural Information Processing Systems*, 2019.
- G. Qu, Y. Lin, A. Wierman, and N. Li. Scalable multi-agent reinforcement learning for networked systems with average reward. In *Proceedings of Advances in Neural Information Processing Systems*, 2020.
- A. Sadeghi, F. Sheikholeslami, and G. B. Giannakis. Optimal and scalable caching for 5g using reinforcement learning of space-time popularities. *IEEE Journal of Selected Topics in Signal Processing*, 12(1):180–190, 2018.
- A. Stooke and P. Abbeel. Accelerated methods for deep reinforcement learning. *arXiv preprint:1803.02811*, 2019.
- T. Sun, R. Hannah, and W. Yin. Asynchronous coordinate descent under more realistic assumptions. In *Proceedings of Advances in Neural Information Processing Systems*, 2017.

- R. S. Sutton. Learning to predict by the methods of temporal differences. *Machine Learning*, 3:9–44, 1988.
- R. S. Sutton and A. G. Barto. *Reinforcement learning: An introduction*. MIT Press, 2018.
- R. S. Sutton, D. McAllester, S. Singh, and Y. Mansour. Policy gradient methods for reinforcement learning with function approximation. In *Proceedings of Advances in Neural Information Processing Systems*, 2000.
- L. Wang, Q. Cai, Z. Yang, and Z. Wang. Neural policy gradient methods: Global optimality and rates of convergence. *arXiv preprint:1909.01150*, 2019.
- R. J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3-4):229–256, 1992.
- T. Wu, K. Yuan, Q. Ling, W. Yin, and A. Sayed. Decentralized consensus optimization with asynchrony and delays. *IEEE Transactions on Signal and Information Processing over Networks*, 4(2):293–307, 2018.
- Y. Wu, W. Zhang, P. Xu, and Q. Gu. A finite time analysis of two time-scale actor critic methods. In *Proceedings of Advances in Neural Information Processing Systems*, 2020.
- Z. Wu, H. Shen, T. Chen, and Q. Ling. Byzantine-resilient decentralized policy evaluation with linear function approximation. *IEEE Transactions on Signal Processing*, 69:3839–3853, 2021.
- T. Xu, Z. Wang, and Y. Liang. Improving sample complexity bounds for (natural) actor-critic algorithms. In *Proceedings of Advances in Neural Information Processing Systems*, 2020a.
- T. Xu, Z. Wang, and Y. Liang. Non-asymptotic convergence analysis of two time-scale (natural) actor-critic algorithms. *arXiv preprint:2005.03557*, 2020b.
- Z. Yang, K. Zhang, M. Hong, and T. Başar. A finite sample analysis of the actor-critic algorithm. In *Proceedings of IEEE Conference on Decision and Control*, 2018.
- J. Zhang, J. Kim, B. Donoghue, and S. Boyd. Sample efficient reinforcement learning with reinforce. In *Proceedings of AAAI Conference on Artificial Intelligence*, 2021.
- K. Zhang, Z. Yang, H. Liu, T. Zhang, and T. Başar. Fully decentralized multi-agent reinforcement learning with networked agents. In *Proceedings of International Conference on Machine Learning*, 2018.
- K. Zhang, A. Koppel, H. Zhu, and T. Başar. Global convergence of policy gradient methods to (almost) locally optimal policies. *arXiv preprint:1906.08383*, 2019.
- S. Zou, T. Xu, and Y. Liang. Finite-sample analysis for SARSA with linear function approximation. In *Proceedings of Advances in Neural Information Processing Systems*, 2019.

Supplementary Material

Appendix A. Preliminary Lemmas

A.1. Geometric mixing

The operation $p \otimes q$ denotes the product between two distributions $p(x)$ and $q(y)$, i.e. $(p \otimes q)(x, y) = p(x) \cdot q(y)$.

Lemma 2 Suppose Assumption 4 holds. For any $\theta \in \mathbb{R}^d$, we have

$$\sup_{s_0 \in \mathcal{S}} d_{TV}(\mathbb{P}((s_t, a_t, s_{t+1}) \in \cdot | s_0, \pi_\theta), \mu_\theta \otimes \pi_\theta \otimes \mathcal{P}) \leq \kappa \rho^t. \quad (25a)$$

and

$$\sup_{s_0 \in \mathcal{S}} d_{TV}(\mathbb{P}((\hat{s}_t, \hat{a}_t, s'_{t+1}) \in \cdot | s_0, \pi_\theta), d_\theta \otimes \pi_\theta \otimes \mathcal{P}) \leq \kappa \rho^t. \quad (25b)$$

where (s_t, a_t, s_{t+1}) is the t th transition on the Markov chain with transition kernel $\mathcal{P}$. $(\hat{s}_t, \hat{a}_t)$ is the t th state-action pair on Markov chain with transition kernel $\hat{\mathcal{P}}$, and $s'_{t+1} \sim \mathcal{P}(\cdot | \hat{s}_t, \hat{a}_t)$.

Proof We start with

$$\begin{aligned} & \sup_{s_0 \in \mathcal{S}} d_{TV}(\mathbb{P}((s_t, a_t, s_{t+1}) \in \cdot | s_0, \pi_\theta), \mu_\theta \otimes \pi_\theta \otimes \mathcal{P}) \\ &= \sup_{s_0 \in \mathcal{S}} d_{TV}(\mathbb{P}(s_t = \cdot | s_0, \pi_\theta) \otimes \pi_\theta \otimes \mathcal{P}, \mu_\theta \otimes \pi_\theta \otimes \mathcal{P}) \\ &= \sup_{s_0 \in \mathcal{S}} \frac{1}{2} \int_{s \in \mathcal{S}} \sum_{a \in \mathcal{A}} \int_{s' \in \mathcal{S}} |\mathbb{P}(s_t = ds | s_0, \pi_\theta) \pi_\theta(a | s) \mathcal{P}(ds' | s, a) - \mu_\theta(ds) \pi_\theta(a | s) \mathcal{P}(ds' | s, a)| \\ &= \sup_{s_0 \in \mathcal{S}} \frac{1}{2} \int_{s \in \mathcal{S}} |\mathbb{P}(s_t = ds | s_0, \pi_\theta) - \mu_\theta(ds)| \sum_{a \in \mathcal{A}} \pi_\theta(a | s) \int_{s' \in \mathcal{S}} \mathcal{P}(ds' | s, a) \\ &= \sup_{s_0 \in \mathcal{S}} d_{TV}(\mathbb{P}(s_t \in \cdot | s_0, \pi_\theta), \mu_\theta) \\ &\leq \kappa \rho^t, \end{aligned} \quad (26)$$

Inequality (25a) along with the fact that the stationary distribution of the Markov chain with transition probability $\hat{\mathcal{P}}$ and policy π_θ is simply d_θ immediately implies (25b). This completes the proof. ■

For the use in the later proof, given $K > 0$, we first define m_K as:

$$m_K := \min \{ m \in \mathbb{N}^+ \mid \kappa \rho^{m-1} \leq \min\{\alpha, \beta\} \}, \quad (27)$$

where κ and ρ are constants defined in (4). m_K is the minimum number of samples needed for the Markov chain to approach the stationary distribution so that the bias incurred by the Markovian sampling is small enough.

A.2. Auxiliary Markov chain

The auxiliary Markov chain is a virtual Markov chain with no policy drifting — a technique developed in (Zou et al., 2019) to analyze stochastic approximation algorithms in non-stationary settings. We provide an analysis here for completeness.

Lemma 3 *Under Assumption 2 and Assumption 3, consider the update (13) in Algorithm 1 with Markovian sampling. For a given number of samples m , consider the Markov chain of the worker that contributes to the k th update:*

$$s_{t-m} \xrightarrow{\theta_{k-d_m}} a_{t-m} \xrightarrow{\mathcal{P}} s_{t-m+1} \xrightarrow{\theta_{k-d_{m-1}}} a_{t-m+1} \cdots s_{t-1} \xrightarrow{\theta_{k-d_1}} a_{t-1} \xrightarrow{\mathcal{P}} s_t \xrightarrow{\theta_{k-d_0}} a_t \xrightarrow{\mathcal{P}} s_{t+1},$$

where $(s_t, a_t, s_{t+1}) = (s_{(k)}, a_{(k)}, s'_{(k)})$, and $\{d_j\}_{j=0}^m$ is some increasing sequence with $d_0 := \tau_k$.

Given $(s_{t-m}, a_{t-m}, s_{t-m+1})$ and θ_{k-d_m} , we construct its auxiliary Markov chain by repeatedly applying $\pi_{\theta_{k-d_m}}$:

$$s_{t-m} \xrightarrow{\theta_{k-d_m}} a_{t-m} \xrightarrow{\mathcal{P}} s_{t-m+1} \xrightarrow{\theta_{k-d_m}} \tilde{a}_{t-m+1} \cdots \tilde{s}_{t-1} \xrightarrow{\theta_{k-d_m}} \tilde{a}_{t-1} \xrightarrow{\mathcal{P}} \tilde{s}_t \xrightarrow{\theta_{k-d_m}} \tilde{a}_t \xrightarrow{\mathcal{P}} \tilde{s}_{t+1}.$$

Then we have:

$$\begin{aligned} & d_{TV} \left(\mathbb{P}((s_t, a_t) \in \cdot | \theta_{k-d_m}, s_{t-m+1}), \mathbb{P}((\tilde{s}_t, \tilde{a}_t) \in \cdot | \theta_{k-d_m}, s_{t-m+1}) \right) \\ & \leq \frac{1}{2} |\mathcal{A}| L_\pi \sum_{i=\tau_k}^{d_m} \mathbb{E} [\|\theta_{k-i} - \theta_{k-d_m}\|_2 | \theta_{k-d_m}, s_{t-m+1}]. \end{aligned} \quad (28)$$

Proof Throughout the lemma, all expectations and probabilities are conditioned on θ_{k-d_m} and s_{t-m+1} . We omit this condition for convenience.

With $\tilde{x}_t := (\tilde{s}_t, \tilde{a}_t, \tilde{s}_{t+1})$, first we have

$$\begin{aligned} & d_{TV} (\mathbb{P}(s_{t+1} \in \cdot), \mathbb{P}(\tilde{s}_{t+1} \in \cdot)) \\ &= \frac{1}{2} \int_{s' \in \mathcal{S}} |\mathbb{P}(s_{t+1} = ds') - \mathbb{P}(\tilde{s}_{t+1} = ds')| \\ &= \frac{1}{2} \int_{s' \in \mathcal{S}} \left| \int_{s \in \mathcal{S}} \sum_{a \in \mathcal{A}} \mathbb{P}(s_t = ds, a_t = a, s_{t+1} = ds') - \mathbb{P}(\tilde{s}_t = ds, \tilde{a}_t = a, \tilde{s}_{t+1} = ds') \right| \\ &\leq \frac{1}{2} \int_{s' \in \mathcal{S}} \int_{s \in \mathcal{S}} \sum_{a \in \mathcal{A}} |\mathbb{P}(s_t = ds, a_t = a, s_{t+1} = ds') - \mathbb{P}(\tilde{s}_t = ds, \tilde{a}_t = a, \tilde{s}_{t+1} = ds')| \\ &= \frac{1}{2} \int_{s \in \mathcal{S}} \sum_{a \in \mathcal{A}} \int_{s' \in \mathcal{S}} |\mathbb{P}(s_t = ds, a_t = a, s_{t+1} = ds') - \mathbb{P}(\tilde{s}_t = ds, \tilde{a}_t = a, \tilde{s}_{t+1} = ds')| \\ &= d_{TV} (\mathbb{P}(x_t \in \cdot), \mathbb{P}(\tilde{x}_t \in \cdot)), \end{aligned} \quad (29)$$

where the second last equality is due to Tonelli's theorem. Next we have

$$\begin{aligned}
 & d_{TV}(\mathbb{P}(x_t \in \cdot), \mathbb{P}(\tilde{x}_t \in \cdot)) \\
 &= \frac{1}{2} \int_{s \in \mathcal{S}} \sum_{a \in \mathcal{A}} \int_{s' \in \mathcal{S}} |\mathbb{P}(s_t = ds, a_t = a, s_{t+1} = ds') - \mathbb{P}(\tilde{s}_t = ds, \tilde{a}_t = a, \tilde{s}_{t+1} = ds')| \\
 &= \frac{1}{2} \int_{s \in \mathcal{S}} \sum_{a \in \mathcal{A}} |\mathbb{P}(s_t = ds, a_t = a) - \mathbb{P}(\tilde{s}_t = ds, \tilde{a}_t = a)| \int_{s' \in \mathcal{S}} \mathcal{P}(s_{t+1} = ds' | s_t = ds, a_t = a) \\
 &= \frac{1}{2} \int_{s \in \mathcal{S}} \sum_{a \in \mathcal{A}} |\mathbb{P}(s_t = ds, a_t = a) - \mathbb{P}(\tilde{s}_t = ds, \tilde{a}_t = a)| \\
 &= d_{TV}(\mathbb{P}((s_t, a_t) \in \cdot), \mathbb{P}((\tilde{s}_t, \tilde{a}_t) \in \cdot)). \tag{30}
 \end{aligned}$$

Due to the fact that $\theta_{k-\tau_k}$ is dependent on s_t , we need to write $\mathbb{P}(s_t, a_t)$ as

$$\begin{aligned}
 \mathbb{P}(s_t, a_t) &= \int_{\theta_{k-\tau_k} \in \mathbb{R}^d} \mathbb{P}(s_t, \theta_{k-\tau_k}, a_t) \\
 &= \int_{\theta \in \mathbb{R}^d} \mathbb{P}(s_t) \mathbb{P}(\theta_{k-\tau_k} = d\theta | s_t) \pi_{\theta_{k-\tau_k}}(a_t | s_t) \\
 &= \mathbb{P}(s_t) \mathbb{E}[\pi_{\theta_{k-\tau_k}}(a_t | s_t) | s_t]. \tag{31}
 \end{aligned}$$

Then we have

$$\begin{aligned}
 & d_{TV}(\mathbb{P}((s_t, a_t) \in \cdot), \mathbb{P}((\tilde{s}_t, \tilde{a}_t) \in \cdot)) \\
 &= \frac{1}{2} \int_{s \in \mathcal{S}} \sum_{a \in \mathcal{A}} |\mathbb{P}(s_t = ds) \mathbb{E}[\pi_{\theta_{k-\tau_k}}(a_t = a | s_t = ds) | s_t = ds] - \mathbb{P}(\tilde{s}_t = ds) \pi_{\theta_{k-dm}}(\tilde{a}_t = a | \tilde{s}_t = ds)| \\
 &\leq \frac{1}{2} \int_{s \in \mathcal{S}} \sum_{a \in \mathcal{A}} |\mathbb{P}(s_t = ds) \mathbb{E}[\pi_{\theta_{k-\tau_k}}(a_t = a | s_t = ds) | s_t = ds] - \mathbb{P}(s_t = ds) \pi_{\theta_{k-dm}}(a_t = a | s_t = ds)| \\
 &\quad + \frac{1}{2} \int_{s \in \mathcal{S}} \sum_{a \in \mathcal{A}} |\mathbb{P}(s_t = ds) \pi_{\theta_{k-dm}}(\tilde{a}_t = a | \tilde{s}_t = ds) - \mathbb{P}(\tilde{s}_t = ds) \pi_{\theta_{k-dm}}(\tilde{a}_t = a | \tilde{s}_t = ds)| \\
 &= \frac{1}{2} \int_{s \in \mathcal{S}} \mathbb{P}(s_t = ds) \sum_{a \in \mathcal{A}} |\mathbb{E}[\pi_{\theta_{k-\tau_k}}(a_t = a | s_t = ds) | s_t = ds] - \pi_{\theta_{k-dm}}(a_t = a | s_t = ds)| \\
 &\quad + \frac{1}{2} \int_{s \in \mathcal{S}} |\mathbb{P}(s_t = ds) - \mathbb{P}(\tilde{s}_t = ds)|. \tag{32}
 \end{aligned}$$

Using Jensen's inequality, we have

$$\begin{aligned}
 & d_{TV}(\mathbb{P}((s_t, a_t) \in \cdot), \mathbb{P}((\tilde{s}_t, \tilde{a}_t) \in \cdot)) \\
 &\leq \frac{1}{2} \int_{s \in \mathcal{S}} \mathbb{P}(s_t = ds) \sum_{a \in \mathcal{A}} \mathbb{E} \left[\left| \pi_{\theta_{k-\tau_k}}(a_t = a | s_t = ds) - \pi_{\theta_{k-dm}}(a_t = a | s_t = ds) \right| \middle| s_t = ds \right] \\
 &\quad + \frac{1}{2} \int_{s \in \mathcal{S}} |\mathbb{P}(s_t = ds) - \mathbb{P}(\tilde{s}_t = ds)| \\
 &\leq \frac{1}{2} \int_{s \in \mathcal{S}} \mathbb{P}(s_t = ds) \sum_{a \in \mathcal{A}} \mathbb{E} [\|\theta_{k-\tau_k} - \theta_{k-dm}\|_2 | s_t = ds] + \frac{1}{2} \int_{s \in \mathcal{S}} |\mathbb{P}(s_t = ds) - \mathbb{P}(\tilde{s}_t = ds)| \\
 &= \frac{1}{2} |\mathcal{A}| L_\pi \mathbb{E} \|\theta_{k-\tau_k} - \theta_{k-dm}\|_2 + d_{TV}(\mathbb{P}(s_t \in \cdot), \mathbb{P}(\tilde{s}_t \in \cdot)) \tag{33}
 \end{aligned}$$

where the last inequality follows Assumption 3.

Now we start to prove (28). First we have

$$\begin{aligned}
& d_{TV}(\mathbb{P}((s_t, a_t) \in \cdot), \mathbb{P}((\tilde{s}_t, \tilde{a}_t) \in \cdot)) \\
& \stackrel{(33)}{\leq} d_{TV}(\mathbb{P}(s_t \in \cdot), \mathbb{P}(\tilde{s}_t \in \cdot)) + \frac{1}{2} |\mathcal{A}| L_\pi \mathbb{E} \|\theta_{k-\tau_k} - \theta_{k-d_m}\|_2 \\
& \stackrel{(29)}{\leq} d_{TV}(\mathbb{P}(x_{t-1} \in \cdot), \mathbb{P}(\tilde{x}_{t-1} \in \cdot)) + \frac{1}{2} |\mathcal{A}| L_\pi \mathbb{E} \|\theta_{k-\tau_k} - \theta_{k-d_m}\|_2 \\
& \stackrel{(30)}{=} d_{TV}(\mathbb{P}((s_{t-1}, a_{t-1}) \in \cdot), \mathbb{P}((\tilde{s}_{t-1}, \tilde{a}_{t-1}) \in \cdot)). \tag{34}
\end{aligned}$$

Since $d_{TV}(\mathbb{P}((s_{t-m}, a_{t-m}) \in \cdot), \mathbb{P}((\tilde{s}_{t-m}, \tilde{a}_{t-m}) \in \cdot)) = 0$, recursively applying (34) for $\{t-1, \dots, t-m\}$ gives

$$\begin{aligned}
d_{TV}(\mathbb{P}((s_t, a_t) \in \cdot), \mathbb{P}((\tilde{s}_t, \tilde{a}_t) \in \cdot)) & \leq \frac{1}{2} |\mathcal{A}| L_\pi \sum_{j=0}^m \mathbb{E} \|\theta_{k-d_j} - \theta_{k-d_m}\|_2 \\
& \leq \frac{1}{2} |\mathcal{A}| L_\pi \sum_{i=\tau_k}^{d_m} \mathbb{E} \|\theta_{k-i} - \theta_{k-d_m}\|_2, \tag{35}
\end{aligned}$$

which completes the proof. $\blacksquare$

A.3. Lipschitz continuity of actor and critic

We first give a proposition regarding the L_λ -Lipschitz continuity of the regularized policy gradient under proper assumptions, which has been shown by (Zhang et al., 2019; Agarwal et al., 2020).

Proposition 1 *Suppose Assumption 3 hold. For any $\theta, \theta' \in \mathbb{R}^d$, we have $\|\nabla J_\lambda(\theta) - \nabla J_\lambda(\theta')\|_2 \leq L_\lambda \|\theta - \theta'\|_2$, where L_λ is a positive constant.*

We provide a justification for Lipschitz continuity of ω_θ^* in the next proposition.

Proposition 2 *Suppose Assumption 1, 3 and 4 hold. For any $\theta_1, \theta_2 \in \mathbb{R}^d$, we have*

$$\|\omega_{\theta_1}^* - \omega_{\theta_2}^*\|_2 \leq L_\omega \|\theta_1 - \theta_2\|_2,$$

where $L_\omega := 2r_{\max} |\mathcal{A}| L_\pi (\lambda^{-1} + \lambda^{-2}(1 + \gamma))(1 + \log_\rho \kappa^{-1} + (1 - \rho)^{-1})$.

Proof We use A_1, A_2, b_1 and b_2 as shorthand notations of $A_{\pi_{\theta_1}}, A_{\pi_{\theta_2}}, b_{\pi_{\theta_1}}$ and $b_{\pi_{\theta_2}}$ respectively. By Assumption 1, $A_{\theta, \phi}$ is invertible for any $\theta \in \mathbb{R}^d$, so we can write $\omega_\theta^* = -A_{\theta, \phi}^{-1} b_{\theta, \phi}$. Then we have

$$\begin{aligned}
\|\omega_1^* - \omega_2^*\|_2 &= \|-A_1^{-1} b_1 + A_2^{-1} b_2\|_2 \\
&= \|-A_1^{-1} b_1 - A_1^{-1} b_2 + A_1^{-1} b_2 + A_2^{-1} b_2\|_2 \\
&= \|-A_1^{-1} (b_1 - b_2) - (A_1^{-1} - A_2^{-1}) b_2\|_2 \\
&\leq \|A_1^{-1} (b_1 - b_2)\|_2 + \|(A_1^{-1} - A_2^{-1}) b_2\|_2 \\
&\leq \|A_1^{-1}\|_2 \|b_1 - b_2\|_2 + \|A_1^{-1} - A_2^{-1}\|_2 \|b_2\|_2 \\
&= \|A_1^{-1}\|_2 \|b_1 - b_2\|_2 + \|A_1^{-1} (A_2 - A_1) A_2^{-1}\|_2 \|b_2\|_2 \\
&\leq \|A_1^{-1}\|_2 \|b_1 - b_2\|_2 + \|A_1^{-1}\|_2 \|A_2^{-1}\|_2 \|b_2\|_2 \|A_2 - A_1\|_2 \\
&\leq \lambda^{-1} \|b_1 - b_2\|_2 + \lambda^{-2} r_{\max} \|A_1 - A_2\|_2, \tag{36}
\end{aligned}$$

where the last inequality follows Assumption 1, and the fact that

$$\|b_2\|_2 = \|\mathbb{E}[r(s, a, s')\phi(s)]\|_2 \leq \mathbb{E}\|r(s, a, s')\phi(s)\|_2 \leq \mathbb{E}[|r(s, a, s')|\|\phi(s)\|_2] \leq r_{\max}.$$

Denote (s^1, a^1, s'^1) and (s^2, a^2, s'^2) as samples drawn with θ_1 and θ_2 respectively, i.e. $s^1 \sim \mu_{\theta_1}$, $a^1 \sim \pi_{\theta_1}$, $s'^1 \sim \mathcal{P}$ and $s^2 \sim \mu_{\theta_2}$, $a^2 \sim \pi_{\theta_2}$, $s'^2 \sim \mathcal{P}$. Then we have

$$\begin{aligned} \|b_1 - b_2\|_2 &= \|\mathbb{E}[r(s^1, a^1, s'^1)\phi(s^1)] - \mathbb{E}[r(s^2, a^2, s'^2)\phi(s^2)]\|_2 \\ &\leq \sup_{s, a, s'} \|r(s, a, s')\phi(s)\|_2 \|\mathbb{P}((s^1, a^1, s'^1) \in \cdot) - \mathbb{P}((s^2, a^2, s'^2) \in \cdot)\|_{TV} \\ &\leq r_{\max} \|\mathbb{P}((s^1, a^1, s'^1) \in \cdot) - \mathbb{P}((s^2, a^2, s'^2) \in \cdot)\|_{TV} \\ &= 2r_{\max} d_{TV}(\mu_{\theta_1} \otimes \pi_{\theta_1} \otimes \mathcal{P}, \mu_{\theta_2} \otimes \pi_{\theta_2} \otimes \mathcal{P}) \\ &\leq 2r_{\max} |\mathcal{A}| L_{\pi} (1 + \log_{\rho} \kappa^{-1} + (1 - \rho)^{-1}) \|\theta_1 - \theta_2\|_2, \end{aligned} \quad (37)$$

where the first inequality follows the definition of total variation (TV) norm, and the last inequality follows Lemma A.1. in (Wu et al., 2020). Similarly we have:

$$\begin{aligned} \|A_1 - A_2\|_2 &\leq 2(1 + \gamma) d_{TV}(\mu_{\theta_1} \otimes \pi_{\theta_1}, \mu_{\theta_2} \otimes \pi_{\theta_2}) \\ &= (1 + \gamma) |\mathcal{A}| L_{\pi} (1 + \log_{\rho} \kappa^{-1} + (1 - \rho)^{-1}) \|\theta_1 - \theta_2\|_2. \end{aligned} \quad (38)$$

Substituting (37) and (38) into (36) completes the proof. $\blacksquare$

Appendix B. Proof of Main Theorems

B.1. Proof of Theorem 1

We first define the exact TD update as:

$$\bar{g}(x, \omega) := \mathbb{E}_{s \sim \mu_{\theta}, a \sim \pi_{\theta}, s' \sim \mathcal{P}} [g(x, \omega)]. \quad (39)$$

We also define constant $C_{\delta} := r_{\max} + (1 + \gamma) \max\{\frac{r_{\max}}{1 - \gamma}, R_{\omega}\}$, and we immediately have

$$\|g(x, \omega)\|_2 \leq |r(x) + \gamma \phi(s')^{\top} \omega - \phi(s)^{\top} \omega| \leq r_{\max} + (1 + \gamma) R_{\omega} \leq C_{\delta} \quad (40)$$

and likewise, we have $\|\bar{g}(x, \omega)\|_2 \leq C_{\delta}$.

The critic update in Algorithm 1 can be written as:

$$\omega_{k+1} = \Pi_{R_{\omega}} (\omega_k + \beta g(x_{(k)}, \omega_{k-\tau_k})), \quad (41)$$

where τ_k is the delay of the parameters used in evaluating the k th stochastic gradient, and $x_{(k)} := (s_{(k)}, a_{(k)}, s'_{(k)})$ is the sample used to evaluate the stochastic gradient at k th update.

Proof Using ω_k^* as shorthand notation of $\omega_{\theta_k}^*$, we start with the optimality gap

$$\begin{aligned}
& \|\omega_{k+1} - \omega_{k+1}^*\|_2^2 \\
&= \|\Pi_{R_\omega}(\omega_k + \beta g(x_{(k)}, \omega_{k-\tau_k})) - \omega_{k+1}^*\|_2^2 \\
&\leq \|\omega_k + \beta g(x_{(k)}, \omega_{k-\tau_k}) - \omega_{k+1}^*\|_2^2 \\
&= \|\omega_k - \omega_k^*\|_2^2 + 2\beta \langle \omega_k - \omega_k^*, g(x_{(k)}, \omega_{k-\tau_k}) \rangle + 2 \langle \omega_k - \omega_k^*, \omega_k^* - \omega_{k+1}^* \rangle \\
&\quad + \|\omega_k^* - \omega_{k+1}^* + \beta g(x_{(k)}, \omega_{k-\tau_k})\|_2^2 \\
&= \|\omega_k - \omega_k^*\|_2^2 + 2\beta \langle \omega_k - \omega_k^*, g(x_{(k)}, \omega_{k-\tau_k}) \rangle \\
&\quad + 2 \langle \omega_k - \omega_k^*, \omega_k^* - \omega_{k+1}^* \rangle + 2 \|\omega_k^* - \omega_{k+1}^*\|_2^2 + 2C_\delta^2 \beta^2.
\end{aligned} \tag{42}$$

The second term in (42) can be decomposed as

$$\begin{aligned}
\langle \omega_k - \omega_k^*, g(x_{(k)}, \omega_{k-\tau_k}) \rangle &= \langle \omega_k - \omega_k^*, \bar{g}(\theta_k, \omega_k) \rangle + \langle \omega_k - \omega_k^*, g(x_{(k)}, \omega_{k-\tau_k}) - g(x_{(k)}, \omega_k) \rangle \\
&\quad + \langle \omega_k - \omega_k^*, g(x_{(k)}, \omega_k) - \bar{g}(\theta_k, \omega_k) \rangle.
\end{aligned} \tag{43}$$

We first bound $\langle \omega_k - \omega_k^*, \bar{g}(\theta_k, \omega_k) \rangle$ in (43) as

$$\begin{aligned}
\langle \omega_k - \omega_k^*, \bar{g}(\theta_k, \omega_k) \rangle &= \langle \omega_k - \omega_k^*, \bar{g}(\theta_k, \omega_k) - \bar{g}(\theta_k, \omega_k^*) \rangle \\
&= \left\langle \omega_k - \omega_k^*, \mathbb{E} \left[(\gamma \phi(s') - \phi(s))^\top (\omega_k - \omega_k^*) \phi(s) \right] \right\rangle \\
&= \left\langle \omega_k - \omega_k^*, \mathbb{E} \left[\phi(s) (\gamma \phi(s') - \phi(s))^\top \right] (\omega_k - \omega_k^*) \right\rangle \\
&= \left\langle \omega_k - \omega_k^*, A_{\pi_{\theta_k}} (\omega_k - \omega_k^*) \right\rangle \\
&\leq -\lambda \|\omega_k - \omega_k^*\|_2^2,
\end{aligned} \tag{44}$$

where the first equality is due to $\bar{g}(\theta, \omega_\theta^*) = A_{\theta, \phi} \omega_\theta^* + b = 0$, and the last inequality follows Assumption 1.

We then bound the term $\langle \omega_k - \omega_k^*, g(x_{(k)}, \omega_{k-\tau_k}) - g(x_{(k)}, \omega_k) \rangle$ in (43) as

$$\begin{aligned}
\langle \omega_k - \omega_k^*, g(x_{(k)}, \omega_{k-\tau_k}) - g(x_{(k)}, \omega_k) \rangle &= \left\langle \omega_k - \omega_k^*, \left(\gamma \phi(s'_{(k)}) - \phi(s_{(k)}) \right)^\top (\omega_{k-\tau_k} - \omega_k) \phi(s_{(k)}) \right\rangle \\
&\leq (1 + \gamma) \|\omega_k - \omega_k^*\|_2 \|\omega_{k-\tau_k} - \omega_k\|_2 \\
&\leq (1 + \gamma) \|\omega_k - \omega_k^*\|_2 \left\| \sum_{i=k-\tau_k}^{k-1} (\omega_{i+1} - \omega_i) \right\|_2 \\
&\leq (1 + \gamma) \|\omega_k - \omega_k^*\|_2 \sum_{i=k-\tau_k}^{k-1} \beta \|g(x_i, \omega_{i-\tau_i})\|_2 \\
&\leq 2C_\delta K_0 \beta \|\omega_k - \omega_k^*\|_2,
\end{aligned} \tag{45}$$

where the last inequality follows the definition of C_δ in (40). Substituting (45) and (44) into (43) gives

$$\begin{aligned}
\langle \omega_k - \omega_k^*, g(x_{(k)}, \omega_{k-\tau_k}) \rangle &\leq -\lambda \|\omega_k - \omega_k^*\|_2^2 + 2C_\delta K_0 \beta \|\omega_k - \omega_k^*\|_2 \\
&\quad + \langle \omega_k - \omega_k^*, g(x_{(k)}, \omega_k) - \bar{g}(\theta_k, \omega_k) \rangle.
\end{aligned} \tag{46}$$

Next we jointly bound the third and fourth term in (42) as

$$\begin{aligned}
 & \langle \omega_k - \omega_k^*, \omega_k^* - \omega_{k+1}^* \rangle + \|\omega_k^* - \omega_{k+1}^*\|_2^2 \\
 & \leq \|\omega_k - \omega_k^*\|_2 \|\omega_k^* - \omega_{k+1}^*\|_2 + \|\omega_k^* - \omega_{k+1}^*\|_2^2 \\
 & \leq 2L_\omega \|\omega_k - \omega_k^*\|_2 \|\theta_k - \theta_{k+1}\|_2 + 2L_\omega^2 \|\theta_k - \theta_{k+1}\|_2^2 \\
 & \leq 2L_\omega C_p \alpha \|\omega_k - \omega_k^*\|_2 + 2L_\omega^2 C_p^2 \alpha^2,
 \end{aligned} \tag{47}$$

where constant $C_p := C_\delta C_\psi + \lambda C_\psi$. The second inequality is due to the L_ω -Lipschitz continuity of ω_θ^* shown in Proposition 2, and the last inequality follows the fact that

$$\|\theta_k - \theta_{k+1}\|_2 = \alpha \|\hat{\delta}(x_{(k)}, \omega_{k-\tau_k}) \psi_{\theta_{k-\tau_k}}(s_{(k)}, a_{(k)}) + \lambda \psi_{\theta_{k-\tau_k}}(x_{(k)}^p)\|_2 \leq \alpha C_p. \tag{48}$$

Substituting (46) and (47) into (42), and taking expectation on both sides yield

$$\begin{aligned}
 \mathbb{E} \|\omega_{k+1} - \omega_{k+1}^*\|_2^2 & \leq (1 - 2\lambda\beta) \mathbb{E} \|\omega_k - \omega_k^*\|_2^2 + 2\beta(C_1 \frac{\alpha}{\beta} + C_2 K_0 \beta) \mathbb{E} \|\omega_k - \omega_k^*\|_2 \\
 & \quad + 2\beta \mathbb{E} \langle \omega_k - \omega_k^*, g(x_{(k)}, \omega_k) - \bar{g}(\theta_k, \omega_k) \rangle + C_q \beta^2,
 \end{aligned} \tag{49}$$

where $C_1 := L_\omega C_p$, $C_2 := 2C_\delta$ and $C_q := 2C_\delta^2 + 2L_\omega^2 C_p^2 \frac{\alpha^2}{\beta^2}$.

For brevity, we use $x \sim \theta$ to denote $s \sim \mu_\theta$, $a \sim \pi_\theta$ and $s' \sim \mathcal{P}$ in this proof. Consider the third term in (49) conditioned on $\theta_k, \omega_k, \theta_{k-\tau_k}$. We bound it as

$$\begin{aligned}
 & \mathbb{E} [\langle \omega_k - \omega_k^*, g(x_{(k)}, \omega_k) - \bar{g}(\theta_k, \omega_k) \rangle | \theta_k, \omega_k, \theta_{k-\tau_k}] \\
 & = \langle \omega_k - \omega_k^*, \mathbb{E}_{x_{(k)} \sim \theta_{k-\tau_k}} [g(x_{(k)}, \omega_k) | \omega_k] - \bar{g}(\theta_k, \omega_k) \rangle \\
 & = \langle \omega_k - \omega_k^*, \bar{g}(\theta_{k-\tau_k}, \omega_k) - \bar{g}(\theta_k, \omega_k) \rangle \\
 & \leq \|\omega_k - \omega_k^*\|_2 \|\bar{g}(\theta_{k-\tau_k}, \omega_k) - \bar{g}(\theta_k, \omega_k)\|_2 \\
 & \leq 2R_\omega \left\| \mathbb{E}_{x \sim \theta_{k-\tau_k}} [g(x, \omega_k)] - \mathbb{E}_{x \sim \theta_k} [g(x, \omega_k)] \right\|_2 \\
 & \leq 2R_\omega \sup_x \|g(x, \omega_k)\|_2 \left\| \mu_{\theta_{k-\tau_k}} \otimes \pi_{\theta_{k-\tau_k}} \otimes \mathcal{P} - \mu_{\theta_k} \otimes \pi_{\theta_k} \otimes \mathcal{P} \right\|_{TV} \\
 & \leq 4R_\omega C_\delta d_{TV}(\mu_{\theta_{k-\tau_k}} \otimes \pi_{\theta_{k-\tau_k}} \otimes \mathcal{P}, \mu_{\theta_k} \otimes \pi_{\theta_k} \otimes \mathcal{P}),
 \end{aligned} \tag{50}$$

where second last inequality follows the definition of TV norm and the last inequality uses the definition of C_δ in (40).

Define constant $C_3 := 2R_\omega C_\delta |\mathcal{A}| L_\pi (1 + \log_\rho \kappa^{-1} + (1 - \rho)^{-1})$. Then by following the third item in (Wu et al., 2020, Lemma A.1), we can write (50) as

$$\begin{aligned}
 & \mathbb{E} [\langle \omega_k - \omega_k^*, g(x_{(k)}, \omega_k) - \bar{g}(\theta_k, \omega_k) \rangle | \theta_k, \omega_k, \theta_{k-\tau_k}] \\
 & \leq 4R_\omega C_\delta d_{TV}(\mu_{\theta_{k-\tau_k}} \otimes \pi_{\theta_{k-\tau_k}} \otimes \mathcal{P}, \mu_{\theta_k} \otimes \pi_{\theta_k} \otimes \mathcal{P}) \\
 & \leq C_3 \|\theta_{k-\tau_k} - \theta_k\|_2 \\
 & \leq C_3 \sum_{i=k-\tau_k}^{k-1} \alpha \|g(x_i, \omega_{i-\tau_i})\|_2 \\
 & \leq C_3 C_\delta K_0 \alpha,
 \end{aligned} \tag{51}$$

Taking total expectation on both sides of (51) and substituting it into (49) yield

$$\begin{aligned} \mathbb{E} \|\omega_{k+1} - \omega_{k+1}^*\|_2^2 &\leq (1 - 2\lambda\beta) \mathbb{E} \|\omega_k - \omega_k^*\|_2^2 + 2\beta \left(C_1 \frac{\alpha}{\beta} + C_2 K_0 \beta \right) \mathbb{E} \|\omega_k - \omega_k^*\|_2 \\ &\quad + 2C_3 C_\delta K_0 \beta \alpha + C_q \beta^2. \end{aligned} \quad (52)$$

which along with the fact $\alpha = \frac{1}{(K+1)^{\frac{3}{5}}}$ and $\beta = \frac{1}{(K+1)^{\frac{2}{5}}}$ implies

$$\frac{1}{K} \sum_{k=1}^K \mathbb{E} \|\omega_k - \omega_k^*\|_2^2 = \mathcal{O} \left(\frac{K_0^2}{K^{\frac{4}{5}}} \right) + \mathcal{O} \left(\frac{K_0}{K^{\frac{3}{5}}} \right) + \mathcal{O} \left(\frac{1}{K^{\frac{2}{5}}} \right). \quad (53)$$

This completes the proof. ■

B.2. Proof of Theorem 2

We first define the ‘optimal’ TD target as:

$$\delta(x, \theta) := r(s, a, s') + \gamma V_{\pi_\theta}(s') - V_{\pi_\theta}(s).$$

The update in Algorithm 1 can be written as:

$$\theta_{k+1} = \theta_k + \alpha \left(\hat{\delta}(\hat{x}_{(k)}, \omega_{k-\tau_k}) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) + \lambda \psi_{\theta_{k-\tau_k}}(x_{(k)}^p) \right). \quad (54)$$

For brevity, we use ω_k^* as shorthand notation of $\omega_{\theta_k}^*$ in this proof. Then we are ready to give the convergence proof.

Proof From L_λ -Lipschitz continuity of regularized policy gradient shown in Proposition 1, we have:

$$\begin{aligned} J_\lambda(\theta_{k+1}) &\geq J_\lambda(\theta_k) + \langle \nabla J_\lambda(\theta_k), \theta_{k+1} - \theta_k \rangle - \frac{L_\lambda}{2} \|\theta_{k+1} - \theta_k\|_2^2 \\ &= J_\lambda(\theta_k) + \alpha \left\langle \nabla J_\lambda(\theta_k), \left(\hat{\delta}(\hat{x}_{(k)}, \omega_{k-\tau_k}) - \hat{\delta}(\hat{x}_{(k)}, \omega_k^*) \right) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) \right\rangle \\ &\quad + \alpha \left\langle \nabla J_\lambda(\theta_k), \hat{\delta}(\hat{x}_{(k)}, \omega_k^*) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) + \lambda \psi_{\theta_{k-\tau_k}}(x_{(k)}^p) \right\rangle - \frac{L_\lambda}{2} \|\theta_{k+1} - \theta_k\|_2^2 \\ &\geq J_\lambda(\theta_k) + \alpha \left\langle \nabla J_\lambda(\theta_k), \left(\hat{\delta}(\hat{x}_{(k)}, \omega_{k-\tau_k}) - \hat{\delta}(\hat{x}_{(k)}, \omega_k^*) \right) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) \right\rangle \\ &\quad + \alpha \left\langle \nabla J_\lambda(\theta_k), \hat{\delta}(\hat{x}_{(k)}, \omega_k^*) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) + \lambda \psi_{\theta_{k-\tau_k}}(x_{(k)}^p) \right\rangle - \frac{L_\lambda}{2} C_p^2 \alpha^2, \end{aligned}$$

where the last inequality follows the definition of C_p in (48).

Taking expectation on both sides of the last inequality yields

$$\begin{aligned} \mathbb{E}[J_\lambda(\theta_{k+1})] &\geq \mathbb{E}[J_\lambda(\theta_k)] + \underbrace{\alpha \mathbb{E} \left\langle \nabla J_\lambda(\theta_k), \left(\hat{\delta}(\hat{x}_{(k)}, \omega_{k-\tau_k}) - \hat{\delta}(\hat{x}_{(k)}, \omega_k^*) \right) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) \right\rangle}_{I_1} \\ &\quad + \underbrace{\alpha \mathbb{E} \left\langle \nabla J_\lambda(\theta_k), \hat{\delta}(\hat{x}_{(k)}, \omega_k^*) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) + \lambda \nabla R(\theta_{k-\tau_k}) \right\rangle}_{I_2} - \frac{L_\lambda}{2} C_p^2 \alpha^2. \end{aligned} \quad (55)$$

where we used the fact that $\mathbb{E}[\psi_{\theta_{k-\tau_k}}(x_{(k)}^p) | \theta_{k-\tau_k}] = \nabla R(\theta_{k-\tau_k})$.

We first decompose I_1 as

$$\begin{aligned}
 I_1 &= \mathbb{E} \left\langle \nabla J_\lambda(\theta_k), \left(\hat{\delta}(\hat{x}_{(k)}, \omega_{k-\tau_k}) - \hat{\delta}(\hat{x}_{(k)}, \omega_k^*) \right) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) \right\rangle \\
 &= \underbrace{\mathbb{E} \left\langle \nabla J_\lambda(\theta_k), \left(\hat{\delta}(\hat{x}_{(k)}, \omega_{k-\tau_k}) - \hat{\delta}(\hat{x}_{(k)}, \omega_k) \right) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) \right\rangle}_{I_1^{(1)}} \\
 &\quad + \underbrace{\mathbb{E} \left\langle \nabla J_\lambda(\theta_k), \left(\hat{\delta}(\hat{x}_{(k)}, \omega_k) - \hat{\delta}(\hat{x}_{(k)}, \omega_k^*) \right) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) \right\rangle}_{I_1^{(2)}}. \tag{56}
 \end{aligned}$$

We bound $I_1^{(1)}$ as

$$\begin{aligned}
 I_1^{(1)} &= \mathbb{E} \left\langle \nabla J_\lambda(\theta_k), \left(\gamma \phi(\hat{s}'_{(k)}) - \phi(\hat{s}_{(k)}) \right)^\top (\omega_{k-\tau_k} - \omega_k) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) \right\rangle \\
 &\geq -\mathbb{E} \left[\|\nabla J_\lambda(\theta_k)\|_2 \|\gamma \phi(\hat{s}'_{(k)}) - \phi(\hat{s}_{(k)})\|_2 \|\omega_k - \omega_{k-\tau_k}\|_2 \|\psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)})\|_2 \right] \\
 &\geq -(1 + \gamma) C_\psi \mathbb{E} [\|\nabla J_\lambda(\theta_k)\|_2 \|\omega_k - \omega_{k-\tau_k}\|_2] \\
 &\geq -(1 + \gamma) C_\psi C_\delta K_0 \beta \mathbb{E} \|\nabla J_\lambda(\theta_k)\|_2, \tag{57}
 \end{aligned}$$

where the last inequality follows

$$\begin{aligned}
 \|\omega_k - \omega_{k-\tau_k}\|_2 &= \left\| \sum_{i=k-\tau_k}^{k-1} (\omega_{i+1} - \omega_i) \right\|_2 \\
 &\leq \sum_{i=k-\tau_k}^{k-1} \|\beta g(x_i, \omega_{i-\tau_i})\|_2 \\
 &\leq \beta K_0 C_\delta, \tag{58}
 \end{aligned}$$

where the second inequality is due to (40).

Then we bound $I_1^{(2)}$ as

$$\begin{aligned}
 I_1^{(2)} &= \mathbb{E} \left\langle \nabla J_\lambda(\theta_k), \left(\hat{\delta}(\hat{x}_{(k)}, \omega_k) - \hat{\delta}(\hat{x}_{(k)}, \omega_k^*) \right) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) \right\rangle \\
 &= -\mathbb{E} \left\langle \nabla J_\lambda(\theta_k), \left(\gamma \phi(\hat{s}'_{(k)}) - \phi(\hat{s}_{(k)}) \right)^\top (\omega_k^* - \omega_k) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) \right\rangle \\
 &\geq -\mathbb{E} \left[\|\nabla J_\lambda(\theta_k)\|_2 \|\gamma \phi(\hat{s}'_{(k)}) - \phi(\hat{s}_{(k)})\|_2 \|\omega_k - \omega_k^*\|_2 \|\psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)})\|_2 \right] \\
 &\geq -(1 + \gamma) C_\psi \mathbb{E} [\|\nabla J_\lambda(\theta_k)\|_2 \|\omega_k - \omega_k^*\|_2]. \tag{59}
 \end{aligned}$$

Collecting lower bounds of $I_1^{(1)}$ and $I_1^{(2)}$ gives

$$\begin{aligned}
 I_1 &\geq -2C_\psi \mathbb{E} \left[\|\nabla J_\lambda(\theta_k)\|_2 (C_\delta K_0 \beta + \|\omega_k - \omega_k^*\|_2) \right] \\
 &\geq -\frac{1}{2} \mathbb{E} \|\nabla J_\lambda(\theta_k)\|_2^2 - 2C_\psi^2 \mathbb{E} [(C_\delta K_0 \beta + \|\omega_k - \omega_k^*\|_2)^2] \\
 &\geq -\frac{1}{2} \mathbb{E} \|\nabla J_\lambda(\theta_k)\|_2^2 - 4C_\psi^2 C_\delta^2 K_0^2 \beta^2 - 4C_\psi^2 \mathbb{E} \|\omega_k - \omega_k^*\|_2^2, \tag{60}
 \end{aligned}$$

where the the second and third inequality follow Young's inequality.

Now we consider I_2 . We first decompose I_2 as

$$\begin{aligned}
I_2 &= \mathbb{E} \left\langle \nabla J_\lambda(\theta_k), \hat{\delta}(\hat{x}_{(k)}, \omega_k^*) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) + \lambda \nabla R(\theta_{k-\tau_k}) \right\rangle \\
&= \underbrace{\mathbb{E} \left\langle \nabla J_\lambda(\theta_k), \left(\hat{\delta}(\hat{x}_{(k)}, \omega_k^*) - \hat{\delta}(\hat{x}_{(k)}, \omega_{k-\tau_k}^*) \right) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) \right\rangle}_{I_2^{(1)}} \\
&\quad + \underbrace{\mathbb{E} \left\langle \nabla J_\lambda(\theta_k), \left(\hat{\delta}(\hat{x}_{(k)}, \omega_{k-\tau_k}^*) - \delta(\hat{x}_{(k)}, \theta_{k-\tau_k}) \right) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) \right\rangle}_{I_2^{(2)}} \\
&\quad + \underbrace{\mathbb{E} \left\langle \nabla J_\lambda(\theta_k), \delta(\hat{x}_{(k)}, \theta_{k-\tau_k}) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) + \lambda \nabla R(\theta_{k-\tau_k}) - \nabla J_\lambda(\theta_k) \right\rangle}_{I_2^{(3)}} + \|\nabla J_\lambda(\theta_k)\|_2^2.
\end{aligned} \tag{61}$$

We bound $I_2^{(1)}$ as

$$\begin{aligned}
I_2^{(1)} &= \mathbb{E} \left\langle \nabla J_\lambda(\theta_k), \left(\hat{\delta}(\hat{x}_{(k)}, \omega_k^*) - \hat{\delta}(\hat{x}_{(k)}, \omega_{k-\tau_k}^*) \right) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) \right\rangle \\
&= \mathbb{E} \left\langle \nabla J_\lambda(\theta_k), \left(\gamma \phi(\hat{s}'_{(k)}) - \phi(\hat{s}_{(k)}) \right)^\top (\omega_k^* - \omega_{k-\tau_k}^*) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) \right\rangle \\
&\geq -\mathbb{E} \left[\|\nabla J_\lambda(\theta_k)\|_2 \left\| \left(\gamma \phi(\hat{s}'_{(k)}) - \phi(\hat{s}_{(k)}) \right)^\top \right\|_2 \|\omega_k^* - \omega_{k-\tau_k}^*\|_2 \|\psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)})\|_2 \right] \\
&\geq -L_V C_\psi (1 + \gamma) \mathbb{E} \|\omega_k^* - \omega_{k-\tau_k}^*\|_2 \\
&\geq -L_V L_\omega C_\psi (1 + \gamma) \mathbb{E} \|\theta_k - \theta_{k-\tau_k}\|_2 \\
&\geq -L_V L_\omega C_\psi C_p (1 + \gamma) K_0 \alpha,
\end{aligned} \tag{62}$$

where $L_V := \frac{r_{\max}}{1-\gamma} C_\psi + \lambda C_\psi$ is the trivial upper bound of $\|\nabla J_\lambda(\theta)\|_2$ and $\|\nabla J(\theta)\|_2$. The second last inequality follows from Proposition 2 and the last inequality uses (48) as

$$\begin{aligned}
\|\theta_k - \theta_{k-\tau_k}\|_2 &\leq \sum_{i=k-\tau_k}^{k-1} \|\theta_{i+1} - \theta_i\|_2 \\
&= \sum_{i=k-\tau_k}^{k-1} \alpha \|\hat{\delta}(\hat{x}_i, \omega_{i-\tau_i}) \psi_{\theta_{i-\tau_i}}(\hat{s}_i, \hat{a}_i)\|_2 \\
&\leq \sum_{i=k-\tau_k}^{k-1} \alpha C_p \leq C_p K_0 \alpha.
\end{aligned} \tag{63}$$

We bound $I_2^{(2)}$ as

$$\begin{aligned}
 I_2^{(2)} &= \mathbb{E} \left\langle \nabla J_\lambda(\theta_k), \left(\hat{\delta}(\hat{x}_{(k)}, \omega_{k-\tau_k}^*) - \delta(\hat{x}_{(k)}, \theta_{k-\tau_k}) \right) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) \right\rangle \\
 &\geq -\mathbb{E} \left[\|\nabla J_\lambda(\theta_k)\|_2 \left| \hat{\delta}(\hat{x}_{(k)}, \omega_{k-\tau_k}^*) - \delta(\hat{x}_{(k)}, \theta_{k-\tau_k}) \right| \|\psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)})\|_2 \right] \\
 &\geq -L_V C_\psi \mathbb{E} \left| \hat{\delta}(\hat{x}_{(k)}, \omega_{k-\tau_k}^*) - \delta(\hat{x}_{(k)}, \theta_{k-\tau_k}) \right| \\
 &= -L_V C_\psi \mathbb{E} \left| \gamma \left(\phi(\hat{s}'_{(k)})^\top \omega_{k-\tau_k}^* - V_{\pi_{\theta_{k-\tau_k}}}(\hat{s}'_{(k)}) \right) + V_{\pi_{\theta_{k-\tau_k}}}(\hat{s}_{(k)}) - \phi(\hat{s}_{(k)})^\top \omega_{k-\tau_k}^* \right| \\
 &\geq -L_V C_\psi \left(\gamma \mathbb{E} \left| \phi(\hat{s}'_{(k)})^\top \omega_{k-\tau_k}^* - V_{\pi_{\theta_{k-\tau_k}}}(\hat{s}'_{(k)}) \right| + \mathbb{E} \left| V_{\pi_{\theta_{k-\tau_k}}}(\hat{s}_{(k)}) - \phi(\hat{s}_{(k)})^\top \omega_{k-\tau_k}^* \right| \right) \\
 &\geq -L_V C_\psi \left(\gamma \sqrt{\mathbb{E} \left| \phi(\hat{s}'_{(k)})^\top \omega_{k-\tau_k}^* - V_{\pi_{\theta_{k-\tau_k}}}(\hat{s}'_{(k)}) \right|^2} + \sqrt{\mathbb{E} \left| V_{\pi_{\theta_{k-\tau_k}}}(\hat{s}_{(k)}) - \phi(\hat{s}_{(k)})^\top \omega_{k-\tau_k}^* \right|^2} \right) \\
 &\geq -L_V C_\psi (1 + \gamma) \epsilon_{\text{app}}.
 \end{aligned} \tag{64}$$

We bound $I_2^{(3)}$ as

$$\begin{aligned}
 I_2^{(3)} &= \mathbb{E} \left\langle \nabla J_\lambda(\theta_k), \delta(\hat{x}_{(k)}, \theta_{k-\tau_k}) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) + \lambda \nabla R(\theta_{k-\tau_k}) - \nabla J_\lambda(\theta_k) \right\rangle \\
 &= \mathbb{E} \left\langle \nabla J_\lambda(\theta_k), \mathbb{E} \left[\delta(\hat{x}_{(k)}, \theta_{k-\tau_k}) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) \middle| \theta_{k-\tau_k}, \theta_k \right] + \lambda \nabla R(\theta_{k-\tau_k}) - \nabla J_\lambda(\theta_k) \right\rangle \\
 &= \mathbb{E} \left\langle \nabla J_\lambda(\theta_k), \mathbb{E}_{\substack{\hat{s}_{(k)} \sim d_{\theta_{k-\tau_k}} \\ \hat{a}_{(k)} \sim \pi_{\theta_{k-\tau_k}}}} \left[A_{\pi_{\theta_{k-\tau_k}}}(\hat{s}_{(k)}, \hat{a}_{(k)}) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) \right] + \lambda \nabla R(\theta_{k-\tau_k}) - \nabla J_\lambda(\theta_k) \right\rangle,
 \end{aligned} \tag{65}$$

where we used the fact that

$$\begin{aligned}
 &\mathbb{E} \left[\delta(\hat{x}_{(k)}, \theta_{k-\tau_k}) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) \middle| \theta_{k-\tau_k}, \theta_k \right] \\
 &= \mathbb{E}_{\substack{\hat{s}_{(k)}, \hat{a}_{(k)} \sim d_{\theta_{k-\tau_k}} \\ \hat{s}'_{(k)} \sim \mathcal{P}}} \left[\left(r(\hat{s}_{(k)}, \hat{a}_{(k)}, \hat{s}'_{(k)}) + \gamma V_{\pi_{\theta_{k-\tau_k}}}(\hat{s}'_{(k)}) - V_{\pi_{\theta_{k-\tau_k}}}(\hat{s}_{(k)}) \right) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) \middle| \theta_{k-\tau_k}, \theta_k \right] \\
 &= \mathbb{E}_{d_{\theta_{k-\tau_k}}} \left[\left(\mathbb{E}_{\hat{s}'_{(k)} \sim \mathcal{P}} \left[r(\hat{s}_{(k)}, \hat{a}_{(k)}, \hat{s}'_{(k)}) + \gamma V_{\pi_{\theta_{k-\tau_k}}}(\hat{s}'_{(k)}) \right] - V_{\pi_{\theta_{k-\tau_k}}}(\hat{s}_{(k)}) \right) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) \middle| \theta_{k-\tau_k}, \theta_k \right] \\
 &= \mathbb{E}_{d_{\theta_{k-\tau_k}}} \left[\left(Q_{\pi_{\theta_{k-\tau_k}}}(\hat{s}_{(k)}, \hat{a}_{(k)}) - V_{\pi_{\theta_{k-\tau_k}}}(\hat{s}_{(k)}) \right) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) \middle| \theta_{k-\tau_k}, \theta_k \right] \\
 &= \mathbb{E}_{d_{\theta_{k-\tau_k}}} \left[A_{\pi_{\theta_{k-\tau_k}}}(\hat{s}_{(k)}, \hat{a}_{(k)}) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) \middle| \theta_{k-\tau_k}, \theta_k \right].
 \end{aligned}$$

Continuing from (65),

$$\begin{aligned}
 I_2^{(3)} &= \left\langle \nabla J_\lambda(\theta_k), \nabla J_\lambda(\theta_{k-\tau_k}) - \nabla J_\lambda(\theta_k) \right\rangle \\
 &\geq -\|\nabla J_\lambda(\theta_k)\|_2 \|\nabla J_\lambda(\theta_{k-\tau_k}) - \nabla J_\lambda(\theta_k)\|_2 \\
 &\geq -L_V L_\lambda \|\theta_{k-\tau_k} - \theta_k\|_2 \geq -L_V L_\lambda C_p K_0 \alpha,
 \end{aligned} \tag{66}$$

where the second last inequality is due to L_λ -Lipschitz continuity of policy gradient shown in Proposition 1, and the last inequality follows (63).

Collecting lower bounds of $I_2^{(1)}$, $I_2^{(2)}$ and $I_2^{(3)}$ gives

$$I_2 \geq -D_1 K_0 \alpha - L_V C_\psi (1 + \gamma) \epsilon_{\text{app}}, \quad (67)$$

where constant $D_1 := L_V L_\omega C_\psi C_p (1 + \gamma) + L_V L_\lambda C_p$.

Substituting (60) and (67) into (55) yields

$$\begin{aligned} \mathbb{E}[J_\lambda(\theta_{k+1})] &\geq \mathbb{E}[J_\lambda(\theta_k)] + \frac{\alpha}{2} \mathbb{E} \|\nabla J_\lambda(\theta_k)\|_2^2 - 4C_\psi^2 \alpha \mathbb{E} \|\omega_k - \omega_k^*\|_2^2 - 4C_\psi^2 C_\delta^2 K_0^2 \alpha \beta^2 \\ &\quad - 2L_V C_\psi \epsilon_{\text{app}} \alpha - (D_1 K_0 + \frac{L_\lambda}{2} C_p^2) \alpha^2. \end{aligned} \quad (68)$$

After telescoping, we have

$$\begin{aligned} \sum_{k=1}^K \frac{1}{2} \mathbb{E} \|\nabla J_\lambda(\theta_k)\|_2^2 &\leq \frac{1}{\alpha} (J^* - J_\lambda(\theta_{K_0})) + 4C_\psi^2 \sum_{k=1}^K \mathbb{E} \|\omega_k - \omega_k^*\|_2^2 + 4K C_\psi^2 C_\delta^2 K_0^2 \beta^2 \\ &\quad + 2KL_V C_\psi \epsilon_{\text{app}} + K(D_1 K_0 + \frac{L_\lambda}{2} C_p^2) \alpha. \end{aligned} \quad (69)$$

Select $\alpha = K^{-\frac{3}{5}}$ and $\beta = K^{-\frac{2}{5}}$, we have

$$\frac{1}{K} \sum_{k=1}^K \mathbb{E} \|\nabla J_\lambda(\theta_k)\|_2^2 = \mathcal{O}\left(\frac{1}{K^{\frac{2}{5}}}\right) + \mathcal{O}\left(\frac{1}{K} \sum_{k=1}^K \mathbb{E} \|\omega_k - \omega_k^*\|_2^2\right) + \mathcal{O}\left(\frac{K_0^2}{K^{\frac{4}{5}}}\right) + \mathcal{O}\left(\frac{K_0}{K^{\frac{3}{5}}}\right) + \mathcal{O}(\epsilon_{\text{app}}). \quad (70)$$

This completes the proof. $\blacksquare$

B.3. Proof of Theorem 3

Given the definition in Section B.1, we now give the convergence proof of critic update in Algorithm 1 with linear function approximation and Markovian sampling.

Proof By following the derivation of (49), we have

$$\begin{aligned} \mathbb{E} \|\omega_{k+1} - \omega_{k+1}^*\|_2^2 &\leq (1 - 2\lambda\beta) \mathbb{E} \|\omega_k - \omega_k^*\|_2^2 + 2\beta \left(C_1 \frac{\alpha}{\beta} + C_2 K_0 \beta \right) \mathbb{E} \|\omega_k - \omega_k^*\|_2 \\ &\quad + 2\beta \mathbb{E} \langle \omega_k - \omega_k^*, g(x_{(k)}, \omega_k) - \bar{g}(\theta_k, \omega_k) \rangle + C_q \beta^2, \end{aligned} \quad (71)$$

where $C_1 := C_p L_\omega$, $C_2 := C_\delta (1 + \gamma)$ and $C_q := 2C_\delta^2 + 2L_\omega^2 C_p^2 \max(k) \frac{\alpha^2}{\beta^2}$.

Now we consider the third item in the last inequality. For some $m \in \mathbb{N}^+$, we define $M := (K_0 + 1)m + K_0$. Following Lemma 4 (to be presented in Section C.1), for some $d_m \leq M$ and positive constants C_4, C_5, C_6, C_7 , we have

$$\begin{aligned} &\mathbb{E} \langle \omega_k - \omega_k^*, g(x_{(k)}, \omega_k) - \bar{g}(\theta_k, \omega_k) \rangle \\ &\leq C_4 \mathbb{E} \|\theta_k - \theta_{k-d_m}\|_2 + C_5 \sum_{i=\tau_k}^{d_m} \mathbb{E} \|\theta_{k-i} - \theta_{k-d_m}\|_2 + C_6 \mathbb{E} \|\omega_k - \omega_{k-d_m}\|_2 + C_7 \kappa \rho^{m-1} \\ &\leq C_4 \sum_{i=k-d_m}^{k-1} \mathbb{E} \|\theta_{i+1} - \theta_i\|_2 + C_5 \sum_{i=\tau_k}^{d_m-1} \sum_{j=k-d_m}^{k-i-1} \mathbb{E} \|\theta_{j+1} - \theta_j\|_2 + C_6 \sum_{i=k-d_m}^{k-1} \mathbb{E} \|\omega_{i+1} - \omega_i\|_2 + C_7 \kappa \rho^{m-1} \\ &\leq C_4 d_m C_p \alpha + C_5 (d_m - \tau_k)^2 C_p \alpha + C_6 d_m C_\delta \beta + C_7 \kappa \rho^{m-1} \\ &\leq (C_4 M + C_5 M^2) C_p \alpha + C_6 M C_\delta \beta + C_7 \kappa \rho^{m-1}, \end{aligned} \quad (72)$$

where the last inequality is due to $\tau_k \geq 0$ and $d_m \leq M$.

Further letting $m = m_K$ which is defined in (27) yields

$$\begin{aligned} & \mathbb{E} \langle \omega_k - \omega_k^*, g(x_{(k)}, \omega_k) - \bar{g}(\theta_k, \omega_k) \rangle \\ &= (C_4 M_K + C_5 M_K^2) C_p \alpha + C_6 C_\delta M_K \beta + C_7 \kappa \rho^{m_K-1} \\ &\leq (C_4 M_K + C_5 M_K^2) C_p \alpha + C_6 C_\delta M_K \beta + C_7 \alpha, \end{aligned} \quad (73)$$

where $M_K = (K_0 + 1)m_K + K_0$, and the last inequality follows from $m_K = \mathcal{O}(\log K)$.

Substituting (73) into (71) gives

$$\begin{aligned} \mathbb{E} \|\omega_{k+1} - \omega_{k+1}^*\|_2^2 &\leq (1 - 2\lambda\beta) \mathbb{E} \|\omega_k - \omega_k^*\|_2^2 + 2\beta \left(C_1 \frac{\alpha}{\beta} + C_2 K_0 \beta \right) \mathbb{E} \|\omega_k - \omega_k^*\|_2 \\ &\quad + 2\beta \left((C_4 M_K + C_5 M_K^2) C_p \alpha + C_6 C_\delta M_K \beta + C_7 \alpha \right) + C_q \beta^2. \end{aligned} \quad (74)$$

Select $\alpha = K^{-\frac{3}{5}}$ and $\beta = K^{-\frac{2}{5}}$. After telescoping, we have

$$\frac{1}{K} \sum_{k=1}^K \mathbb{E} \|\omega_k - \omega_k^*\|_2^2 = \mathcal{O} \left(\frac{1}{K^{\frac{2}{5}}} \right) + \mathcal{O} \left(\frac{K_0^2 \log^2 K}{K^{\frac{3}{5}}} \right) + \mathcal{O} \left(\frac{K_0 \log K}{K^{\frac{2}{5}}} \right).$$

This completes the proof. ■

B.4. Proof of Theorem 4

Given the definition in section B.2, we now give the convergence proof of actor update in Algorithm 1 with linear value function approximation and Markovian sampling method.

Proof By following the derivation of (55), we have

$$\begin{aligned} \mathbb{E}[J_\lambda(\theta_{k+1})] &\geq \mathbb{E}[J_\lambda(\theta_k)] + \underbrace{\alpha \mathbb{E} \left\langle \nabla J_\lambda(\theta_k), \left(\hat{\delta}(\hat{x}_{(k)}, \omega_{k-\tau_k}) - \hat{\delta}(\hat{x}_{(k)}, \omega_k^*) \right) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) \right\rangle}_{I_1} \\ &\quad + \underbrace{\alpha \mathbb{E} \left\langle \nabla J_\lambda(\theta_k), \hat{\delta}(\hat{x}_{(k)}, \omega_k^*) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) + \lambda \nabla R(\theta_{k-\tau_k}) \right\rangle}_{I_2} - \frac{L_\lambda}{2} C_p^2 \alpha^2. \end{aligned} \quad (75)$$

The item I_1 can be bounded by following (60) as

$$I_1 \geq -\frac{1}{2} \mathbb{E} \|\nabla J_\lambda(\theta_k)\|_2^2 - 4C_\psi^2 C_\delta^2 K_0^2 \beta^2 - 4C_\psi^2 \mathbb{E} \|\omega_k - \omega_k^*\|_2^2. \quad (76)$$

Next we consider I_2 . We first decompose it as

$$\begin{aligned}
I_2 &= \mathbb{E} \left\langle \nabla J_\lambda(\theta_k), \hat{\delta}(x_{(k)}, \omega_k^*) \psi_{\theta_{k-\tau_k}}(s_{(k)}, a_{(k)}) + \lambda \nabla R(\theta_{k-\tau_k}) \right\rangle \\
&= \underbrace{\mathbb{E} \left\langle \nabla J_\lambda(\theta_k), \left(\hat{\delta}(x_{(k)}, \omega_k^*) - \delta(x_{(k)}, \theta_k) \right) \psi_{\theta_{k-\tau_k}}(s_{(k)}, a_{(k)}) \right\rangle}_{I_2^{(1)}} \\
&\quad + \underbrace{\mathbb{E} \left\langle \nabla J_\lambda(\theta_k), \delta(x_{(k)}, \theta_k) \psi_{\theta_{k-\tau_k}}(s_{(k)}, a_{(k)}) - \nabla J(\theta_k) \right\rangle}_{I_2^{(2)}} \\
&\quad + \underbrace{\mathbb{E} \left\langle \nabla J_\lambda(\theta_k), \lambda \nabla R(\theta_{k-\tau_k}) - \lambda \nabla R(\theta_k) \right\rangle}_{I_2^{(3)}} + \mathbb{E} \|\nabla J_\lambda(\theta_k)\|_2^2. \tag{77}
\end{aligned}$$

For some $m \in \mathbb{N}^+$, define $M := (K_0 + 1)m + K_0$. Following Lemma 5, for some $d_m \leq M$ and positive constants $D_2, D_3, D_4, D_5, I_2^{(1)}$ can be bounded as

$$\begin{aligned}
I_2^{(1)} &= \mathbb{E} \left\langle \nabla J_\lambda(\theta_k), \left(\hat{\delta}(x_{(k)}, \omega_k^*) - \delta(x_{(k)}, \theta_k) \right) \psi_{\theta_{k-\tau_k}}(s_{(k)}, a_{(k)}) \right\rangle \\
&\geq -D_2 \mathbb{E} \|\theta_{k-\tau_k} - \theta_{k-d_m}\|_2 - D_3 \mathbb{E} \|\theta_k - \theta_{k-d_m}\|_2 - D_4 \sum_{i=k-d_m}^{k-\tau_k} \mathbb{E} \|\theta_i - \theta_{k-d_m}\|_2 \\
&\quad - D_5 \kappa \rho^{m-1} - L_V C_\psi (1 + \gamma) \epsilon_{\text{app}} \\
&\geq -D_2 (d_m - \tau_k) C_p \alpha - D_3 d_m C_p \alpha - D_4 (d_m - \tau_k)^2 C_p \alpha \\
&\quad - D_5 \kappa \rho^{m-1} - (1 + \gamma) L_V C_\psi \epsilon_{\text{app}}, \tag{78}
\end{aligned}$$

where the derivation of the last inequality is similar to that of (72). By setting $m = m_K$ in (78), and following the fact that $d_{m_K} \leq M_K$ and $\tau_k \geq 0$, we have

$$\begin{aligned}
I_2^{(1)} &\geq -D_2 M_K C_p \alpha - D_3 M_K C_p \alpha - D_4 M_K^2 C_p \alpha - D_5 \kappa \rho^{m_K-1} - (1 + \gamma) L_V C_\psi \epsilon_{\text{app}} \\
&= -((D_2 + D_3) C_p M_K + D_4 C_p M_K^2) \alpha - D_5 \kappa \rho^{m_K-1} - (1 + \gamma) L_V C_\psi \epsilon_{\text{app}} \\
&\geq -((D_2 + D_3) C_p M_K + D_4 C_p M_K^2) \alpha - D_5 \alpha - (1 + \gamma) L_V C_\psi \epsilon_{\text{app}}, \tag{79}
\end{aligned}$$

where the last inequality is due to the definition of m_K .

Following Lemma 6, for some positive constants D_6, D_7 and D_8 , we bound $I_2^{(2)}$ as

$$\begin{aligned}
I_2^{(2)} &= \mathbb{E} \left\langle \nabla J_\lambda(\theta_k), \delta(x_{(k)}, \theta_k) \psi_{\theta_{k-\tau_k}}(s_{(k)}, a_{(k)}) - \nabla J(\theta_k) \right\rangle \\
&\geq -D_6 \mathbb{E} \|\theta_{k-\tau_k} - \theta_{k-d_m}\|_2 - D_7 \mathbb{E} \|\theta_k - \theta_{k-d_m}\|_2 - D_8 \sum_{i=\tau_k}^{d_m} \mathbb{E} \|\theta_{k-i} - \theta_{k-d_m}\|_2 - D_9 \kappa \rho^{m-1}.
\end{aligned}$$

Similar to the derivation of (79), we have

$$I_2^{(2)} \geq -(D_6 C_p M_K + D_7 C_p M_K + D_8 C_p M_K^2) \alpha - D_9 \alpha. \tag{80}$$

Term $I_2^{(3)}$ can be bounded as

$$\begin{aligned} I_2^{(3)} &\geq -\lambda L_V \|\nabla R(\theta_k) - \nabla R(\theta_{k-\tau_k})\|_2 \geq -\lambda L_V L_\psi \|\theta_k - \theta_{k-\tau_k}\|_2 \\ &\geq -\lambda L_V L_\psi K_0 C_p \alpha. \end{aligned} \quad (81)$$

Collecting the lower bounds of $I_2^{(1)}$, $I_2^{(2)}$ and $I_2^{(3)}$ yields

$$I_2 \geq -D_K \alpha - (1 + \gamma) L_V C_\psi \epsilon_{\text{app}} + \mathbb{E} \|\nabla J_\lambda(\theta_k)\|_2^2, \quad (82)$$

where we define $D_K := C_p(D_4 + D_8)M_K^2 + C_p(D_2 + D_3 + D_6 + D_7)M_K + \lambda L_V L_\psi K_0 C_p + D_5 + D_9$ for brevity.

Substituting (76) and (82) into (75) yields

$$\begin{aligned} \mathbb{E}[J_\lambda(\theta_{k+1})] &\geq \mathbb{E}[J_\lambda(\theta_k)] + \frac{\alpha}{2} \mathbb{E} \|\nabla J_\lambda(\theta_k)\|_2^2 - 4C_\psi^2 \alpha \mathbb{E} \|\omega_k - \omega_k^*\|_2^2 - 4C_\psi^2 C_\delta^2 K_0^2 \alpha \beta^2 \\ &\quad - 2L_V C_\psi \epsilon_{\text{app}} \alpha - D_K \alpha^2. \end{aligned} \quad (83)$$

Choose step size $\alpha = K^{-\frac{3}{5}}$, $\beta = K^{-\frac{2}{5}}$. With $D_K = \mathcal{O}(M_K^2) = \mathcal{O}(K_0^2 \log^2 K)$, the last inequality implies

$$\frac{1}{K} \sum_{k=1}^K \mathbb{E} \|\nabla J_\lambda(\theta_k)\|_2^2 = \mathcal{O}\left(\frac{1}{K} \sum_{k=1}^K \mathbb{E} \|\omega_k - \omega_k^*\|_2^2\right) + \mathcal{O}\left(\frac{K_0^2 \log^2 K}{K^{\frac{3}{5}}}\right) + \mathcal{O}(\epsilon_{\text{app}}). \quad (84)$$

This completes the proof. ■

B.5. Proof of Theorem 5

Proof We define an event E_k as $\|\nabla J_\lambda(\theta_k)\| \leq \frac{\lambda}{2|\mathcal{S}||\mathcal{A}|}$ and its complement E_k^c as $\|\nabla J_\lambda(\theta_k)\| > \frac{\lambda}{2|\mathcal{S}||\mathcal{A}|}$. We use $\mathbf{1}_{E_k}$ to indicate whether the event happens or not, i.e. $\mathbf{1}_{E_k} = 1$ if E_k happens and $\mathbf{1}_{E_k} = 0$ if E_k^c happens. Then we have for the optimality gap:

$$\begin{aligned} \sum_{k=1}^K \mathbb{E}[J^* - J(\theta_k)] &= \sum_{k=1}^K \mathbb{E}[(J^* - J(\theta_k))\mathbf{1}_{E_k}] + \sum_{k=1}^K \mathbb{E}[(J^* - J(\theta_k))\mathbf{1}_{E_k^c}] \\ &\leq \frac{2\lambda}{1-\gamma} \left\| \frac{d_{\pi^*}}{\eta} \right\|_\infty \sum_{k=1}^K \mathbb{E}[\mathbf{1}_{E_k}] + \sum_{k=1}^K \mathbb{E}[(J^* - J(\theta_k))\mathbf{1}_{E_k^c}] \\ &\leq \frac{2\lambda}{1-\gamma} \left\| \frac{d_{\pi^*}}{\eta} \right\|_\infty \sum_{k=1}^K \mathbb{E}[\mathbf{1}_{E_k}] + J^* \sum_{k=1}^K \mathbb{E}[\mathbf{1}_{E_k^c}] \\ &\leq \frac{2\lambda}{1-\gamma} \left\| \frac{d_{\pi^*}}{\eta} \right\|_\infty K + J^* \sum_{k=1}^K \mathbb{E}[\mathbf{1}_{E_k^c}], \end{aligned} \quad (85)$$

where the first inequality follows from Lemma 1.

Now it suffices to bound $\sum_{k=1}^K \mathbb{E}[\mathbf{1}_{E_k^c}]$.

$$\begin{aligned} \sum_{k=1}^K \mathbb{E} \|\nabla J_\lambda(\theta_k)\|^2 &\geq \sum_{k=1}^K \mathbb{E} [\|\nabla J_\lambda(\theta_k)\|^2 \mathbf{1}_{E_k^c}] \\ &\geq \sum_{k=1}^K \frac{\lambda^2}{4|\mathcal{S}|^2|\mathcal{A}|^2} \mathbb{E}[\mathbf{1}_{E_k^c}] \end{aligned} \quad (86)$$

Substituting the above inequality into (85) and dividing both sides by K give

$$\frac{1}{K} \sum_{k=1}^K \mathbb{E}[J^* - J(\theta_k)] \leq \frac{2\lambda}{1-\gamma} \left\| \frac{d_{\pi^*}}{\eta} \right\|_\infty + \frac{4|\mathcal{S}|^2|\mathcal{A}|^2}{\lambda^2} \frac{1}{K} \sum_{k=1}^K \mathbb{E} \|\nabla J_\lambda(\theta_k)\|^2 \quad (87)$$

It is known that the softmax policy satisfies Assumption 3, thus we immediately know that Theorem 2 and Theorem 4 hold. Furthermore, by assumption 5, we have $\phi(s)^\top \omega_\theta^* = V_{\pi_\theta}(s)$, leading to $\epsilon_{\text{app}} = 0$. Applying Theorem 2 and 4 to (87) completes the proof. $\blacksquare$

Appendix C. Supporting Lemmas

C.1. Supporting Lemmas for Theorem 3

Lemma 4 For any $m \geq 1$ and $k \geq (K_0 + 1)m + K_0 + 1$, we have

$$\begin{aligned} \mathbb{E} \langle \omega_k - \omega_{\theta_k}^*, g(x_{(k)}, \omega_k) - \bar{g}(\theta_k, \omega_k) \rangle &\leq C_4 \mathbb{E} \|\theta_k - \theta_{k-d_m}\|_2 + C_5 \sum_{i=\tau_k}^{d_m} \mathbb{E} \|\theta_{k-i} - \theta_{k-d_m}\|_2 \\ &\quad + C_6 \mathbb{E} \|\omega_k - \omega_{k-d_m}\|_2 + C_7 \kappa \rho^{m-1}, \end{aligned}$$

where constant $d_m \leq (K_0 + 1)m + K_0$, and $C_4 := 2C_\delta L_\omega + 4R_\omega C_\delta |\mathcal{A}| L_\pi (1 + \log_\rho \kappa^{-1} + (1 - \rho)^{-1})$, $C_5 := 4R_\omega C_\delta |\mathcal{A}| L_\pi$ and $C_6 := 4(1 + \gamma)R_\omega + 2C_\delta$, $C_7 := 8R_\omega C_\delta$.

Proof Consider the collection of random samples $\{x_{(k-K_0-1)}, x_{(k-K_0)}, \dots, x_{(k)}\}$. Suppose $x_{(k)}$ is sampled by worker n , then due to Assumption 2, $\{x_{(k-K_0-1)}, x_{(k-K_0)}, \dots, x_{(k-1)}\}$ will contain at least another sample drawn by worker n . Therefore, $\{x_{(k-(K_0+1)m)}, x_{(k-(K_0+1)m+1)}, \dots, x_{(k-1)}\}$ will contain at least m samples from worker n .

Consider the Markov chain formed by $m+1$ samples in $\{x_{(k-(K_0+1)m)}, x_{(k-(K_0+1)m+1)}, \dots, x_{(k)}\}$:

$$s_{t-m} \xrightarrow{\theta_{k-d_m}} a_{t-m} \xrightarrow{\mathcal{P}} s_{t-m+1} \xrightarrow{\theta_{k-d_{m-1}}} a_{t-m+1} \cdots s_{t-1} \xrightarrow{\theta_{k-d_1}} a_{t-1} \xrightarrow{\mathcal{P}} s_t \xrightarrow{\theta_{k-d_0}} a_t \xrightarrow{\mathcal{P}} s_{t+1},$$

where $(s_t, a_t, s_{t+1}) = (s_{(k)}, a_{(k)}, s'_{(k)})$, and $\{d_j\}_{j=0}^m$ is some increasing sequence with $d_0 := \tau_k$.

Suppose θ_{k-d_m} was used to generate the k_m th update, then we have $x_{t-m} = x_{(k_m)}$. Following Assumption 2, we have $\tau_{k_m} = k_m - (k - d_m) \leq K_0$. Since $x_{(k_m)}$ is in $\{x_{(k-(K_0+1)m)}, \dots, x_{(k)}\}$, we have $k_m \geq k - (K_0 + 1)m$. Combining these two inequalities, we have

$$d_m \leq (K_0 + 1)m + K_0. \quad (88)$$

Given $(s_{t-m}, a_{t-m}, s_{t-m+1})$ and θ_{k-d_m} , we construct an auxiliary Markov chain as Lemma 3:

$$s_{t-m} \xrightarrow{\theta_{k-d_m}} a_{t-m} \xrightarrow{\mathcal{P}} s_{t-m+1} \xrightarrow{\theta_{k-d_m}} \tilde{a}_{t-m+1} \cdots \tilde{s}_{t-1} \xrightarrow{\theta_{k-d_m}} \tilde{a}_{t-1} \xrightarrow{\mathcal{P}} \tilde{s}_t \xrightarrow{\theta_{k-d_m}} \tilde{a}_t \xrightarrow{\mathcal{P}} \tilde{s}_{t+1}.$$

For brevity, we define

$$\Delta_1(x, \theta, \omega) := \langle \omega - \omega_\theta^*, g(x, \omega) - \bar{g}(\theta, \omega) \rangle.$$

Throughout this proof, we use $\theta, \theta', \omega, \omega', x$ and $\tilde{x}$ as shorthand notations of $\theta_k, \theta_{k-d_m}, \omega_k, \omega_{k-d_m}, x_t$ and $\tilde{x}_t := (\tilde{s}_t, \tilde{a}_t, \tilde{s}_{t+1})$, respectively.

First we decompose $\Delta_1(x, \theta, \omega)$ as

$$\begin{aligned} \Delta_1(x, \theta, \omega) &= \underbrace{\Delta_1(x, \theta, \omega) - \Delta_1(x, \theta', \omega)}_{I_1} + \underbrace{\Delta_1(x, \theta', \omega) - \Delta_1(x, \theta', \omega')}_{I_2} \\ &\quad + \underbrace{\Delta_1(x, \theta', \omega') - \Delta_1(\tilde{x}, \theta', \omega')}_{I_3} + \underbrace{\Delta_1(\tilde{x}, \theta', \omega')}_{I_4}. \end{aligned} \quad (89)$$

We bound I_1 in (89) as

$$\begin{aligned} \Delta_1(x, \theta, \omega) - \Delta_1(x, \theta', \omega) &= \langle \omega - \omega_\theta^*, g(x, \omega) - \bar{g}(\theta, \omega) \rangle - \langle \omega - \omega_{\theta'}^*, g(x, \omega) - \bar{g}(\theta', \omega) \rangle \\ &\leq |\langle \omega - \omega_\theta^*, g(x, \omega) - \bar{g}(\theta, \omega) \rangle - \langle \omega - \omega_{\theta'}^*, g(x, \omega) - \bar{g}(\theta, \omega) \rangle| \\ &\quad + |\langle \omega - \omega_{\theta'}^*, g(x, \omega) - \bar{g}(\theta, \omega) \rangle - \langle \omega - \omega_{\theta'}^*, g(x, \omega) - \bar{g}(\theta', \omega) \rangle|. \end{aligned} \quad (90)$$

For the first term in (90), we have

$$\begin{aligned} |\langle \omega - \omega_\theta^*, g(x, \omega) - \bar{g}(\theta, \omega) \rangle - \langle \omega - \omega_{\theta'}^*, g(x, \omega) - \bar{g}(\theta, \omega) \rangle| &= |\langle \omega_\theta^* - \omega_{\theta'}^*, g(x, \omega) - \bar{g}(\theta, \omega) \rangle| \\ &\leq \|\omega_\theta^* - \omega_{\theta'}^*\|_2 \|g(x, \omega) - \bar{g}(\theta, \omega)\|_2 \\ &\leq 2C_\delta \|\omega_\theta^* - \omega_{\theta'}^*\|_2 \\ &\leq 2C_\delta L_\omega \|\theta - \theta'\|_2, \end{aligned} \quad (91)$$

where the last inequality is due to Proposition 2.

We use $x \sim \theta'$ as shorthand notations to represent that $s \sim \mu_{\theta'}$, $a \sim \pi_{\theta'}$, $s' \sim \mathcal{P}$. For the second term in (90), we have

$$\begin{aligned} &|\langle \omega - \omega_{\theta'}^*, g(x, \omega) - \bar{g}(\theta, \omega) \rangle - \langle \omega - \omega_{\theta'}^*, g(x, \omega) - \bar{g}(\theta', \omega) \rangle| \\ &= |\langle \omega - \omega_{\theta'}^*, \bar{g}(\theta', \omega) - \bar{g}(\theta, \omega) \rangle| \\ &\leq \|\omega - \omega_{\theta'}^*\|_2 \|\bar{g}(\theta', \omega) - \bar{g}(\theta, \omega)\|_2 \\ &\leq 2R_\omega \|\bar{g}(\theta', \omega) - \bar{g}(\theta, \omega)\|_2 \\ &= 2R_\omega \|\mathbb{E}_{x \sim \theta'} [g(x, \omega)] - \mathbb{E}_{x \sim \theta} [g(x, \omega)]\|_2 \\ &\leq 2R_\omega \sup_x \|g(x, \omega)\|_2 \|\mu_{\theta'} \otimes \pi_{\theta'} \otimes \mathcal{P} - \mu_\theta \otimes \pi_\theta \otimes \mathcal{P}\|_{TV} \\ &\leq 2R_\omega C_\delta \|\mu_{\theta'} \otimes \pi_{\theta'} \otimes \mathcal{P} - \mu_\theta \otimes \pi_\theta \otimes \mathcal{P}\|_{TV} \\ &= 4R_\omega C_\delta d_{TV}(\mu_{\theta'} \otimes \pi_{\theta'} \otimes \mathcal{P}, \mu_\theta \otimes \pi_\theta \otimes \mathcal{P}) \\ &\leq 4R_\omega C_\delta |\mathcal{A}| L_\pi (1 + \log_\rho \kappa^{-1} + (1 - \rho)^{-1}) \|\theta - \theta'\|_2, \end{aligned} \quad (92)$$

where the third inequality follows the definition of TV norm, the second last inequality follows (40), and the last inequality follows (Wu et al., 2020, Lemma A.1).

Collecting the upper bounds of the two terms in (90) yields

$$I_1 \leq [2C_\delta L_\omega + 4R_\omega C_\delta |\mathcal{A}| L_\pi (1 + \log_\rho \kappa^{-1} + (1 - \rho)^{-1})] \|\theta - \theta'\|_2. \quad (93)$$

Next we bound $\mathbb{E}[I_2]$ in (89) as

$$\begin{aligned} \mathbb{E}[I_2] &= \mathbb{E}[\Delta_1(x, \theta', \omega) - \Delta_1(x, \theta', \omega')] \\ &= \mathbb{E} \langle \omega - \omega_{\theta'}^*, g(x, \omega) - \bar{g}(\theta', \omega) \rangle - \langle \omega' - \omega_{\theta'}^*, g(x, \omega') - \bar{g}(\theta', \omega') \rangle \\ &\leq \mathbb{E} |\langle \omega - \omega_{\theta'}^*, g(x, \omega) - \bar{g}(\theta', \omega) \rangle - \langle \omega - \omega_{\theta'}^*, g(x, \omega') - \bar{g}(\theta', \omega') \rangle| \\ &\quad + \mathbb{E} |\langle \omega - \omega_{\theta'}^*, g(x, \omega') - \bar{g}(\theta', \omega') \rangle - \langle \omega' - \omega_{\theta'}^*, g(x, \omega') - \bar{g}(\theta', \omega') \rangle|. \end{aligned} \quad (94)$$

We bound the first term in (94) as

$$\begin{aligned} &\mathbb{E} |\langle \omega - \omega_{\theta'}^*, g(x, \omega) - \bar{g}(\theta', \omega) \rangle - \langle \omega - \omega_{\theta'}^*, g(x, \omega') - \bar{g}(\theta', \omega') \rangle| \\ &= \mathbb{E} |\langle \omega - \omega_{\theta'}^*, g(x, \omega) - g(x, \omega') + \bar{g}(\theta', \omega') - \bar{g}(\theta', \omega) \rangle| \\ &\leq 2R_\omega (\mathbb{E} \|g(x, \omega) - g(x, \omega')\|_2 + \mathbb{E} \|\bar{g}(\theta', \omega') - \bar{g}(\theta', \omega)\|_2) \\ &\leq 2R_\omega (\mathbb{E} \|g(x, \omega) - g(x, \omega')\|_2 + \mathbb{E} \|\mathbb{E}_{x \sim \theta'}[g(x, \omega')] - \mathbb{E}_{x \sim \theta'}[g(x, \omega)]\|_2) \\ &= 2R_\omega (\mathbb{E} \|(\gamma\phi(s') - \phi(s))^\top (\omega - \omega')\|_2 + \mathbb{E} \|\mathbb{E}_{x \sim \theta'}[(\gamma\phi(s') - \phi(s))^\top] (\omega' - \omega)\|_2) \\ &\leq 2R_\omega ((1 + \gamma)\mathbb{E} \|\omega - \omega'\|_2 + (1 + \gamma)\mathbb{E} \|\omega - \omega'\|_2) \\ &= 4R_\omega (1 + \gamma)\mathbb{E} \|\omega - \omega'\|_2. \end{aligned} \quad (95)$$

We bound the second term in (94) as

$$\begin{aligned} &\mathbb{E} |\langle \omega - \omega_{\theta'}^*, g(x, \omega') - \bar{g}(\theta', \omega') \rangle - \langle \omega' - \omega_{\theta'}^*, g(x, \omega') - \bar{g}(\theta', \omega') \rangle| \\ &= \mathbb{E} |\langle \omega - \omega', g(x, \omega') - \bar{g}(\theta', \omega') \rangle| \\ &\leq 2C_\delta \mathbb{E} \|\omega - \omega'\|_2. \end{aligned} \quad (96)$$

Collecting the upper bounds of the two terms in (94) yields

$$\mathbb{E}[I_2] \leq (4(1 + \gamma)R_\omega + 2C_\delta) \mathbb{E} \|\omega_k - \omega_{k-d_m}\|_2. \quad (97)$$

We first bound I_3 as

$$\begin{aligned} \mathbb{E}[I_3 | \theta', \omega', s_{t-m+1}] &= \mathbb{E} [\Delta_1(x, \theta', \omega') - \Delta_1(\tilde{x}, \theta', \omega') | \theta', \omega', s_{t-m+1}] \\ &\leq |\mathbb{E} [\Delta_1(x, \theta', \omega') | \theta', \omega', s_{t-m+1}] - \mathbb{E} [\Delta_1(\tilde{x}, \theta', \omega') | \theta', \omega', s_{t-m+1}]| \\ &\leq \sup_x |\Delta_1(x, \theta', \omega')| \|\mathbb{P}(x \in \cdot | \theta', \omega', s_{t-m+1}) - \mathbb{P}(\tilde{x} \in \cdot | \theta', \omega', s_{t-m+1})\|_{TV} \\ &\leq 8R_\omega C_\delta d_{TV} (\mathbb{P}(x \in \cdot | \theta', s_{t-m+1}), \mathbb{P}(\tilde{x} \in \cdot | \theta', s_{t-m+1})), \end{aligned} \quad (98)$$

where the second last inequality follows the definition of TV norm, and the last inequality follows

$$|\Delta_1(x, \theta', \omega')| \leq \|\omega' - \omega_{\theta'}^*\|_2 \|g(x, \omega') - \bar{g}(\theta', \omega')\|_2 \leq 4R_\omega C_\delta. \quad (99)$$

By following (28) in Lemma 3, we have

$$d_{TV}(\mathbb{P}(x \in \cdot | \theta', s_{t-m+1}), \mathbb{P}(\tilde{x} \in \cdot | \theta', s_{t-m+1})) \leq \frac{1}{2} |\mathcal{A}| L_\pi \sum_{i=\tau_k}^{d_m} \mathbb{E} [\|\theta_{k-i} - \theta_{k-d_m}\|_2 | \theta', s_{t-m+1}].$$

Substituting the last inequality into (98), then taking total expectation on both sides yield

$$\mathbb{E}[I_3] \leq 4R_\omega C_\delta |\mathcal{A}| L_\pi \sum_{i=\tau_k}^{d_m} \mathbb{E} \|\theta_{k-i} - \theta_{k-d_m}\|_2. \quad (100)$$

Next we bound I_4 . Define $\bar{x} := (\bar{s}, \bar{a}, \bar{s}')$ where $\bar{s} \sim \mu_{\theta'}$, $\bar{a} \sim \pi_{\theta'}$ and $\bar{s}' \sim \mathcal{P}$. It is immediate that

$$\begin{aligned} \mathbb{E}[\Delta_1(\bar{x}, \theta', \omega') | \theta', \omega', s_{t-m+1}] &= \langle \omega' - \omega_{\theta'}^*, \mathbb{E}[g(\bar{x}, \omega') | \theta', \omega', s_{t-m+1}] - \bar{g}(\theta', \omega') \rangle \\ &= \langle \omega' - \omega_{\theta'}^*, \bar{g}(\theta', \omega') - \bar{g}(\theta', \omega') \rangle = 0. \end{aligned} \quad (101)$$

Then we have

$$\begin{aligned} \mathbb{E}[I_4 | \theta', \omega', s_{t-m+1}] &= \mathbb{E} [\Delta_1(\tilde{x}, \theta', \omega') - \Delta_1(\bar{x}, \theta', \omega') | \theta', \omega', s_{t-m+1}] \\ &\leq |\mathbb{E} [\Delta_1(\tilde{x}, \theta', \omega') | \theta', \omega', s_{t-m+1}] - \mathbb{E} [\Delta_1(\bar{x}, \theta', \omega') | \theta', \omega', s_{t-m+1}]| \\ &\leq \sup_x |\Delta_1(x, \theta', \omega')| \|\mathbb{P}(\tilde{x} \in \cdot | \theta', s_{t-m+1}) - \mathbb{P}(\bar{x} \in \cdot | \theta', s_{t-m+1})\|_{TV} \\ &\leq 8R_\omega C_\delta d_{TV}(\mathbb{P}(\tilde{x} \in \cdot | \theta', s_{t-m+1}), \mathbb{P}(\bar{x} \in \cdot | \theta', s_{t-m+1})) \\ &= 8R_\omega C_\delta d_{TV}(\mathbb{P}(\tilde{x} \in \cdot | \theta', s_{t-m+1}), \mu_{\theta'} \otimes \pi_{\theta'} \otimes \mathcal{P}), \end{aligned} \quad (102)$$

where the second inequality follows the definition of TV norm, and the third inequality follows (101).

The auxiliary Markov chain with policy $\pi_{\theta'}$ starts from initial state s_{t-m+1} , and $\tilde{s}_t$ is the $(m-1)$ th state on the chain. Following Lemma 2, we have:

$$\begin{aligned} d_{TV}(\mathbb{P}(\tilde{x} \in \cdot | \theta', s_{t-m+1}), \mu_{\theta'} \otimes \pi_{\theta'} \otimes \mathcal{P}) \\ = d_{TV}(\mathbb{P}((\tilde{s}_t, \tilde{a}_t, \tilde{s}_{t+1}) \in \cdot | \theta', s_{t-m+1}), \mu_{\theta'} \otimes \pi_{\theta'} \otimes \mathcal{P}) \leq \kappa \rho^{m-1}. \end{aligned} \quad (103)$$

Substituting the last inequality into (102) and taking total expectation on both sides yield

$$\mathbb{E}[I_4] \leq 8R_\omega C_\delta \kappa \rho^{m-1}. \quad (104)$$

Taking total expectation on (89) and collecting bounds of I_1, I_2, I_3, I_4 yield

$$\begin{aligned} \mathbb{E}[\Delta_1(x, \theta, \omega)] &\leq C_4 \mathbb{E} \|\theta_k - \theta_{k-d_m}\|_2 + C_5 \sum_{i=\tau_k}^{d_m} \mathbb{E} \|\theta_{k-i} - \theta_{k-d_m}\|_2 \\ &\quad + C_6 \mathbb{E} \|\omega_k - \omega_{k-d_m}\|_2 + C_7 \kappa \rho^{m-1}, \end{aligned} \quad (105)$$

where $C_4 := 2C_\delta L_\omega + 4R_\omega C_\delta |\mathcal{A}| L_\pi (1 + \log_\rho \kappa^{-1} + (1 - \rho)^{-1})$, $C_5 := 4R_\omega C_\delta |\mathcal{A}| L_\pi$, $C_6 := 4(1 + \gamma)R_\omega + 2C_\delta$ and $C_7 := 8R_\omega C_\delta$. $\blacksquare$

C.2. Supporting Lemmas for Theorem 4

Lemma 5 For any $m \geq 1$ and $k \geq (K_0 + 1)m + K_0 + 1$, we have

$$\begin{aligned} \mathbb{E} \left\langle \nabla J_\lambda(\theta_k), \left(\hat{\delta}(\hat{x}_{(k)}, \omega_k^*) - \delta(\hat{x}_{(k)}, \theta_k) \right) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) \right\rangle &\geq -D_2 \mathbb{E} \|\theta_{k-\tau_k} - \theta_{k-d_m}\|_2 \\ &- D_3 \mathbb{E} \|\theta_k - \theta_{k-d_m}\|_2 - D_4 \sum_{i=\tau_k}^{d_m} \mathbb{E} \|\theta_{k-i} - \theta_{k-d_m}\|_2 - D_5 \kappa \rho^{m-1} - L_V C_\psi (1 + \gamma) \epsilon_{\text{app}}, \end{aligned}$$

where $D_2 := 2L_V L_\psi C_\delta$, $D_3 := (2C_\delta C_\psi L_\lambda + L_V C_\psi (L_\omega + L_V)(1 + \gamma))$, $D_4 := 2L_V C_\psi C_\delta |\mathcal{A}| L_\pi$ and $D_5 := 4L_V C_\psi C_\delta$.

Proof For the worker that contributes to the k th update, we construct its Markov chain:

$$\hat{s}_{t-m} \xrightarrow{\theta_{k-d_m}} \hat{a}_{t-m} \xrightarrow{\hat{P}} \hat{s}_{t-m+1} \xrightarrow{\theta_{k-d_{m-1}}} \hat{a}_{t-m+1} \cdots \hat{s}_{t-1} \xrightarrow{\theta_{k-d_1}} \hat{a}_{t-1} \xrightarrow{\hat{P}} \hat{s}_t \xrightarrow{\theta_{k-d_0}} a_t \xrightarrow{\hat{P}} \hat{s}_{t+1},$$

where $(s_t, a_t, \hat{s}_{t+1}) = (\hat{s}_{(k)}, \hat{a}_{(k)}, s'_{(k)})$, and $\{d_j\}_{j=0}^m$ is some increasing sequence with $d_0 := \tau_k$. By (88) in Lemma 4, we have $d_m \leq (K_0 + 1)m + K_0$.

Given $(\hat{s}_{t-m}, \hat{a}_{t-m}, \hat{s}_{t-m+1})$ and θ_{k-d_m} , we construct an auxiliary Markov chain:

$$\hat{s}_{t-m} \xrightarrow{\theta_{k-d_m}} \hat{a}_{t-m} \xrightarrow{\hat{P}} \hat{s}_{t-m+1} \xrightarrow{\theta_{k-d_m}} \tilde{a}_{t-m+1} \cdots \tilde{s}_{t-1} \xrightarrow{\theta_{k-d_m}} \tilde{a}_{t-1} \xrightarrow{\hat{P}} \tilde{s}_t \xrightarrow{\theta_{k-d_m}} \tilde{a}_t \xrightarrow{\hat{P}} \tilde{s}_{t+1}.$$

First we have

$$\begin{aligned} &\left\langle \nabla J_\lambda(\theta_k), \left(\hat{\delta}(\hat{x}_{(k)}, \omega_k^*) - \delta(\hat{x}_{(k)}, \theta_k) \right) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) \right\rangle \\ &= \left\langle \nabla J_\lambda(\theta_k), \left(\hat{\delta}(\hat{x}_{(k)}, \omega_k^*) - \delta(\hat{x}_{(k)}, \theta_k) \right) \left(\psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) - \psi_{\theta_{k-d_m}}(\hat{s}_{(k)}, \hat{a}_{(k)}) \right) \right\rangle \\ &\quad + \left\langle \nabla J_\lambda(\theta_k), \left(\hat{\delta}(\hat{x}_{(k)}, \omega_k^*) - \delta(\hat{x}_{(k)}, \theta_k) \right) \psi_{\theta_{k-d_m}}(\hat{s}_{(k)}, \hat{a}_{(k)}) \right\rangle. \end{aligned} \quad (106)$$

We first bound the first term in (106) as

$$\begin{aligned} &\left\langle \nabla J_\lambda(\theta_k), \left(\hat{\delta}(\hat{x}_{(k)}, \omega_k^*) - \delta(\hat{x}_{(k)}, \theta_k) \right) \left(\psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) - \psi_{\theta_{k-d_m}}(\hat{s}_{(k)}, \hat{a}_{(k)}) \right) \right\rangle \\ &\geq -\|\nabla J_\lambda(\theta_k)\|_2 \|\hat{\delta}(\hat{x}_{(k)}, \omega_k^*) - \delta(\hat{x}_{(k)}, \theta_k)\| \|\psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) - \psi_{\theta_{k-d_m}}(\hat{s}_{(k)}, \hat{a}_{(k)})\|_2 \\ &\geq -\|\nabla J_\lambda(\theta_k)\|_2 \left(|\hat{\delta}(\hat{x}_{(k)}, \omega_k^*)| + |\delta(\hat{x}_{(k)}, \theta_k)| \right) \|\psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) - \psi_{\theta_{k-d_m}}(\hat{s}_{(k)}, \hat{a}_{(k)})\|_2 \\ &\geq -L_V \left(|\hat{\delta}(\hat{x}_{(k)}, \omega_k^*)| + |\delta(\hat{x}_{(k)}, \theta_k)| \right) \|\psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) - \psi_{\theta_{k-d_m}}(\hat{s}_{(k)}, \hat{a}_{(k)})\|_2 \\ &\geq -2L_V C_\delta \|\psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) - \psi_{\theta_{k-d_m}}(\hat{s}_{(k)}, \hat{a}_{(k)})\|_2 \\ &\geq -2L_V L_\psi C_\delta \|\theta_{k-\tau_k} - \theta_{k-d_m}\|_2, \end{aligned} \quad (107)$$

where the last inequality follows Assumption 3 and second last inequality follows

$$\begin{aligned} |\hat{\delta}(x, \omega_\theta^*)| &\leq |r(x)| + \gamma \|\phi(s')\|_2 \|\omega_\theta^*\|_2 + \|\phi(s)\|_2 \|\omega_\theta^*\|_2 \leq r_{\max} + (1 + \gamma) R_\omega \leq C_\delta, \\ |\delta(x, \theta)| &\leq |r(x)| + \gamma |V_{\pi_\theta}(s')| + |V_{\pi_\theta}(s)| \leq r_{\max} + (1 + \gamma) \frac{r_{\max}}{1 - \gamma} \leq C_\delta. \end{aligned}$$

Substituting (107) into (106) gives

$$\begin{aligned} & \left\langle \nabla J_\lambda(\theta_k), \left(\hat{\delta}(\hat{x}_{(k)}, \omega_k^*) - \delta(\hat{x}_{(k)}, \theta_k) \right) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) \right\rangle \\ & \geq -2L_V L_\psi C_\delta \|\theta_{k-\tau_k} - \theta_{k-d_m}\|_2 + \left\langle \nabla J_\lambda(\theta_k), \left(\hat{\delta}(\hat{x}_{(k)}, \omega_k^*) - \delta(\hat{x}_{(k)}, \theta_k) \right) \psi_{\theta_{k-d_m}}(\hat{s}_{(k)}, \hat{a}_{(k)}) \right\rangle. \end{aligned} \quad (108)$$

Then we start to bound the second term in (108). For brevity, we define

$$\Delta_2(x, \theta) := \left\langle \nabla J_\lambda(\theta), \left(\hat{\delta}(x, \omega_\theta^*) - \delta(x, \theta) \right) \psi_{\theta_{k-d_m}}(s, a) \right\rangle.$$

In the following proof, we use $\theta, \theta', \omega_\theta^*, \omega_{\theta'}^*, x$ as shorthand notations for $\theta_k, \theta_{k-d_m}, \omega_k^*, \omega_{k-d_m}^*, x_t$ respectively. We also define $\bar{x} := (\bar{s}, \bar{a}, \bar{s}')$, where $\bar{s} \sim d_{\theta'}$, $\bar{a} \sim \pi_{\theta'}$ and $\bar{s}' \sim \mathcal{P}$. Define $\tilde{x} := (\tilde{s}_{t-d_m}, \tilde{a}_{t-d_m}, \tilde{s}'_{t-d_m})$ where $\tilde{s}_{t-d_m}, \tilde{a}_{t-d_m}$ are state and action on the auxiliary Markov chain, and $\tilde{s}'_{t-d_m} \sim \mathcal{P}(\cdot | \tilde{s}_{t-d_m}, \tilde{a}_{t-d_m})$ is a virtual sample. We decompose the second term in (108) as

$$\Delta_2(x, \theta) = \underbrace{\Delta_2(x, \theta) - \Delta_2(x, \theta')}_{I_1} + \underbrace{\Delta_2(x, \theta') - \Delta_2(\tilde{x}, \theta')}_{I_2} + \underbrace{\Delta_2(\tilde{x}, \theta') - \Delta_2(\bar{x}, \theta')}_{I_3} + \underbrace{\Delta_2(\bar{x}, \theta')}_{I_4}.$$

We bound the term I_1 as

$$\begin{aligned} I_1 &= \left\langle \nabla J_\lambda(\theta), \left(\hat{\delta}(x, \omega_\theta^*) - \delta(x, \theta) \right) \psi_{\theta'}(s, a) \right\rangle - \left\langle \nabla J_\lambda(\theta'), \left(\hat{\delta}(x, \omega_{\theta'}^*) - \delta(x, \theta') \right) \psi_{\theta'}(s, a) \right\rangle \\ &= \left\langle \nabla J_\lambda(\theta), \left(\hat{\delta}(x, \omega_\theta^*) - \delta(x, \theta) \right) \psi_{\theta'}(s, a) \right\rangle - \left\langle \nabla J_\lambda(\theta'), \left(\hat{\delta}(x, \omega_\theta^*) - \delta(x, \theta) \right) \psi_{\theta'}(s, a) \right\rangle \\ &\quad + \left\langle \nabla J_\lambda(\theta'), \left(\hat{\delta}(x, \omega_\theta^*) - \delta(x, \theta) \right) \psi_{\theta'}(s, a) \right\rangle - \left\langle \nabla J_\lambda(\theta'), \left(\hat{\delta}(x, \omega_{\theta'}^*) - \delta(x, \theta') \right) \psi_{\theta'}(s, a) \right\rangle. \end{aligned} \quad (109)$$

For the first term in I_1 , we have

$$\begin{aligned} & \left\langle \nabla J_\lambda(\theta), \left(\hat{\delta}(x, \omega_\theta^*) - \delta(x, \theta) \right) \psi_{\theta'}(s, a) \right\rangle - \left\langle \nabla J_\lambda(\theta'), \left(\hat{\delta}(x, \omega_\theta^*) - \delta(x, \theta) \right) \psi_{\theta'}(s, a) \right\rangle \\ &= \left\langle \nabla J_\lambda(\theta) - \nabla J_\lambda(\theta'), \left(\hat{\delta}(x, \omega_\theta^*) - \delta(x, \theta) \right) \psi_{\theta'}(s, a) \right\rangle \\ &\geq -\|\nabla J_\lambda(\theta) - \nabla J_\lambda(\theta')\|_2 \|\hat{\delta}(x, \omega_\theta^*) - \delta(x, \theta)\|_2 \|\psi_{\theta'}(s, a)\|_2 \\ &\geq -2C_\delta C_\psi \|\nabla J_\lambda(\theta) - \nabla J_\lambda(\theta')\|_2 \\ &\geq -2C_\delta C_\psi L_\lambda \|\theta - \theta'\|_2, \end{aligned} \quad (110)$$

where the last inequality is due to the L_λ -Lipschitz continuity of policy gradient shown in Proposition 1.

For the second term in I_1 , we have

$$\begin{aligned}
& \left\langle \nabla J_\lambda(\theta'), \left(\hat{\delta}(x, \omega_\theta^*) - \delta(x, \theta) \right) \psi_{\theta'}(s, a) \right\rangle - \left\langle \nabla J_\lambda(\theta'), \left(\hat{\delta}(x, \omega_{\theta'}^*) - \delta(x, \theta') \right) \psi_{\theta'}(s, a) \right\rangle \\
&= \left\langle \nabla J_\lambda(\theta'), \left(\hat{\delta}(x, \omega_\theta^*) - \hat{\delta}(x, \omega_{\theta'}^*) + \delta(x, \theta') - \delta(x, \theta) \right) \psi_{\theta'}(s, a) \right\rangle \\
&\geq -L_V C_\psi \left| \hat{\delta}(x, \omega_\theta^*) - \hat{\delta}(x, \omega_{\theta'}^*) + \delta(x, \theta') - \delta(x, \theta) \right| \\
&\geq -L_V C_\psi \left| \gamma \phi(s')^\top (\omega_\theta^* - \omega_{\theta'}^*) + \phi(s)^\top (\omega_{\theta'}^* - \omega_\theta^*) + \gamma V_{\pi_{\theta'}}(s') - \gamma V_{\pi_\theta}(s') + V_{\pi_\theta}(s) - V_{\pi_{\theta'}}(s) \right| \\
&\geq -L_V C_\psi (\gamma \|\omega_\theta^* - \omega_{\theta'}^*\|_2 + \|\omega_{\theta'}^* - \omega_\theta^*\|_2 + \gamma |V_{\pi_{\theta'}}(s') - V_{\pi_\theta}(s')| + |V_{\pi_\theta}(s) - V_{\pi_{\theta'}}(s)|) \\
&\geq -L_V C_\psi (\gamma L_\omega \|\theta - \theta'\|_2 + L_\omega \|\theta - \theta'\|_2 + \gamma L_V \|\theta - \theta'\|_2 + L_V \|\theta - \theta'\|_2) \\
&= -L_V C_\psi (L_\omega + L_V)(1 + \gamma) \|\theta - \theta'\|_2, \tag{111}
\end{aligned}$$

where the last inequality is due to the L_ω -Lipschitz continuity of ω_θ^* shown in Proposition 2 and L_V -Lipschitz continuity of $V_{\pi_\theta}(s)$. Collecting the upper bounds of I_1 yields

$$I_1 \geq -(2C_\delta C_\psi L_\lambda + L_V C_\psi (L_\omega + L_V)(1 + \gamma)) \|\theta - \theta'\|_2. \tag{112}$$

First we bound I_2 as

$$\begin{aligned}
\mathbb{E}[I_2 | \theta', \hat{s}_{t-m+1}] &= \mathbb{E} [\Delta_2(x, \theta') - \Delta_2(\tilde{x}, \theta') | \theta', \hat{s}_{t-m+1}] \\
&\geq -\left| \mathbb{E} [\Delta_2(x, \theta') | \theta', \hat{s}_{t-m+1}] - \mathbb{E} [\Delta_2(\tilde{x}, \theta') | \theta', \hat{s}_{t-m+1}] \right| \\
&\geq -\sup_x |\Delta_2(x, \theta')| \left\| \mathbb{P}(x \in \cdot | \theta', \hat{s}_{t-m+1}) - \mathbb{P}(\tilde{x} \in \cdot | \theta', \hat{s}_{t-m+1}) \right\|_{TV} \\
&\geq -4L_V C_\psi C_\delta d_{TV} (\mathbb{P}(x \in \cdot | \theta', \hat{s}_{t-m+1}), \mathbb{P}(\tilde{x} \in \cdot | \theta', \hat{s}_{t-m+1})) \\
&= -4L_V C_\psi C_\delta d_{TV} (\mathbb{P}((\hat{s}_t, \hat{a}_t) \in \cdot | \theta', \hat{s}_{t-m+1}), \mathbb{P}((\tilde{s}_t, \tilde{a}_t) \in \cdot | \theta', \hat{s}_{t-m+1})) \\
&\geq -2L_V C_\psi C_\delta |\mathcal{A}| L_\pi \sum_{i=\tau_k}^{d_m} \mathbb{E} [\|\theta_{k-i} - \theta_{k-d_m}\|_2 | \theta', \hat{s}_{t-m+1}], \tag{113}
\end{aligned}$$

where the second inequality is due to the definition of TV norm, the last inequality follows (28) in Lemma 3, and the second last inequality follows the fact that

$$|\Delta_2(x, \theta')| \leq \|\nabla J_\lambda(\theta')\|_2 |\hat{\delta}(x, \omega_{\theta'}^*) - \delta(x, \theta')| \|\psi_{\theta'}(s, a)\|_2 \leq 2L_V C_\delta C_\psi. \tag{114}$$

Taking total expectation on both sides of (113) yields

$$\mathbb{E}[I_2] \geq -2L_V C_\psi C_\delta |\mathcal{A}| L_\pi \sum_{i=\tau_k}^{d_m} \mathbb{E} \|\theta_{k-i} - \theta_{k-d_m}\|_2. \tag{115}$$

Next we bound I_3 as

$$\begin{aligned}
\mathbb{E}[I_3 | \theta', \hat{s}_{t-m+1}] &= \mathbb{E} [\Delta_2(\tilde{x}, \theta') - \Delta_2(\bar{x}, \theta') | \theta', \hat{s}_{t-m+1}] \\
&\geq -\left| \mathbb{E} [\Delta_2(\tilde{x}, \theta') | \theta', \hat{s}_{t-m+1}] - \mathbb{E} [\Delta_2(\bar{x}, \theta') | \theta', \hat{s}_{t-m+1}] \right| \\
&\geq -\sup_x |\Delta_2(x, \theta')| \left\| \mathbb{P}(\tilde{x} \in \cdot | \theta', \hat{s}_{t-m+1}) - \mathbb{P}(\bar{x} \in \cdot | \theta', \hat{s}_{t-m+1}) \right\|_{TV} \\
&\geq -4L_V C_\psi C_\delta d_{TV} (\mathbb{P}(\tilde{x} \in \cdot | \theta', \hat{s}_{t-m+1}), d_{\theta'} \otimes \pi_{\theta'} \otimes \mathcal{P}), \tag{116}
\end{aligned}$$

where the second inequality is due to the definition of TV norm, and the last inequality follows (114).

The auxiliary Markov chain with policy $\pi_{\theta'}$ starts from initial state $\hat{s}_{t-m+1}$, and $\tilde{s}_t$ is the $(m-1)$ th state on the chain. Following Lemma 2, we have:

$$d_{TV} \left(\mathbb{P}(\tilde{x} \in \cdot | \theta', \hat{s}_{t-m+1}), d_{\theta'} \otimes \pi_{\theta'} \otimes \hat{\mathcal{P}} \right) = d_{TV} \left(\mathbb{P}((\tilde{s}_t, \tilde{a}_t, \tilde{s}_{t+1}) \in \cdot | \theta', \hat{s}_{t-m+1}), d_{\theta'} \otimes \pi_{\theta'} \otimes \mathcal{P} \right) \leq \kappa \rho^{m-1}. \quad (117)$$

Substituting the last inequality into (116) and taking total expectation on both sides yield

$$\mathbb{E}[I_3] \geq -4L_V C_\psi C_\delta \kappa \rho^{m-1}. \quad (118)$$

We bound I_4 as

$$\begin{aligned} \mathbb{E}[I_4 | \theta'] &= \mathbb{E} \left[\left\langle \nabla J_\lambda(\theta'), \left(\hat{\delta}(\bar{x}, \omega_{\theta'}^*) - \delta(\bar{x}, \theta') \right) \psi_{\theta'}(s, a) \right\rangle \middle| \theta' \right] \\ &\geq -L_V C_\psi \mathbb{E} \left[\left\| \hat{\delta}(\bar{x}, \omega_{\theta'}^*) - \delta(\bar{x}, \theta') \right\| \middle| \theta' \right] \\ &= -L_V C_\psi \mathbb{E} \left[\left\| \gamma \left(\phi(\bar{s}')^\top \omega_{\theta'}^* - V_{\pi_{\theta'}}(\bar{s}') \right) + V_{\pi_{\theta'}}(\bar{s}) - \phi(\bar{s})^\top \omega_{\theta'}^* \right\| \middle| \theta' \right] \\ &\geq -L_V C_\psi \left(\gamma \mathbb{E} \left[\left\| \phi(\bar{s}')^\top \omega_{\theta'}^* - V_{\pi_{\theta'}}(\bar{s}') \right\| \middle| \theta' \right] + \mathbb{E} \left[\left\| V_{\pi_{\theta'}}(\bar{s}) - \phi(\bar{s})^\top \omega_{\theta'}^* \right\| \middle| \theta' \right] \right) \\ &\geq -L_V C_\psi \left(\gamma \sqrt{\mathbb{E} \left[\left\| \phi(\bar{s}')^\top \omega_{\theta'}^* - V_{\pi_{\theta'}}(\bar{s}') \right\|^2 \middle| \theta' \right]} + \sqrt{\mathbb{E} \left[\left\| V_{\pi_{\theta'}}(\bar{s}) - \phi(\bar{s})^\top \omega_{\theta'}^* \right\|^2 \middle| \theta' \right]} \right) \\ &= -L_V C_\psi \left(\gamma \sqrt{\mathbb{E}_{\bar{s}' \sim \mu_{\theta'}} \left[\left\| \phi(\bar{s}')^\top \omega_{\theta'}^* - V_{\pi_{\theta'}}(\bar{s}') \right\|^2 \right]} + \sqrt{\mathbb{E}_{\bar{s} \sim \mu_{\theta'}} \left[\left\| V_{\pi_{\theta'}}(\bar{s}) - \phi(\bar{s})^\top \omega_{\theta'}^* \right\|^2 \right]} \right) \\ &\geq -L_V C_\psi (1 + \gamma) \epsilon_{\text{app}}, \end{aligned} \quad (119)$$

where the second last inequality follows Jensen's inequality.

Taking total expectation on both sides of (108), and collecting lower bounds of I_1 , I_2 , I_3 and I_4 yield

$$\begin{aligned} &\mathbb{E} \left\langle \nabla J_\lambda(\theta_k), \left(\hat{\delta}(\hat{x}_{(k)}, \omega_k^*) - \delta(\hat{x}_{(k)}, \theta_k) \right) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) \right\rangle \\ &\geq -D_2 \mathbb{E} \|\theta_{k-\tau_k} - \theta_{k-d_m}\|_2 - D_3 \mathbb{E} \|\theta_k - \theta_{k-d_m}\|_2 - D_4 \sum_{i=\tau_k}^{d_m} \mathbb{E} \|\theta_{k-i} - \theta_{k-d_m}\|_2 \\ &\quad - D_5 \kappa \rho^{m-1} - L_V C_\psi (1 + \gamma) \epsilon_{\text{app}}, \end{aligned} \quad (120)$$

where $D_2 := 2L_V L_\psi C_\delta$, $D_3 := (2C_\delta C_\psi L_\lambda + L_V C_\psi (L_\omega + L_V)(1 + \gamma))$, $D_4 := 2L_V C_\psi C_\delta |\mathcal{A}| L_\pi$ and $D_5 := 4L_V C_\psi C_\delta$. $\blacksquare$

Lemma 6 For any $m \geq 1$ and $k \geq (K_0 + 1)m + K_0 + 1$, we have

$$\begin{aligned} &\mathbb{E} \left\langle \nabla J_\lambda(\theta_k), \delta(\hat{x}_{(k)}, \theta_k) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) - \nabla J(\theta_k) \right\rangle \\ &\geq -D_6 \mathbb{E} \|\theta_{k-\tau_k} - \theta_{k-d_m}\|_2 - D_7 \mathbb{E} \|\theta_k - \theta_{k-d_m}\|_2 - D_8 \sum_{i=\tau_k}^{d_m} \mathbb{E} \|\theta_{k-i} - \theta_{k-d_m}\|_2 - D_9 \kappa \rho^{m-1}, \end{aligned}$$

where $d_m \leq (K_0 + 1)m + K_0$, $D_6 := L_V C_\delta L_\psi$, $D_7 := C_p L_\lambda + (1 + \gamma) L_V^2 C_\psi + 2L_V L_\lambda$, $D_8 := L_V C_p |\mathcal{A}| L_\pi$ and $D_9 := 2L_V C_p$.

Proof For the worker that contributes to the k th update, we construct its Markov chain:

$$\hat{s}_{t-m} \xrightarrow{\theta_{k-d_m}} \hat{a}_{t-m} \xrightarrow{\hat{P}} \hat{s}_{t-m+1} \xrightarrow{\theta_{k-d_{m-1}}} \hat{a}_{t-m+1} \cdots \hat{s}_{t-1} \xrightarrow{\theta_{k-d_1}} \hat{a}_{t-1} \xrightarrow{\hat{P}} s_t \xrightarrow{\theta_{k-d_0}} a_t \xrightarrow{\hat{P}} \hat{s}_{t+1},$$

where $(s_t, a_t, \hat{s}_{t+1}) = (\hat{s}_{(k)}, \hat{a}_{(k)}, s'_{(k)})$, and $\{d_j\}_{j=0}^m$ is some increasing sequence with $d_0 := \tau_k$. By (88) in Lemma 4, we have $d_m \leq (K_0 + 1)m + K_0$.

Given $(\hat{s}_{t-m}, \hat{a}_{t-m}, \hat{s}_{t-m+1})$ and θ_{k-d_m} , we construct an auxiliary Markov chain:

$$\hat{s}_{t-m} \xrightarrow{\theta_{k-d_m}} \hat{a}_{t-m} \xrightarrow{\hat{P}} \hat{s}_{t-m+1} \xrightarrow{\theta_{k-d_m}} \tilde{a}_{t-m+1} \cdots \tilde{s}_{t-1} \xrightarrow{\theta_{k-d_m}} \tilde{a}_{t-1} \xrightarrow{\hat{P}} \tilde{s}_t \xrightarrow{\theta_{k-d_m}} \tilde{a}_t \xrightarrow{\hat{P}} \tilde{s}_{t+1}.$$

First we have

$$\begin{aligned} & \left\langle \nabla J_\lambda(\theta_k), \delta(\hat{x}_{(k)}, \theta_k) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) - \nabla J(\theta_k) \right\rangle \\ &= \left\langle \nabla J_\lambda(\theta_k), \delta(\hat{x}_{(k)}, \theta_k) \left(\psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) - \psi_{\theta_{k-d_m}}(\hat{s}_{(k)}, \hat{a}_{(k)}) \right) \right\rangle \\ & \quad + \left\langle \nabla J_\lambda(\theta_k), \delta(\hat{x}_{(k)}, \theta_k) \psi_{\theta_{k-d_m}}(\hat{s}_{(k)}, \hat{a}_{(k)}) - \nabla J(\theta_k) \right\rangle. \end{aligned} \quad (121)$$

We bound the first term in (121) as

$$\begin{aligned} & \left\langle \nabla J_\lambda(\theta_k), \delta(\hat{x}_{(k)}, \theta_k) \left(\psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) - \psi_{\theta_{k-d_m}}(\hat{s}_{(k)}, \hat{a}_{(k)}) \right) \right\rangle \\ & \geq -\|\nabla J_\lambda(\theta_k)\|_2 \|\delta(\hat{x}_{(k)}, \theta_k)\|_2 \|\psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) - \psi_{\theta_{k-d_m}}(\hat{s}_{(k)}, \hat{a}_{(k)})\|_2 \\ & \geq -L_V C_\delta \|\psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) - \psi_{\theta_{k-d_m}}(\hat{s}_{(k)}, \hat{a}_{(k)})\|_2 \\ & \geq -L_V C_\delta L_\psi \|\theta_{k-\tau_k} - \theta_{k-d_m}\|_2, \end{aligned} \quad (122)$$

where the last inequality follows Assumption 3. Substituting (122) into (121) gives

$$\begin{aligned} & \left\langle \nabla J_\lambda(\theta_k), \delta(\hat{x}_{(k)}, \theta_k) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) - \nabla J_\lambda(\theta_k) \right\rangle \\ & \geq -L_V C_\delta L_\psi \|\theta_{k-\tau_k} - \theta_{k-d_m}\|_2 + \left\langle \nabla J_\lambda(\theta_k), \delta(\hat{x}_{(k)}, \theta_k) \psi_{\theta_{k-d_m}}(\hat{s}_{(k)}, \hat{a}_{(k)}) - \nabla J_\lambda(\theta_k) \right\rangle. \end{aligned} \quad (123)$$

Then we start to bound the second term in (123). For brevity, we define

$$\Delta_3(x, \theta) := \left\langle \nabla J_\lambda(\theta), \delta(x, \theta) \psi_{\theta_{k-d_m}}(s, a) - \nabla J(\theta) \right\rangle.$$

Throughout the following proof, we use θ, θ', x as shorthand notations of $\theta_k, \theta_{k-d_m}, x_t$ respectively. Define $\tilde{x} := (\tilde{s}_{t-d_m}, \tilde{a}_{t-d_m}, \tilde{s}'_{t-d_m})$ where $\tilde{s}_{t-d_m}, \tilde{a}_{t-d_m}$ are state and action on the auxiliary Markov chain, and $\tilde{s}'_{t-d_m} \sim \mathcal{P}(\cdot | \tilde{s}_{t-d_m}, \tilde{a}_{t-d_m})$ is a virtual sample.

We decompose $\Delta_3(x, \theta)$ as

$$\Delta_3(x, \theta) = \underbrace{\Delta_3(x, \theta) - \Delta_3(x, \theta')}_{I_1} + \underbrace{\Delta_3(x, \theta') - \Delta_3(\tilde{x}, \theta')}_{I_2} + \underbrace{\Delta_3(\tilde{x}, \theta')}_{I_3}.$$

We first bound I_1 as

$$\begin{aligned}
 |I_1| &= |\Delta_3(x, \theta) - \Delta_3(x, \theta')| \\
 &= |\langle \nabla J_\lambda(\theta), \delta(x, \theta) \psi_{\theta'}(s, a) \rangle - \langle \nabla J_\lambda(\theta), \nabla J(\theta) \rangle - \langle \nabla J_\lambda(\theta'), \delta(x, \theta') \psi_{\theta'}(s, a) \rangle + \langle \nabla J_\lambda(\theta'), \nabla J(\theta') \rangle| \\
 &\leq |\langle \nabla J_\lambda(\theta), \delta(x, \theta) \psi_{\theta'}(s, a) \rangle - \langle \nabla J_\lambda(\theta'), \delta(x, \theta') \psi_{\theta'}(s, a) \rangle| + |\langle \nabla J_\lambda(\theta), \nabla J(\theta) \rangle - \langle \nabla J_\lambda(\theta'), \nabla J(\theta') \rangle| \\
 &\leq |\langle \nabla J_\lambda(\theta), \delta(x, \theta) \psi_{\theta'}(s, a) \rangle - \langle \nabla J_\lambda(\theta'), \delta(x, \theta') \psi_{\theta'}(s, a) \rangle| + 2L_V L_\lambda \|\theta - \theta'\|_2, \quad (124)
 \end{aligned}$$

where the last inequality uses the fact that $\langle \nabla J_\lambda(\theta), \nabla J(\theta) \rangle$ is $2L_V L_\lambda$ -lipschitz continuous. We bound the first term in (124) as

$$\begin{aligned}
 &|\langle \nabla J_\lambda(\theta), \delta(x, \theta) \psi_{\theta'}(s, a) \rangle - \langle \nabla J_\lambda(\theta'), \delta(x, \theta') \psi_{\theta'}(s, a) \rangle| \\
 &\leq |\langle \nabla J_\lambda(\theta), \delta(x, \theta) \psi_{\theta'}(s, a) \rangle - \langle \nabla J_\lambda(\theta), \delta(x, \theta') \psi_{\theta'}(s, a) \rangle| \\
 &\quad + |\langle \nabla J_\lambda(\theta), \delta(x, \theta') \psi_{\theta'}(s, a) \rangle - \langle \nabla J_\lambda(\theta'), \delta(x, \theta') \psi_{\theta'}(s, a) \rangle| \\
 &= |\langle \nabla J_\lambda(\theta), (\delta(x, \theta) - \delta(x, \theta')) \psi_{\theta'}(s, a) \rangle| + |\langle \nabla J_\lambda(\theta) - \nabla J_\lambda(\theta'), \delta(x, \theta') \psi_{\theta'}(s, a) \rangle| \\
 &\leq L_V C_\psi |\delta(x, \theta) - \delta(x, \theta')| + C_p \|\nabla J_\lambda(\theta) - \nabla J_\lambda(\theta')\|_2 \\
 &= L_V C_\psi |\gamma(V_{\pi_\theta}(s') - V_{\pi_{\theta'}}(s')) + V_{\pi_{\theta'}}(s) - V_{\pi_\theta}(s)| + C_p \|\nabla J_\lambda(\theta) - \nabla J_\lambda(\theta')\|_2 \\
 &\leq L_V C_\psi (\gamma |V_{\pi_\theta}(s') - V_{\pi_{\theta'}}(s')| + |V_{\pi_{\theta'}}(s) - V_{\pi_\theta}(s)|) + C_p \|\nabla J_\lambda(\theta) - \nabla J_\lambda(\theta')\|_2 \\
 &\leq L_V C_\psi (\gamma L_V \|\theta - \theta'\|_2 + L_V \|\theta' - \theta\|) + C_p L_\lambda \|\theta - \theta'\|_2 \\
 &= (C_p L_\lambda + (1 + \gamma) L_V^2 C_\psi) \|\theta - \theta'\|_2. \quad (125)
 \end{aligned}$$

Substituting the above inequality into (124) gives the lower bound of I_1 :

$$I_1 \geq - (C_p L_\lambda + (1 + \gamma) L_V^2 C_\psi + 2L_V L_\lambda) \|\theta - \theta'\|_2.$$

First we bound I_2 as

$$\begin{aligned}
 \mathbb{E}[I_2|\theta', \hat{s}_{t-m+1}] &= \mathbb{E}[\Delta_3(x, \theta') - \Delta_3(\tilde{x}, \theta')|\theta', \hat{s}_{t-m+1}] \\
 &\geq -|\mathbb{E}[\Delta_3(x, \theta')|\theta', \hat{s}_{t-m+1}] - \mathbb{E}[\Delta_3(\tilde{x}, \theta')|\theta', \hat{s}_{t-m+1}]| \\
 &\geq -\sup_x |\Delta_3(x, \theta')| \|\mathbb{P}(x \in \cdot | \theta', \hat{s}_{t-m+1}) - \mathbb{P}(\tilde{x} \in \cdot | \theta', \hat{s}_{t-m+1})\|_{TV} \\
 &\geq -2L_V(C_p + L_V) d_{TV}(\mathbb{P}(x \in \cdot | \theta', \hat{s}_{t-m+1}), \mathbb{P}(\tilde{x} \in \cdot | \theta', \hat{s}_{t-m+1})) \\
 &\geq -L_V(C_p + L_V) |\mathcal{A}| L_\pi \sum_{i=\tau_k}^{d_m} \mathbb{E}[\|\theta_{k-i} - \theta_{k-d_m}\|_2 | \theta', \hat{s}_{t-m+1}], \quad (126)
 \end{aligned}$$

where the second inequality is due to the definition of TV norm, the last inequality is due to (28) in Lemma 3, and the second last inequality follows the fact that

$$|\Delta_3(x, \theta')| \leq \|\nabla J_\lambda(\theta)\|_2 (\|\delta(x, \theta) \psi_{\theta_{k-d_m}}(s, a)\|_2 + \|\nabla J(\theta)\|_2) \leq L_V(C_p + L_V). \quad (127)$$

Taking total expectation on both sides of (126) yields

$$\mathbb{E}[I_2] \geq -L_V(C_p + L_V) |\mathcal{A}| L_\pi \sum_{i=\tau_k}^{d_m} \mathbb{E}\|\theta_{k-i} - \theta_{k-d_m}\|_2. \quad (128)$$

Define $\bar{x} := (\bar{s}, \bar{a}, \bar{s}')$, where $\bar{s} \sim d_{\theta'}$, $\bar{a} \sim \pi_{\theta'}$ and $\bar{s}' \sim \mathcal{P}$. Then we can further decompose I_3 as

$$\mathbb{E}[I_3|\theta', \hat{s}_{t-m+1}] = \mathbb{E}[\Delta_3(\tilde{x}, \theta') - \Delta_3(\bar{x}, \theta')|\theta', \hat{s}_{t-m+1}] + \mathbb{E}[\Delta_3(\bar{x}, \theta')|\theta', \hat{s}_{t-m+1}]. \quad (129)$$

The first term in (129) can be bounded as

$$\begin{aligned} & \mathbb{E}[\Delta_3(\tilde{x}, \theta') - \Delta_3(\bar{x}, \theta')|\theta', \hat{s}_{t-m+1}] \\ & \geq -|\mathbb{E}[\Delta_3(\tilde{x}, \theta')|\theta', \hat{s}_{t-m+1}] - \mathbb{E}[\Delta_3(\bar{x}, \theta')|\theta', \hat{s}_{t-m+1}]| \\ & \geq -\sup_x |\Delta_3(x, \theta')| \|\mathbb{P}(\tilde{x} \in \cdot|\theta', \hat{s}_{t-m+1}) - \mathbb{P}(\bar{x} \in \cdot|\theta', \hat{s}_{t-m+1})\|_{TV} \\ & \geq -2L_V(C_p + L_V)d_{TV}(\mathbb{P}(\tilde{x} \in \cdot|\theta', \hat{s}_{t-m+1}), \mathbb{P}(\bar{x} \in \cdot|\theta', \hat{s}_{t-m+1})) \\ & = -2L_V(C_p + L_V)d_{TV}(\mathbb{P}(\tilde{x} \in \cdot|\theta', \hat{s}_{t-m+1}), d_{\theta'} \otimes \pi_{\theta'} \otimes \mathcal{P}) \end{aligned} \quad (130)$$

where the second inequality follows the definition of TV -norm, and the third one follows (127).

The auxiliary Markov chain with policy $\pi_{\theta'}$ starts from initial state $\hat{s}_{t-m+1}$, and $\tilde{s}_t$ is the $(m-1)$ th state on the chain. Following Lemma 2, we have:

$$\begin{aligned} d_{TV}(\mathbb{P}(\tilde{x} \in \cdot|\theta', \hat{s}_{t-m+1}), d_{\theta'} \otimes \pi_{\theta'} \otimes \mathcal{P}) &= d_{TV}(\mathbb{P}((\tilde{s}_t, \tilde{a}_t, \tilde{s}_{t+1}) \in \cdot|\theta', \hat{s}_{t-m+1}), d_{\theta'} \otimes \pi_{\theta'} \otimes \mathcal{P}) \\ &\leq \kappa\rho^{m-1}. \end{aligned}$$

Substituting the last inequality into (130) yields

$$\mathbb{E}[\Delta_3(\tilde{x}, \theta') - \Delta_3(\bar{x}, \theta')|\theta', \hat{s}_{t-m+1}] \geq -2L_V(C_p + L_V)\kappa\rho^{m-1}. \quad (131)$$

The second term in (129) is simply

$$\mathbb{E}[\Delta_3(\bar{x}, \theta')|\theta', \hat{s}_{t-m+1}] = \mathbb{E}[\langle \nabla J_\lambda(\theta'), \delta(\bar{x}, \theta')\psi_{\theta'}(\bar{s}, \bar{a}) - \nabla J(\theta') \rangle|\theta', \hat{s}_{t-m+1}] = 0. \quad (132)$$

Substituting (131) and (132) into (129) yields

$$\mathbb{E}[I_3|\theta', \hat{s}_{t-m+1}] \geq -2L_V(C_p + L_V)\kappa\rho^{m-1}. \quad (133)$$

Taking total expectation on $\Delta_3(x, \theta)$ and collecting lower bounds of I_1, I_2, I_3 yield

$$\begin{aligned} \mathbb{E}[\Delta_3(x, \theta)] &\geq -(C_p L_\lambda + (1 + \gamma)L_V^2 C_\psi + 2L_V L_\lambda) \mathbb{E}\|\theta_k - \theta_{k-d_m}\|_2 \\ &\quad - L_V(C_p + L_V)|\mathcal{A}|L_\pi \sum_{i=\tau_k}^{d_m} \mathbb{E}\|\theta_{k-i} - \theta_{k-d_m}\|_2 - 2L_V(C_p + L_V)\kappa\rho^{m-1}. \end{aligned} \quad (134)$$

Taking total expectation on (123) and substituting the above inequality into it yield

$$\begin{aligned} & \mathbb{E}\left\langle \nabla J_\lambda(\theta_k), \delta(\hat{x}_{(k)}, \theta_k)\psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) - \nabla J(\theta_k) \right\rangle \\ & \geq -D_6 \mathbb{E}\|\theta_{k-\tau_k} - \theta_{k-d_m}\|_2 - D_7 \mathbb{E}\|\theta_k - \theta_{k-d_m}\|_2 - D_8 \sum_{i=\tau_k}^{d_m} \mathbb{E}\|\theta_{k-i} - \theta_{k-d_m}\|_2 - D_9 \kappa\rho^{m-1}, \end{aligned} \quad (135)$$

where $D_6 := L_V C_\delta L_\psi$, $D_7 := C_p L_\lambda + (1 + \gamma)L_V^2 C_\psi + 2L_V L_\lambda$, $D_8 := L_V(C_p + L_V)|\mathcal{A}|L_\pi$, $D_9 := 2L_V(C_p + L_V)$. $\blacksquare$

The Implementation of Asynchronous Advantage Actor-Critic with Stigmergy in Network-assisted Multi-agent System

Kun Chen*, Rongpeng Li*, Zhifeng Zhao[†], and Honggang Zhang*

*Zhejiang University, [†]Zhejiang Lab

Email: {beatther,lirongpeng, honggangzhang}@zju.edu.cn, zhaozf@zhejianglab.com

Abstract—Multi-agent system (MAS) needs to mobilize multiple simple agents to complete complex tasks. However, it is difficult to coherently coordinate distributed agents by means of limited local information. In this paper, we propose a decentralized collaboration method named as “stigmergy” in network-assisted MAS, by exploiting digital pheromones (DP) as an indirect medium of communication and utilizing deep reinforcement learning (DRL) on top. Correspondingly, we implement an experimental platform, where KHEPERA IV robots form targeted specific shapes in a decentralized manner. Experimental results demonstrate the effectiveness and efficiency of the proposed method. Our platform could be conveniently extended to investigate the impact of network factors (e.g., latency, data rate, etc) on the level of collective intelligence.

Index Terms—multi-agent system, stigmergy mechanism, digital pheromones, deep reinforcement learning, KHEPERA IV robots.

I. INTRODUCTION

With the development of artificial intelligence (AI) and the Internet, multi-agent system (MAS) has received the attention on many researchers and become a research hotspot in the field of control disciplines and AI [1]. MAS is a large-scale system composed of numerous intelligent agents with perception, communication, computation, and execution capabilities. Because of its autonomous and distributed characteristics [2], connected intelligent agents can collaborate to complete high-complexity tasks. Based on this property of MAS, it has been widely used in various scenarios such as swarm robots control, UAV collaboration and satellite formation [3], [4].

However, in a distributed MAS, due to the limited communication resources and computing capability, agents can neither communicate with each other in real-time regularly nor update the status of controller in a timely manner. Besides, the fact that actions performed by distributed agents inevitably change the surrounding environment, which further affects the decisions on other agents and produces a time-lasting impact. This interaction between agents and non-stationary environment makes it difficult to model the entire system.

On the other hand, there have been some efforts in a centralized controller, which receives information generated during the interaction between all distributed agents and the environment, such as the state of the environment observed by distributed agents at each moment decision-making, etc [5]. In this way, the centralized controller can be regarded as a

joint agent, and the environmental state that it receives at each moment is a collection of states observed by all agents, which can approximate the research on multi-agents by a simplified single-agent solution. Nevertheless, in practical scenarios such as intelligent transportation systems, smart grids and sensor networks, the existence of centralized controllers cannot be guaranteed [6]–[8]. In addition, for MAS with large action space and state space, the introduction of a centralized controller will suffer from the curse of dimensionality, significantly incurring the amount of calculation of a centralized controller and even being impossible to calculate.

Stigmergy is an indirect communication mechanism existing on social insect species, which relies on “pheromone” deposited in the environment to coordinate individual insects’ activities, and exhibit strong robustness with simple form [9]. Stigmergy mechanism is widely used in the implementation of collective intelligence. In order to seek the characteristic of “pheromone” in the engineering system, digital pheromone (DP) is employed to simulate the existence form of pheromone in the environment, acting as medium in stigmergy mechanism to enable indirect communication with distributed agents [10]. Agents judge the state of the surrounding environment, leave a certain number of digital pheromones at the current location, and generate a DP map by integrating the digital pheromones in the environment. As time goes by, the increasingly refined DP map enables all distributed agents to gather toward the destinations with high digital pheromones volume.

However, using stigmergy mechanism alone to guide the behavior of agents often fails to complete the task efficiently. In the field of AI, a generalized purpose of reinforcement learning (RL) is to continuously try and optimize its decision-making mechanism by interacting with agent and environment, so as to better adapt to the particular environment [11], [12]. Recently, with deep reinforcement learning (DRL) combining RL and deep neural networks (DNN) making a splash in various fields, many advanced DRL algorithms have greatly improved the efficiency with which agents learn from environment constantly [13], [14]. On top of DRL, multi-agent reinforcement learning (MARL) has manifested itself in many application scenarios and complex modeling [15], [16], which trains multiple agents in a specific manner to accomplish tricky mission collaboratively.

Meanwhile, inspired by federated learning which designs a

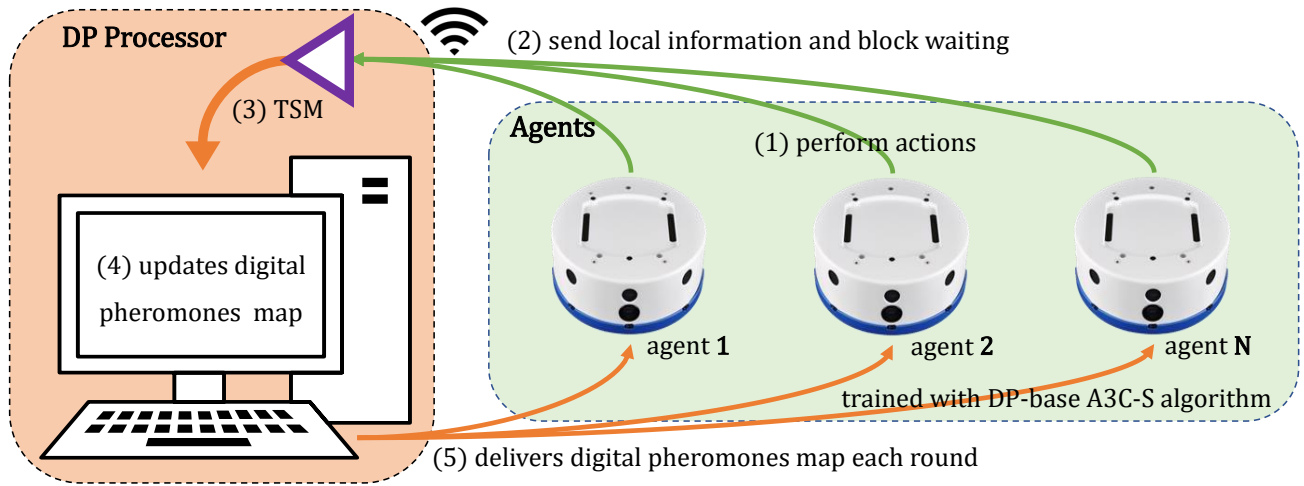


Fig. 1. The decentralized control framework with the DP-based A3C-S algorithm in network-assisted MAS.

virtual center to collaborate different parties without exchanging data [17], we propose a decentralized control framework based on stigmergy in network-assisted MAS by establishing a WLAN. Besides, in order to strengthen the sense of collaboration between agents, we combine it with asynchronous advantage actor-critic (A3C) algorithm, which is named as asynchronous advantage actor-critic with stigmergy (A3C-S) algorithm, to train agents to efficiently handle complex tasks in MAS [18].

To verify the validity of this framework, we design an experimental scenario where several randomly placed agents arrive at tracking points with as few rounds as possible in a certain area. After extensive simulation experiments to train and test, we exploit KHEPERA IV robots to implement the well-trained model in practical settings and introduce temporal synchronization mechanism (TSM) to overcome the problem of time asynchrony in robot information exchanges.

The remainder of this paper is mainly organized as follows. In Section II, we elaborate on the scenario and purpose of the experiment, and present the decentralized control framework in network-assisted MAS, including composition and execution procedure of system framework. In Section III, we mathematically discuss the details of the DP-based A3C-S Algorithm included DP in stigmergy and the A3C-S algorithm, and describe the design philosophy of TSM. In Section IV, the well-trained model is deployed on KHEPERA IV robots to verify the efficiency of system framework via extensive experiments. Finally, we conclude this paper with a summary.

II. SYSTEM MODEL

In the experimental scenario designed in this paper, the objective is that agents who are randomly placed within a certain area reach the target points in as few rounds as possible to form targeted specific shapes. Based on implicit information such as DP, agents can obtain the local environmental state through indirect communication. In contrast to the approach from planning a global course of action at the outset, we

expect that agents move in the general direction of a target point originally, and determine to the specific target point based on the judgment on their surroundings. Depending on the properties of DP in stigmergy, agents are capable to converge on a high probability towards a place with high digital pheromones volume in their favor.

However, directly applying DP in stigmergy to guide agents may cause agents that have occupied the target points become immobile, making other agents spend more rounds seeking unoccupied target points and incurring the system inefficiency. The intuitive solution is to keep all distributed agents in an “active” state at all times. In other words, even when agents reach a location with the highest pheromone volume in the observation range, they still need to choose to “move” or “stay” depending on the state of their surroundings.

In order to achieve this more flexible system, we propose a decentralized control framework with the DP-based A3C-S algorithm in network-assisted MAS, as shown in Fig. 1. It consists of two parts, that is, DP processor and agents, which form a WLAN to connect multiple distributed agents to establish communication. Furthermore, since agents and DP processor often communicate with each other out of synchronization in actual scenarios, we design and calibrate a TSM manner to align the agents’ time of performing actions and communication to overcome this conundrum.

- **DP processor.** DP processor collects local information on the procession of TSM, and calculates the digital pheromones of agents’ location based on the distance between agents and target points. With simulating the superposition, decay and diffusion properties of pheromone, DP processor generates a DP map that contains the distribution of digital pheromones for providing more relevant information of the state space in the whole activity area, which is delivered to agents each round to be foundation for decision-making.
- **Agents.** Each agent only acquires limited DP map around itself via interaction with DP processor, which determine

the location of next action without directly communicating with other agents. Each agent has a limited perception range of digital pheromones, and the ability of perception decreases along with the increase in the distance, which attracts agents to move towards at target points gradually because of the property that agents have a tendency to congregate to high pheromone volume places. Besides, distributed agents are trained with A3C-S algorithm to strengthen the collaboration and accomplish tasks efficiently.

Fig. 1 illustrates the whole execution procedure of the system framework.

- 1) Agents sense the state of surrounding environment based on DP map, compute the next decision and perform actions.
- 2) While agents are acting, they simultaneously upload local information (including the current location and the decision) to the DP processor and wait for the feedback information on DP processor in a blocking manner.
- 3) The information uploaded by agents is processed by TSM, and the entire information is delivered to the DP processor.
- 4) With simulating the superposition, decay and diffusion properties of pheromone, DP processor merges the collected information and updates DP map.
- 5) DP processor delivers the updated DP map to each agent, and repeats recursively until all agents reach the target points.

III. THE DP-BASED A3C-S ALGORITHM

This section elaborates on the problems of experimental scenario and the corresponding solutions, including DP in stigmergy, the A3C-S algorithm and TSM.

A. DP in Stigmergy

In the system model of this paper, DP is regarded as the medium in stigmergy mechanism and is constantly updated after agents perform actions, which is used as a bridge to instruct distributed agents with a more efficient manner to seek the target points. Inspired by the fact that the chemical pheromone left by different ants during swarm foraging can be superposed together to enhance the total influence, we model the accumulation of digital pheromones left by different agents as a linear superposition, which can be express as:

$$\mu_{(i,j)}(t) = \begin{cases} \mu_{(i,j)}(t-1) + a, & \text{if } (i,j) \text{ is target point,} \\ \mu_{(i,j)}(t-1) \cdot b, & \text{otherwise.} \end{cases} \quad (1)$$

where $\mu_{(i,j)}$ represents the amount of digital pheromones at the position (i,j) , a represents the fixed amount of digital pheromones left by an agent at a time, and $b \in (0, 1)$ denotes the decaying rate. After being superposed and updated, digital pheromones are diffused to the surrounding area at a fixed *diffusion rate*, and reduce the number of digital pheromones in the position already occupied by agents with a fixed *decay rate*. Similar to the ant colony searching for food, based on

the local state space, each agent needs to choose a direction that is most beneficial to itself from the direction of several potential candidates within its perception range (to gather at location with high pheromone volume).

B. The A3C-S Algorithm

During the experiment, we observe that modeling a DP map can make all distributed agents gather in the direction of high pheromone volume, but they frequently cannot cooperate well to accomplish relatively complex tasks efficiently. Therefore, we utilize A3C-S algorithm to train agents to efficiently handle complex tasks.

In the agents training phase, it is particularly essential to define the input of the neural network and the rewards. With the guidance of DP map obtained from the DP processor, each agent can perceive the number of digital pheromones within a certain range. We regard any block (unit area) filled with digital pheromones as an "attractor" in the local environment, which has an attractive effect on nearby agents to effectively observe the local state space. Such an attractor can be expressed as:

$$P_{(i,j),(m,n)} = \frac{\mu_{(m,n)} \cdot D(d_{(i,j),(m,n)})}{\sum_{(m,n) \in \zeta_{(i,j)}} \mu_{(m,n)} \cdot D(d_{(i,j),(m,n)})} \quad (2)$$

where $P_{(i,j),(m,n)}$ is the probability that agent chooses the attractor at position (m,n) when the agent is at position (i,j) . $\zeta_{(i,j)}$ is the set of attractors within the sensing range of agent. $d_{(i,j),(m,n)}$ is the Euclidean distance between the position of agent and attractor. $D(\cdot)$ is a monotonous function used to decrease the effect of digital pheromones as the inter-distance $d_{(i,j),(m,n)}$ increases [10]. The attractor of each unit block around the agent is calculated, which is encoded as the input of neural network to make corresponding decisions under various environmental conditions.

Based on the calculated attractor, distributed agents perform actions to move closer to the position of high digital pheromones volume. Since the purpose of experiment is that all agents reach the target points in as few rounds as possible, a reward function is needed to measure the global impact of the agents' action. We define the global minimum distance as the degree of proximity between the swarm of agents and target points. Therefore, when agents complete a round, a smaller global distance means that the swarm of agents is moving closer to the target points. Positive feedback is given to the agents to ensure them to learn the behavior that is beneficial to the global system. Conversely, the system is moving in a direction that is detrimental to the experimental purpose, and agents are given a penalty to prevent them from performing the actions later. The above process can be expressed as follows:

$$R(t) = \gamma \cdot \left(\min_{\alpha \in \Phi, \beta \in \Xi} d_{\alpha,\beta}(t-1) - \min_{\alpha \in \Phi, \beta \in \Xi} d_{\alpha,\beta}(t) \right) \quad (3)$$

where $R(t)$ represents the global reward after agents perform actions. Φ and Ξ belong to the set of the agents' and the target

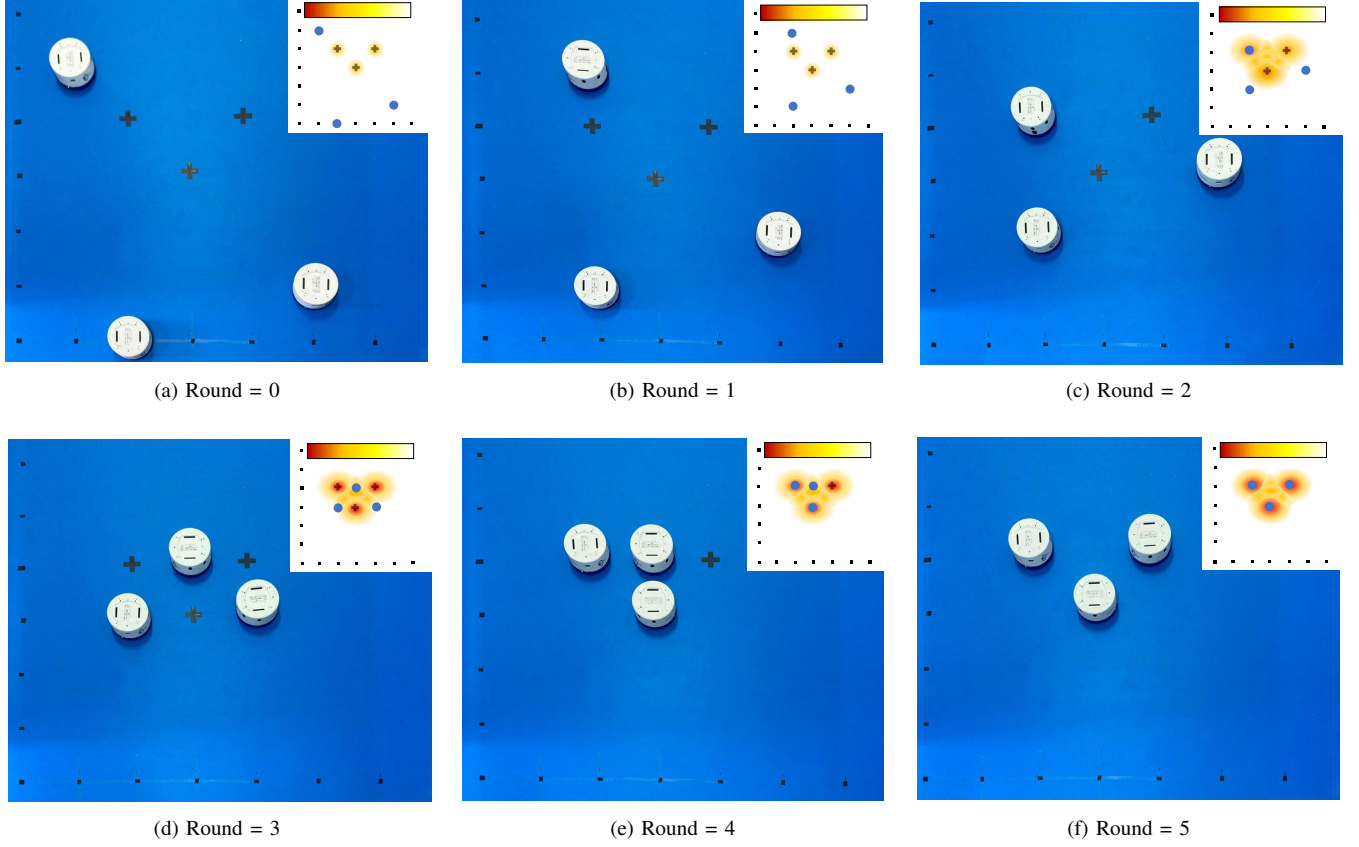


Fig. 2. The experimental platform of 3 KHEPERA IV robots with the system model. Marks "+" in the figure represents the target points. The upper right corner of each figure represents the DP map. The darker the color, the higher the digital pheromones volume.

points' position respectively. $d_{\alpha,\beta}$ is the Manhattan distance between agent and target point. γ is a constant matching coefficient, whose function is to calculate the reward value within a certain range. By training agents to adopt different strategies in different environments, the overall efficiency of decision-making is greatly improved compared with methods of only using DP maps to guide agents to seek target points (that is, enhancing agents' sense of cooperation).

C. Temporal Synchronization Mechanism

When deploying the system model to the real swarm robots, the timing of performing actions and communication varies each round, causing the updated DP map to be scrambled, which will result in sending the wrong DP map to agents to perform unanticipated actions.

To prevent information mismatch when agents communicate with DP processor, TSM is introduced to follow strict timing requirements to overcome the above problems when interact between distributed agents and DP processor. Similar to a buffer, DP processor stores the message received in the memory before receiving all local messages from agents, and process all messages uniformly when they have been received. DP processor also sets up a mechanism to discard and re-transmit timeouts to prevent system crashes in the event of packet loss. The role of TSM is that system sacrifices part

of the agents' moving time to coordinate the behavior of distributed agents, as well as reduces the probability of collision between the agents to some extent. We will investigate the impact on asynchronous methods of the future.

IV. EXPERIMENTAL RESULTS

In this part, we establish an experimental platform based on a DP processor and 3 KHEPERA IV robots, and deploy the system framework with the DP-based A3C-S algorithm to the swarm of KHEPERA IV robots. In experiment, a variety of models are also deployed on this experimental platform, and compared to the model of this paper to verify the efficiency of these models by evaluating performance in terms of degree of completion (DoC) and number of total steps. Our Demonstrator implementation is shown in Fig. 2 and the result of experiment is shown in Fig. 3 and Fig. 4. We specifically introduce involved experimental platform, deployment and result as follows.

A. Experimental Platform

We use a PC with WiFi module as the DP processor, and three KHEPERA IV robots as distributed agents. The user datagram protocol (UDP) is utilized to establish the connection between DP processor and distributed agents.

- **DP processor.** Being an implementer of DP processor does not require high performance, just the ability of communication and simple calculation. DP processor is employed as the server side of UDP to send updated DP maps to agents using the manner of multicast.
- **Agents.** The KHEPERA IV is a compact robot designed for any indoor lab application, which is as the client side of UDP to send the local status information to DP processor in system framework. Since the robots are not equipped with positioning modules, we employ odometry and leverage dead reckoning method to estimate the position [15]. By informing robots of initial position and obtaining encoder values of fixed time intervals, the robots' distance and angle for movement are calculated to enable positioning function. The CPU processor of each KHEPERA IV robot is Texas Instruments DaVinci DM3730 @ 800MHz.

B. Experimental Deployment

To verify the efficiency of decentralized control framework with DP-based A3C-S algorithm in network-assisted MAS, we design an experimental scenario where three randomly placed robots within a certain area reach three target points with as few rounds as possible, and the robots can only perform one action per round. We use a simulation software to simulate the behavior of the robots in a real environment, apply the system framework to train the robots centrally, and load the results of training to each KHEPERA IV robot distributedly. Fig. 2 shows a complete experiment with KHEPERA IV robots, which is described as follows.

First, we initialize the robots by placing them randomly within a certain area and informing the initial position. Similar to the real environment, we assume that a small number of digital pheromones exist at the target points, as shown in Fig. 2a. In the earlier phase, since the robots are randomly scattered and far away from the target points, the collaborative action between robots is not yet involved, but the general direction of the increase in digital pheromones is perceived, which motivates the robots to approach the target points more strongly, and attract more robots to approach the target points, as shown in Fig. 2b and Fig. 2c. As the number of rounds increase, distributed robots move into a narrow space closed to the target points, where each robot is trained with DP-based A3C-S algorithm to compute the optimal action under the current environmental conditions. In particular, during the experiment, even if server distributed robots have already reached the target points, in order to achieve the global optimality efficiently (reach the target points with as few rounds as possible), robots who have already reached the target points will continue to perform actions based on the state of environment, as shown in Fig. 2d and Fig. 2e. This design can not only improve the overall performance of the system, but also make the whole system more flexible. Fig. 2f shows the position of robots and distribution of DP map at the end of experiment.

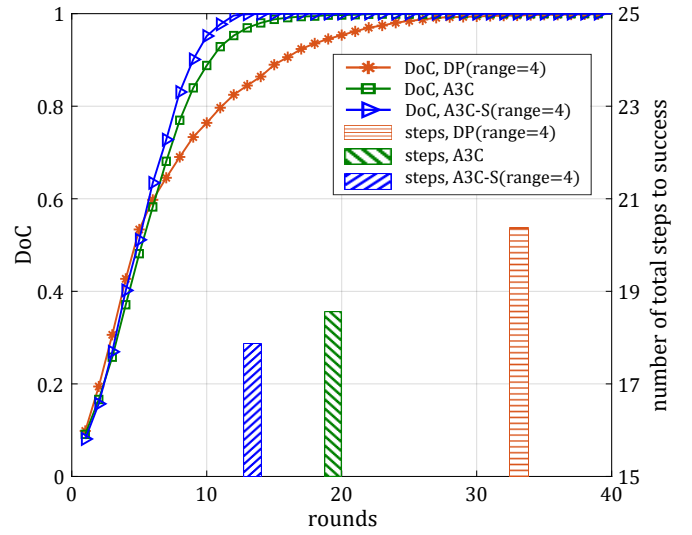


Fig. 3. The performance of DP-based A3C-S algorithm compared with the cases of adopting DP or A3C algorithm only. The performance of evaluating performance is in terms of DoC and number of total steps.

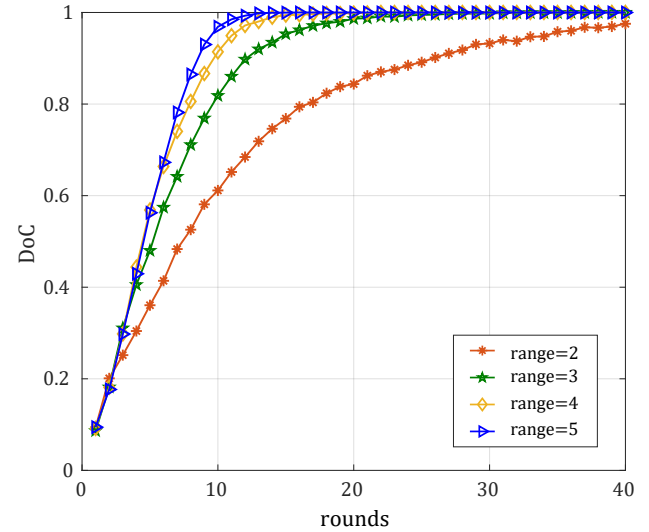


Fig. 4. The effect of changing the range of digital pheromones perception on experimental performance.

C. Experimental Results

We evaluate the performance of the aforementioned procedures by repeating the experiments above, and compare approaches using the DP-based A3C-S algorithm with the cases of adopting DP or A3C algorithm only, and Fig. 3 presents the numerical results in detail. When DoC arrives at 1, it means that all robots have reached the target points, and the histogram shows the number of rounds and total steps taken at completing the purpose. It can be observed from the figure that in the first few rounds of the experiment, applying the framework designed in this paper produce similar to the results as using only DP in stigmergy guidance, but as the distance

between robots became smaller, the presence or absence of swarm collaboration becomes particularly important.

The robot perceives a certain range of digital pheromones as foundation for decision making. In our experiments, we observed that changing the range of robots' perception has significant impact on the efficiency with which these distributed robots complete task, as shown in Fig. 4. When the perceptual range is short (e.g., range = 2), the system converges slowly and fails to complete the purpose even within the specified maximum number of steps; when the perceptual range increases above a certain threshold (e.g., range = 5), the performance of the system does not change significantly and may even attract more noisier information to interfere with the decision.

In summary, DP can attract robots to places with high pheromones volume at the beginning, and A3C algorithm can coordinate the behavior of agents when robots close to the target points. When selecting the appropriate perceptual range for the robot, we can conclude that compared to the cases of adopting DP or A3C algorithm only, the proposed DP-based A3C-S algorithm yields superior performance (shown in the Fig. 3 as the number of rounds when DoC first reached 1).

V. CONCLUSION AND DISCUSSIONS

In this paper, we have demonstrated the implementation means for achieving the decentralized control framework in network-assisted MAS. Furthermore, we have verified the effectiveness and efficiency of exploiting the DP-based A3C-S algorithm in a practical MAS with KHEPERA IV robots.

We boldly argue that the cooperation of collective intelligence implied in this paper could have significant merits in the future, especially in the field of wireless communications networks, and have a lot of issues to be addressed.

- As the working frequency becomes higher, the coverage of base stations gradually shrink. Along with the increasingly maturity of the internet of vehicles, it will be of great importance to coordinate the BSs and network entities by collective intelligence.
- How to develop the collective intelligence remains an issue? Ideas from brains could contribute a lot, as similar mechanism like stigmergy has been found and manifested [10].

Therefore, we hope this paper could act as a starting point to solicit interesting ideas and promote the development.

ACKNOWLEDGMENT

This work was supported in part by National Key R&D Program of China (2019YFB1802900), National Natural Science Foundation of China (No. 61701439, 61731002), Zhejiang Key R&D Plan (No. 2019C01002, 2019C03131), the Project sponsored by Zhejiang Lab (No. 2019LC0AB01), Zhejiang Provincial Natural Science Foundation of China (No. LY20F010016), the Huawei Cooperation Project.

REFERENCES

- [1] W. Ren, R. W. Beard, and E. M. Atkins, "Information consensus in multivehicle cooperative control," *IEEE Control Systems*, vol. 27, no. 2, pp. 71–82, 2007.
- [2] R. Li, Z. Zhao, X. Xu, F. Ni, and H. Zhang, "The collective advantage for advancing communications and intelligence," *IEEE Wireless Communications*, 2020.
- [3] L. Matignon, L. Jeanpierre, and A.-I. Mouaddib, "Coordinated multi-robot exploration under communication constraints using decentralized markov decision processes," in *Twenty-sixth AAAI Conference on Artificial Intelligence*, 2012.
- [4] A. K. Whitten, H.-L. Choi, L. B. Johnson, and J. P. How, "Decentralized task allocation with coupled constraints in complex missions," in *Proceedings of the 2011 American Control Conference*. IEEE, 2011, pp. 1642–1649.
- [5] M. Tan, "Multi-agent reinforcement learning: Independent vs. cooperative agents," in *Proceedings of the Tenth International Conference on Machine Learning*, 1993, pp. 330–337.
- [6] J. L. Adler and V. J. Blue, "A cooperative multi-agent transportation management and route guidance system," *Transportation Research Part C: Emerging Technologies*, vol. 10, no. 5–6, pp. 433–454, 2002.
- [7] D. S. Callaway and I. A. Hiskens, "Achieving controllability of electric loads," *Proceedings of the IEEE*, vol. 99, no. 1, pp. 184–199, 2010.
- [8] M. Rabbat and R. Nowak, "Distributed optimization in sensor networks," in *Proceedings of the 3rd International Symposium on Information Processing in Sensor Networks*, 2004, pp. 20–27.
- [9] H. V. D. Parunak, S. A. Brueckner, and J. Sauter, "Digital pheromones for coordination of unmanned vehicles," in *International Workshop on Environments for Multi-Agent Systems*. Springer, 2004, pp. 246–263.
- [10] X. Xu, Z. Zhao, R. Li, and H. Zhang, "Brain-inspired stigmergy learning," *IEEE Access*, vol. 7, pp. 54 410–54 424, 2019.
- [11] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. MIT press, 2018.
- [12] G. Tesauro, "Temporal difference learning and td-gammon," *Communications of the ACM*, vol. 38, no. 3, pp. 58–68, 1995.
- [13] K. Arulkumaran, M. P. Deisenroth, M. Brundage, and A. A. Bharath, "Deep reinforcement learning: A brief survey," *IEEE Signal Processing Magazine*, vol. 34, no. 6, pp. 26–38, 2017.
- [14] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [15] L. Busoniu, R. Babuska, and B. De Schutter, "A comprehensive survey of multiagent reinforcement learning," *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, vol. 38, no. 2, pp. 156–172, 2008.
- [16] M. L. Littman, "Markov games as a framework for multi-agent reinforcement learning," in *Machine Learning Proceedings 1994*. Elsevier, 1994, pp. 157–163.
- [17] H. Brendan McMahan, E. Moore, D. Ramage, S. Hampson, and B. Agüera y Arcas, "Communication-Efficient Learning of Deep Networks from Decentralized Data," *arXiv e-prints*, p. arXiv:1602.05629, Feb. 2016.
- [18] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. Harley, T. P. Lillicrap, D. Silver, and K. Kavukcuoglu, "Asynchronous methods for deep reinforcement learning," in *Proceedings of the 33rd International Conference on Machine Learning - Volume 48*, ser. ICML'16. JMLR.org, 2016, p. 1928–1937.


```
#!/usr/bin/env python3
"""
A minimal A3C implementation for Gymnasium using PyTorch.
- Environment: CartPole-v1 (default; change via --env).
- No nested functions (per style constraints).
- Uses torch.multiprocessing for async workers.
Run:
python a3c_cartpole.py --env CartPole-v1 --workers 8 --t-max 5 --max-steps 200000
"""
import argparse
import time
import math
import random
from dataclasses import dataclass
from typing import Tuple, List

import gymnasium as gym
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.multiprocessing as mp

# -----
# Utilities
# -----

def set_seed_everywhere(seed: int):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)

@dataclass
class A3CConfig:
    env_id: str = "CartPole-v1"
    gamma: float = 0.99
    lr: float = 1e-3
    workers: int = 8
    t_max: int = 5 # number of steps before an update (n-step target)
    max_global_steps: int = 2_00_000
    entropy_coef: float = 0.01
    value_loss_coef: float = 0.5
    grad_norm_clip: float = 40.0
    seed: int = 1
    device: str = "cpu"

# -----
# Model
# -----

class ActorCritic(nn.Module):
    def __init__(self, obs_dim: int, act_dim: int):
        super().__init__()
        self.fc1 = nn.Linear(obs_dim, 128)
        self.fc2 = nn.Linear(128, 128)
        self.policy = nn.Linear(128, act_dim)
        self.value = nn.Linear(128, 1)

        for m in self.modules():
            if isinstance(m, nn.Linear):
                nn.init.orthogonal_(m.weight, gain=math.sqrt(2))
                nn.init.constant_(m.bias, 0.0)
            nn.init.orthogonal_(self.policy.weight, gain=0.01)
            nn.init.constant_(self.policy.bias, 0.0)
            nn.init.orthogonal_(self.value.weight, gain=1.0)
            nn.init.constant_(self.value.bias, 0.0)

    def forward(self, x: torch.Tensor) -> Tuple[torch.Tensor, torch.Tensor]:
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        logits = self.policy(x)
        value = self.value(x).squeeze(-1)
        return logits, value

# -----
# Shared Adam (for async)
# -----

class SharedAdam(torch.optim.Adam):
    def __init__(self, params, lr=1e-3, betas=(0.9, 0.999), eps=1e-8, weight_decay=0):
        super().__init__(params, lr=lr, betas=betas, eps=eps, weight_decay=weight_decay)
        for group in self.param_groups:
            for p in group['params']:
                state = self.state[p]
                state['step'] = torch.zeros(1)
                state['exp_avg'] = torch.zeros_like(p.data)
                state['exp_avg_sq'] = torch.zeros_like(p.data)
                # Share across processes
                state['exp_avg'].share_memory_()
                state['exp_avg_sq'].share_memory_()
                state['step'].share_memory_()

    def share_memory(self):
        # Kept for API symmetry; states are already shared in __init__
        pass

# -----
# Worker
# -----

class Worker(mp.Process):
    def __init__(self, wid: int, cfg: A3CConfig, global_net: ActorCritic,
                 optimizer: SharedAdam, global_counter: mp.Value, done_flag: mp.Value):
        super().__init__()
        self.wid = wid
        self.cfg = cfg
        self.global_net = global_net
        self.optimizer = optimizer
        self.global_counter = global_counter
        self.done_flag = done_flag
        self.local_net = None
        self.env = None

    def build_env_and_model(self):
        self.env = gym.make(self.cfg.env_id)
        obs_dim = self.env.observation_space.shape[0]
        act_dim = self.env.action_space.n
        self.local_net = ActorCritic(obs_dim, act_dim)

```

```

self.local_net.load_state_dict(self.global_net.state_dict())

def choose_action(self, logits: torch.Tensor):
    dist = torch.distributions.Categorical(logits=logits)
    action = dist.sample()
    logprob = dist.log_prob(action)
    entropy = dist.entropy()
    return action.item(), logprob, entropy

def compute_returns(self, rewards: List[float], dones: List[bool],
                    last_value: float, gamma: float) -> torch.Tensor:
    R = last_value
    returns = []
    for r, done in zip(reversed(rewards), reversed(dones)):
        if done:
            R = 0.0
        R = r + gamma * R
        returns.append(R)
    returns.reverse()
    return torch.tensor(returns, dtype=torch.float32)

def sync_local_with_global(self):
    self.local_net.load_state_dict(self.global_net.state_dict())

def run(self):
    set_seed_everywhere(self.cfg.seed + self.wid)
    self.build_env_and_model()

    obs, _ = self.env.reset(seed=self.cfg.seed + self.wid)
    obs = torch.tensor(obs, dtype=torch.float32)

    episode_reward = 0.0
    episode = 0

    while True:
        if self.done_flag.value == 1:
            break

        self.sync_local_with_global()

        values = []
        logprobs = []
        entropies = []
        rewards = []
        dones = []

        t_steps = 0
        while t_steps < self.cfg.t_max:
            logits, value = self.local_net(obs.unsqueeze(0))
            logits = logits.squeeze(0)
            value = value.squeeze(0)
            action, logprob, entropy = self.choose_action(logits)

            next_obs, reward, terminated, truncated, _ = self.env.step(action)
            done = terminated or truncated

            values.append(value)
            logprobs.append(logprob)
            entropies.append(entropy)
            rewards.append(float(reward))
            dones.append(done)

            episode_reward += reward

            with self.global_counter.get_lock():
                self.global_counter.value += 1
                cur_steps = self.global_counter.value

            if done:
                episode += 1
                next_obs, _ = self.env.reset()
                print(f"[Worker {self.wid}] Episode {episode} reward: {episode_reward:.1f} (global step {cur_steps})", flush=True)
                episode_reward = 0.0

            obs = torch.tensor(next_obs, dtype=torch.float32)

            t_steps += 1
            if cur_steps >= self.cfg.max_global_steps:
                self.done_flag.value = 1
                break

        with torch.no_grad():
            if dones[-1]:
                bootstrap_value = 0.0
            else:
                logits_b, value_b = self.local_net(obs.unsqueeze(0))
                bootstrap_value = value_b.squeeze(0).item()

        returns = self.compute_returns(rewards, dones, bootstrap_value, self.cfg.gamma)

        logprobs_t = torch.stack(logprobs)
        entropies_t = torch.stack(entropies)
        values_t = torch.stack(values)
        returns_t = returns

        advantages = returns_t - values_t.detach()
        policy_loss = -(logprobs_t * advantages).mean() - self.cfg.entropy_coef * entropies_t.mean()
        value_loss = self.cfg.value_loss_coef * (returns_t - values_t).pow(2).mean()
        loss = policy_loss + value_loss

        self.optimizer.zero_grad()
        loss.backward()
        torch.nn.utils.clip_grad_norm_(self.local_net.parameters(), self.cfg.grad_norm_clip)
        # Push local grads to global and step
        for lp, gp in zip(self.local_net.parameters(), self.global_net.parameters()):
            if gp.grad is None:
                gp.grad = lp.grad.clone()
            else:
                gp.grad.copy_(lp.grad)
        self.optimizer.step()

    self.env.close()

# -----
# Main
# -----

def build_parser():
    p = argparse.ArgumentParser()
    p.add_argument("--env", type=str, default="CartPole-v1")
    p.add_argument("--gamma", type=float, default=0.99)
    p.add_argument("--lr", type=float, default=1e-3)

```

```

p.add_argument("--workers", type=int, default=8)
p.add_argument("--t-max", type=int, default=5)
p.add_argument("--max-steps", type=int, default=200_000)
p.add_argument("--entropy-coef", type=float, default=0.01)
p.add_argument("--value-loss-coef", type=float, default=0.5)
p.add_argument("--grad-norm-clip", type=float, default=40.0)
p.add_argument("--seed", type=int, default=1)
return p

```

```

def make_config(args) -> A3CConfig:

```

```

    cfg = A3CConfig(
        env_id=args.env,
        gamma=args.gamma,
        lr=args.lr,
        workers=args.workers,
        t_max=args.t_max,
        max_global_steps=args.max_steps,
        entropy_coef=args.entropy_coef,
        value_loss_coef=args.value_loss_coef,
        grad_norm_clip=args.grad_norm_clip,
        seed=args.seed,
        device="cpu",
    )
    return cfg

```

```

def init_global_net(cfg: A3CConfig) -> ActorCritic:

```

```

    env = gym.make(cfg.env_id)
    obs_dim = env.observation_space.shape[0]
    act_dim = env.action_space.n
    env.close()
    net = ActorCritic(obs_dim, act_dim)
    net.share_memory()
    return net

```

```

def main():

```

```

    args = build_parser().parse_args()
    cfg = make_config(args)

```

```

    set_seed_everywhere(cfg.seed)
    mp.set_start_method("spawn", force=True)

```

```

    global_net = init_global_net(cfg)
    optimizer = SharedAdam(global_net.parameters(), lr=cfg.lr)
    optimizer.share_memory() # no-op but explicit

```

```

    global_counter = mp.Value('i', 0)
    done_flag = mp.Value('i', 0)

```

```

    workers = []
    for wid in range(cfg.workers):
        w = Worker(wid, cfg, global_net, optimizer, global_counter, done_flag)
        w.daemon = True
        workers.append(w)

```

```

    for w in workers:
        w.start()

```

```

    for w in workers:
        w.join()

```

```

    print("Training finished.")

```

```

if __name__ == "__main__":
    main()

```